Elliptic Curve Cryptography

Theory, Algorithms, Attacks and Protocols

Lecture Notes for MPhil / PhD Students

Five lectures of three hours each, with exercises and appendices

October 20, 2026

Preface and Course Plan

These notes accompany a graduate course on elliptic curve cryptography (ECC). The course consists of five lectures of about three hours each, and every chapter ends with exercises that serve as problem sheets. Throughout, we try to *build intuition first*: most definitions are followed by a picture or a small numerical example that you can check by hand. The notes balance three viewpoints, all of which a researcher in the area must master:

(i) **Mathematics:** the arithmetic geometry of elliptic curves (group law, torsion, Frobenius, divisors, pairings, isogenies).
(ii) **Algorithms:** finite field and curve arithmetic, scalar multiplication, point counting, discrete logarithm algorithms.
(iii) **Cryptography and engineering:** protocols (ECDH, ECDSA, EdDSA, BLS), standards, parameter choice, and implementation attacks.

Prerequisites. Undergraduate algebra (groups, rings, fields, finite fields), elementary number theory, and basic cryptography (what a public-key scheme and a digital signature are). Algebraic geometry is *not* assumed: the necessary facts (projective plane, divisors, Riemann–Roch) are introduced where needed and collected in the appendices.

Course plan.

Lecture	Content
1	Motivation, finite fields, Weierstrass curves, the group law, curves over $\mathbb{R}$, $\mathbb{C}$, $\mathbb{Q}$
2	Curves over finite fields: Hasse, structure, Frobenius, point counting, curve models, coordinates, scalar multiplication
3	The elliptic curve discrete logarithm problem: generic and special attacks, parameter selection
4	Protocols, standards, and implementation security (ECDH, ECDSA, EdDSA, side channels, invalid curves)
5	Divisors, pairings, pairing-based cryptography, isogenies, post-quantum perspectives

Conventions. Throughout, K is a field, $\overline{K}$ an algebraic closure, $q = p^n$ a prime power and $\mathbb{F}_q$ the finite field with q elements. The point at infinity of an elliptic curve is written $\mathcal{O}$. Sections marked with a star ($\star$) are optional or more advanced and can be skipped in a first pass. Boxes labelled *Key idea* summarise the central point of a section, and boxes labelled *Warning* flag common mistakes. All numerical examples in the text were verified by computer; a self-contained Python implementation is given in Lecture B.

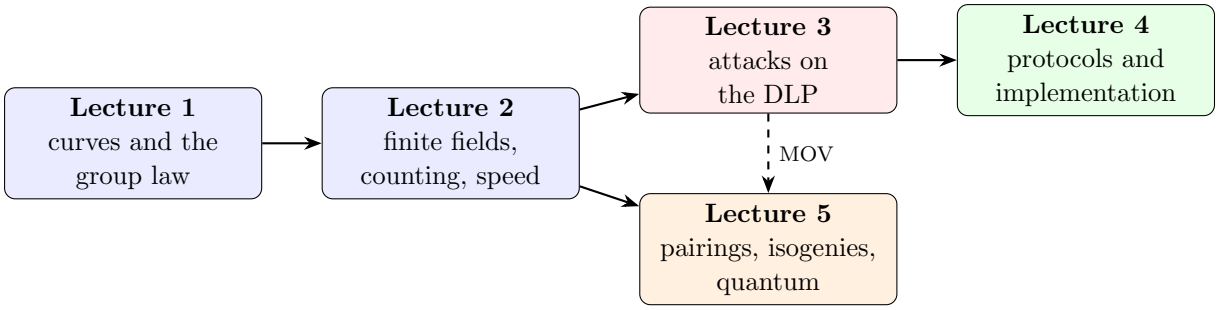


Figure 1: Roadmap. Lectures 1–2 build the mathematical and algorithmic toolbox, Lecture 3 explains why the problem is hard (and when it is not), Lecture 4 turns it into protocols, and Lecture 5 introduces the richer structure (pairings, isogenies) that gives new cryptography and new attacks.

Notation for complexity. $\mathbf{M}, \mathbf{S}, \mathbf{I}$ denote the cost of a field multiplication, squaring, and inversion respectively. $\mathcal{O}(\cdot)$ is the usual Landau symbol, $\widetilde{\mathcal{O}}(\cdot)$ hides polylogarithmic factors.

Contents

Preface and Course Plan **i**

1 Foundations: Motivation, Weierstrass Curves and the Group Law **1**

1.1 Why elliptic curves? A short history and motivation 1

1.1.1 Public-key cryptography from the discrete logarithm 1

1.1.2 Which group? 1

1.2 Algebraic preliminaries 2

1.2.1 Finite fields 2

1.2.2 Square roots and quadratic residues 3

1.2.3 Groups and the discrete logarithm 3

1.3 The projective plane and projective curves 4

1.4 Weierstrass equations 4

1.4.1 Short Weierstrass form 4

1.4.2 Elliptic curves over the reals 5

1.5 The chord-and-tangent group law 5

1.5.1 Explicit formulas 6

1.5.2 Elliptic curves over $\mathbb{Q}$: a glimpse 7

1.5.3 Example: a complete group over $\mathbb{F}_{17}$ 8

1.6 Elliptic curves over $\mathbb{C}$: the torus picture 8

1.7 Application: Lenstra's elliptic curve factoring method 9

Exercises for Lecture 1 10

2 Curves over Finite Fields: Structure, Counting and Arithmetic **12**

2.1 Counting points: the Hasse bound 12

2.1.1 The Frobenius endomorphism 12

2.2 Group structure and cryptographic subgroups 14

2.3 Counting points efficiently 15

2.3.1 Schoof's algorithm 15

2.4 Other curve models 16

2.4.1 Montgomery curves 16

2.4.2 Twisted Edwards curves . . . 16
2.5 Coordinate systems and the cost of a group operation . . . 18
2.6 Scalar multiplication . . . 18
2.6.1 Double-and-add . . . 18
2.6.2 Signed digits and windows . . . 19
2.6.3 The Montgomery ladder . . . 19
2.6.4 Endomorphism-accelerated multiplication (GLV/GLS) ⋆ . . . 20
Exercises for Lecture 2 . . . 21

3 The Elliptic Curve Discrete Logarithm Problem 22
3.1 Problems: ECDLP, CDH, DDH . . . 22
3.2 Generic algorithms . . . 23
3.2.1 Lower bound in the generic group model . . . 23
3.2.2 Baby-step giant-step (Shanks) . . . 23
3.2.3 Pohlig–Hellman . . . 23
3.2.4 Pollard's rho . . . 25
3.3 Why not index calculus? . . . 26
3.4 Special attacks on weak curves . . . 27
3.4.1 Anomalous curves (Smart; Satoh–Araki; Semaev) . . . 27
3.4.2 The MOV / Frey–Rück attack . . . 27
3.4.3 Other considerations: CM discriminant, twists, rigidity . . . 28
3.5 Putting it together: choosing a curve . . . 28
Exercises for Lecture 3 . . . 29

4 Protocols, Standards and Implementation Security 31
4.1 Key agreement: ECDH . . . 31
4.2 Public-key encryption: ElGamal and ECIES . . . 32
4.2.1 EC-ElGamal encryption, step by step . . . 32
4.2.2 ECIES / hashed ElGamal as a KEM . . . 38
4.3 Digital signatures . . . 39
4.3.1 ECDSA . . . 39
4.3.2 Nonce misuse: why k is the crown jewel . . . 39
4.3.3 Schnorr signatures and EdDSA . . . 40
4.4 Encoding of points and hashing to the curve . . . 41
4.5 Invalid-curve and small-subgroup attacks . . . 42
4.6 Side-channel and fault attacks . . . 43
4.7 Standards, deployment and the lesson of Dual_EC . . . 44
Exercises for Lecture 4 . . . 44

5 Divisors, Pairings, Isogenies and the Post-Quantum Horizon **46**

5.1 Functions and divisors on an elliptic curve 46

5.2 The Weil pairing 47

5.3 The Tate pairing and Miller's algorithm 47

5.3.1 Miller's algorithm 48

5.4 The MOV attack as a theorem 49

5.5 Pairing-based cryptography 49

5.5.1 Joux's tripartite Diffie–Hellman (2000) 49

5.5.2 BLS short signatures (Boneh–Lynn–Shacham, 2001) 50

5.5.3 Identity-based encryption (Boneh–Franklin, 2001) 50

5.5.4 Other applications 50

5.5.5 Pairing-friendly curves 50

5.6 Isogenies 51

5.6.1 Isogeny-based cryptography 52

5.7 Quantum computers and post-quantum perspectives 53

5.8 Research directions and open problems 54

Exercises for Lecture 5 54

A Proof of Associativity via Divisors **56**

B Self-Contained Python Code **57**

C SageMath Cheat Sheet **61**

D One-Page Formula Sheet **62**

Bibliography and Further Reading **63**

List of Figures

1 Roadmap. Lectures 1–2 build the mathematical and algorithmic toolbox, Lecture 3 explains why the problem is hard (and when it is not), Lecture 4 turns it into protocols, and Lecture 5 introduces the richer structure (pairings, isogenies) that gives new cryptography and new attacks. ii

1.1 The one-way behaviour that discrete-logarithm cryptography relies on. Computing kG from k is fast; recovering k from kG is believed to be infeasible in a well-chosen group. 2

1.2 Key sizes needed for a given security level: RSA/finite-field DLP grows (roughly) cubically in the security parameter, ECC grows linearly. 3

1.3 Cubic curves over $\mathbb{R}$. Left two: elliptic curves ($\Delta \neq 0$), with two connected components (f has three real roots) or one component (f has one real root). Right two: singular cubics ($\Delta = 0$), which are *not* elliptic curves. 5

1.4 Left: addition on $E : y^2 = x^3 - x + 1$ over $\mathbb{R}$. The line through $P = (0, 1)$ and $Q = (3, 5)$ meets E again at $R = (-\frac{11}{9}, -\frac{17}{27})$; reflecting R gives $P + Q = (-\frac{11}{9}, \frac{17}{27})$. Right: doubling Q via the tangent line; $2Q = (\frac{19}{25}, \frac{103}{125})$. 6

1.5 Special cases of the chord-and-tangent rule. (a) The line through $P = (x, y)$ and $-P = (x, -y)$ is vertical; it meets the curve only a third time "at infinity", so $P + (-P) = \mathcal{O}$. (b) On $y^2 = x^3 - x$ the three points T_i with $y = 0$ have vertical tangent lines, hence $2T_i = \mathcal{O}$: they are the points of order 2, and $E[2] = \{\mathcal{O}, T_1, T_2, T_3\} \cong (\mathbb{Z}/2)^2$. 6

1.6 The 18 affine points of $E : y^2 = x^3 + 2x + 2$ over $\mathbb{F}_{17}$ (the point $\mathcal{O}$ is not drawn). The group law has no visible geometric meaning in this picture: the structure is purely algebraic. The dashed line $y = 17/2$ is the axis of the negation symmetry. 9

1.7 The group of Theorem 1.21 as a "clock" with 19 positions. Position k holds the point kG (coordinates outside). Adding G rotates by one position, adding kG rotates by k positions; the point $-kG$ sits at the mirror position $19 - k$. Finding $\log_G Q$ means reading the position of Q on this clock – trivial here, impossible when the clock has 2^{256} positions. 10

1.8 A lattice $\Lambda \subset \mathbb{C}$ with fundamental parallelogram. Gluing opposite sides yields the complex torus $E(\mathbb{C})$; the nine red points are the 3-torsion points $E[3] \cong (\mathbb{Z}/3\mathbb{Z})^2$. 10

2.1 Hasse's theorem in action: the trace of Frobenius of the fixed curve $y^2 = x^3 + x + 1$ for all primes $p < 400$ (except $p = 31$, where the curve is singular). Every point lies in the strip $|t| \leq 2\sqrt{p}$, and the values fluctuate "randomly" inside it, as the random-walk heuristic predicts. 13

2.2 Real points of the Edwards curves $x^2+y^2 = 1+dx^2y^2$ for $d = 0, -\frac{1}{2}, -5$. The group law generalises the angle addition formulas of the circle ($d = 0$): $(\cos\theta_1, \sin\theta_1) + (\cos\theta_2, \sin\theta_2) = (\cos(\theta_1+\theta_2), \sin(\theta_1+\theta_2))$. 17

2.3 The three curve models and how they are related by birational maps. All three describe the *same* group; they differ in which formulas are fast and which implementation pitfalls they avoid. 17

3.1 Baby-step giant-step on the exponent line $0, \ldots, m^2-1$ ($m = 5$). The table covers the first m exponents; the giant steps (jumps of length m) are guaranteed to land in it after at most m jumps. Here $k = 17$ lands at $j = 2$ after $i = 3$ jumps, so $k = 3 \cdot 5 + 2$. 24

3.2 Pohlig–Hellman: the DLP is only as hard as the *largest prime factor* of the group order. If N is smooth, every sub-problem is tiny; if $N = r$ is prime, there is nothing to decompose. 24

3.3 The "ρ" shape of the sequence $x_{i+1} = \mathcal{W}(x_i)$ for a pseudo-random map $\mathcal{W}$ on a finite set. A collision $x_i = x_j$ with $i \neq j$ yields (via the stored coefficients) a linear relation between $\log_G Q$ and known integers. Here $\mu = 4$, $\lambda = 5$. 25

3.4 A schematic curve-selection procedure. If any security check fails, a parameter (for instance the coefficient A or b) is changed and the search restarts. 29

4.1 Elliptic-curve Diffie–Hellman. The key is derived from the x-coordinate of the shared point via a key-derivation function (KDF); the public values must be validated. 32

4.2 EC-ElGamal as ECDH plus masking. The point $C_1 = kG$ is Alice's ECDH public value; the shared point $S = kQ = dC_1$ is the ECDH secret; it is used to hide M by adding it. 33

4.3 ECDSA signing and verification. The nonce k must be fresh and secret for every signature (see Theorem 4.16); verification recomputes kG from public data only. 39

4.4 The Schnorr identification protocol. The Fiat–Shamir transform replaces the verifier's random challenge by $c = H(R \,\|\, Q \,\|\, m)$, which turns the protocol into a signature scheme. 40

4.5 The invalid-curve attack. Bob's group-law formulas work on *any* curve $y^2 = x^3 + ax + b'$, so a point from a weak curve E' makes him compute in a tiny group. 42

4.6 Idealised power traces (D = doubling, A = addition). In double-and-add the pattern "D A" vs. "D" spells out the key bits: here D A, D, D A, D A, i.e. bits $1, 0, 1, 1$. The ladder repeats "D, A" for every bit and reveals nothing by its shape. 43

5.1 A (Type-3) pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ of three groups of prime order r. E' is a degree-d twist of E allowing the second group to live over a smaller field $\mathbb{F}_{q^{k/d}}$. . 49

5.2 BLS signatures. The verification equation is bilinearity in action: the secret exponent x can be moved from one argument of the pairing to the other. 50

5.3 An isogeny of degree 2. It is a group homomorphism; every point of E_2 has exactly $\deg\phi$ preimages, forming a coset of the kernel. The dual isogeny goes back, and going there and back is multiplication by the degree. 51

5.4 Schematic 3-isogeny volcano of height 1 for ordinary curves: $\ell = 3$ and $\left(\frac{D}{\ell}\right) = 1$ (crater vertices have 2 neighbours in the crater and $\ell - 1 = 2$ below). 52

5.5 The commutative square behind isogeny key exchange (SIDH and CSIDH). Alice and Bob each apply a secret isogeny, exchange their curves (plus auxiliary information) and apply their secret again to the other's curve; both reach the same curve, the "shared secret" (its j-invariant). It is the analogue of $(g^a)^b = (g^b)^a$. . . 53

Chapter 1

Foundations: Motivation, Weierstrass Curves and the Group Law

Learning goals

By the end of this lecture students should be able to

- explain why discrete-logarithm cryptography needs groups in which the DLP is hard, and why elliptic curves are attractive;
- define an elliptic curve in Weierstrass form, compute discriminant and j-invariant, and recognise singular cubics;
- perform point addition and doubling geometrically and algebraically, over $\mathbb{R}$, $\mathbb{Q}$ and $\mathbb{F}_p$;
- state (and sketch the proof of) the group law theorem and relate $E(\mathbb{C})$ to a complex torus.

1.1 Why elliptic curves? A short history and motivation

1.1.1 Public-key cryptography from the discrete logarithm

In 1976 Diffie and Hellman introduced the idea that two parties can agree on a secret over a public channel. Their construction uses a finite cyclic group $G = \langle g \rangle$ of order n in which exponentiation $k \mapsto g^k$ is easy but its inverse is believed to be hard.

Definition 1.1 (Discrete logarithm problem)**.** Let $G = \langle g \rangle$ be a finite cyclic group of order n (written multiplicatively). The *discrete logarithm problem* (DLP) in G is: given $h \in G$, find the unique $x \in \{0, \dots, n-1\}$ with $g^x = h$. We write $x = \log_g h$.

The Diffie–Hellman protocol is: Alice picks a, sends $A = g^a$; Bob picks b, sends $B = g^b$; both compute $g^{ab} = B^a = A^b$. An eavesdropper sees (g, g^a, g^b) and must compute g^{ab} (the *computational Diffie–Hellman* problem, CDH). Clearly CDH $\leq$ DLP (if one can compute logarithms then one can solve CDH); the converse is open in general (see Section 3.1).

1.1.2 Which group?

The security of the scheme depends entirely on the group, *not* just on its size. Consider cyclic groups of order about 2^{256}:

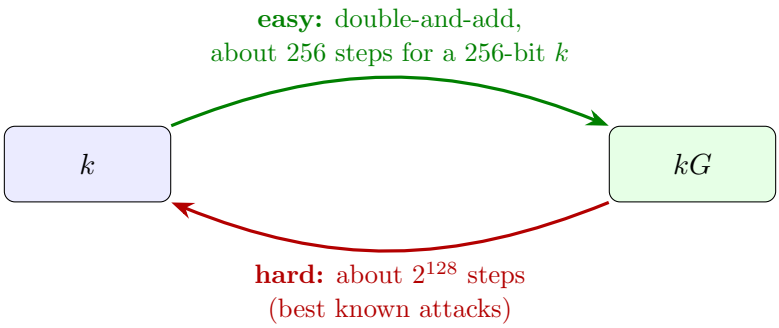


Figure 1.1: The one-way behaviour that discrete-logarithm cryptography relies on. Computing kG from k is fast; recovering k from kG is believed to be infeasible in a well-chosen group.

- In $G = (\mathbb{Z}/n\mathbb{Z}, +)$ the DLP is trivial: $x = h \cdot g^{-1} \bmod n$ via the extended Euclidean algorithm.
- In $G = \mathbb{F}_p^\times$ the DLP can be attacked by *index calculus* in subexponential time $L_p[1/3, c] = \exp\left((c + o(1))(\log p)^{1/3}(\log\log p)^{2/3}\right)$ (number field sieve). Consequently, to reach 128-bit security one needs p of about 3072 bits.
- In a group of points on a well-chosen elliptic curve over $\mathbb{F}_p$, no attack better than the *generic* $\mathcal{O}(\sqrt{n})$ algorithms (Pollard's rho) is known, so 256-bit curves already give about 128 bits of security.

This was the insight of Koblitz and Miller (independently, 1985–87): replace $\mathbb{F}_p^\times$ by $E(\mathbb{F}_p)$. Further, Lenstra (1987) used elliptic curves to factor integers (the ECM, see Section 1.7), establishing the "elliptic curve" as a general-purpose tool of computational number theory.

Table 1.1: Comparable security levels (NIST SP 800-57, approximate). "ECC" is the size of the group order in bits.

Security (bits)	Symmetric (e.g. AES)	RSA / DH modulus (bits)	ECC (bits)
80	2TDEA	1024	160–223
112	3TDEA	2048	224–255
128	AES-128	3072	256–383
192	AES-192	7680	384–511
256	AES-256	15360	512+

1.2 Algebraic preliminaries

1.2.1 Finite fields

Theorem 1.2 (Classification of finite fields)**.** *For every prime power $q = p^n$ there is a field $\mathbb{F}_q$ with q elements, unique up to isomorphism, and every finite field is of this form. The multiplicative group $\mathbb{F}_q^\times$ is cyclic of order $q - 1$. The map $\varphi : x \mapsto x^p$ is a field automorphism of $\mathbb{F}_q$ whose powers generate* $\mathrm{Gal}(\mathbb{F}_q/\mathbb{F}_p) \cong \mathbb{Z}/n\mathbb{Z}$.

Example 1.3 (Prime field $\mathbb{F}_7$ and extension field $\mathbb{F}_9$)**.** $\mathbb{F}_7 = \mathbb{Z}/7\mathbb{Z}$. Squares: $\{0, 1, 2, 4\}$, so $3, 5, 6$ are non-squares. $\mathbb{F}_9 = \mathbb{F}_3[i]/(i^2+1)$ since -1 is a non-square mod 3: elements $a+bi$ with $a, b \in \mathbb{F}_3$,

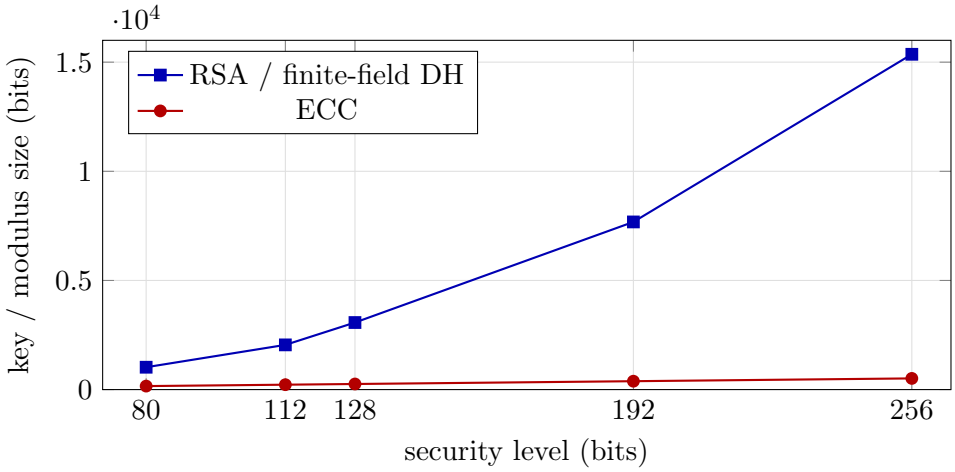


Figure 1.2: Key sizes needed for a given security level: RSA/finite-field DLP grows (roughly) cubically in the security parameter, ECC grows linearly.

e.g. $(1+i)^2 = 2i$, and $\mathbb{F}_9^\times = \langle 1+i \rangle$ is cyclic of order 8. Frobenius: $\varphi(a+bi) = a + b\,i^3 = a - bi$ (complex conjugation).

1.2.2 Square roots and quadratic residues

Elliptic-curve arithmetic requires solving $y^2 = f(x)$, so square roots in $\mathbb{F}_p$ matter (p odd).

Proposition 1.4 (Euler's criterion)**.** *For $a \in \mathbb{F}_p^\times$, $a^{(p-1)/2} = \left(\frac{a}{p}\right) \in \{\pm 1\}$, and a is a square iff this equals 1. Exactly half of $\mathbb{F}_p^\times$ consists of squares.*

Proposition 1.5 (Square roots)**.** *If $p \equiv 3 \pmod 4$ and a is a square, then $\sqrt{a} = \pm a^{(p+1)/4}$. If $p \equiv 1 \pmod 4$, one uses the Tonelli–Shanks algorithm (expected polynomial time) or Cipolla's algorithm.*

Proof. $(a^{(p+1)/4})^2 = a^{(p+1)/2} = a \cdot a^{(p-1)/2} = a$ by Euler's criterion. □

Example 1.6 (Square roots modulo 23)**.** $23 \equiv 3 \pmod 4$. Is 2 a square? $2^{(23-1)/2} = 2^{11} = 2048 \equiv 1 \pmod{23}$, so yes. Then $\sqrt{2} = \pm 2^{(23+1)/4} = \pm 2^6 = \pm 64 \equiv \pm 18$. Check: $18^2 = 324 = 14 \cdot 23 + 2$. ✓ (Note $18 \equiv -5$, and indeed $5^2 = 25 \equiv 2$.) By contrast $5^{11} \equiv -1$, so 5 is a non-square, and the equation $y^2 = 5$ has no solution.

Remark 1.7. Curve25519 uses $p = 2^{255} - 19 \equiv 1 \pmod 4$ ($\equiv 5 \pmod 8$), so square roots are computed via Atkin's variant with one exponentiation and possibly one multiplication by $\sqrt{-1}$. The curve secp256k1 has $p \equiv 3 \pmod 4$.

1.2.3 Groups and the discrete logarithm

We recall: for P in a group G of order N, $\mathrm{ord}(P)$ divides N (Lagrange); a finite abelian group is a product of cyclic groups (structure theorem); G of prime order is cyclic and each non-identity element is a generator. Hardness of DLP in G will be measured by the running time of the best-known algorithm as a function of the size of G (Lecture 3).

1.3 The projective plane and projective curves

Definition 1.8 (Projective plane)**.** $\mathbb{P}^2(K) = \big(K^3 \setminus \{0\}\big)/\sim$ where $(X,Y,Z) \sim (\lambda X, \lambda Y, \lambda Z)$ for $\lambda \in K^\times$. The class is written $[X : Y : Z]$. The *affine plane* $\mathbb{A}^2(K)$ embeds as $(x,y) \mapsto [x : y : 1]$; the complement $\{Z = 0\}$ is the *line at infinity*.

A polynomial $F(X,Y,Z)$ that is *homogeneous* of degree d has a well-defined zero set in $\mathbb{P}^2$. Given an affine equation $f(x,y) = 0$ of degree d we homogenise $F(X,Y,Z) = Z^d f(X/Z, Y/Z)$. For example

$$y^2 = x^3 + ax + b \quad \rightsquigarrow \quad Y^2Z = X^3 + aXZ^2 + bZ^3.$$

Setting $Z = 0$ gives $X^3 = 0$, i.e. exactly one point at infinity, $\mathcal{O} = [0 : 1 : 0]$. This is the identity element of the group law.

Theorem 1.9 (Bézout, weak form for a line and a cubic)**.** *A line $L \subset \mathbb{P}^2$ not contained in a plane cubic C meets C in exactly three points of $\mathbb{P}^2(\overline{K})$, counted with multiplicity.*

Idea. Parametrise L by $[s : t] \mapsto [X(s,t) : Y(s,t) : Z(s,t)]$ with linear forms. Then $F|_L$ is a nonzero binary cubic form in (s,t), which has exactly 3 roots in $\mathbb{P}^1(\overline{K})$ with multiplicity. The multiplicity is the *intersection multiplicity*. □

This theorem is the geometric heart of the group law: a line through two points of a cubic meets it in a *third* point.

1.4 Weierstrass equations

Definition 1.10 (Elliptic curve)**.** An *elliptic curve over K* is a smooth projective curve E of genus 1 together with a point $\mathcal{O} \in E(K)$. Every such curve is isomorphic to a curve given by a *(generalised) Weierstrass equation*

$$E:\ y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6, \qquad a_i \in K, \tag{1.1}$$

with $\mathcal{O} = [0 : 1 : 0]$ the unique point at infinity, *and nonzero discriminant* (below).

The *discriminant* is $\Delta = -b_2^2b_8 - 8b_4^3 - 27b_6^2 + 9b_2b_4b_6$ where

$$b_2 = a_1^2 + 4a_2, \qquad b_4 = 2a_4 + a_1a_3, \qquad b_6 = a_3^2 + 4a_6,$$
$$b_8 = a_1^2a_6 + 4a_2a_6 - a_1a_3a_4 + a_2a_3^2 - a_4^2,$$
$$c_4 = b_2^2 - 24b_4, \qquad j = c_4^3/\Delta.$$

The quantity $j = j(E)$ is the *j-invariant*. The curve is nonsingular iff $\Delta \neq 0$.

1.4.1 Short Weierstrass form

If $\operatorname{char} K \neq 2, 3$ the substitution $y \mapsto y - \frac{1}{2}(a_1x + a_3)$ followed by $x \mapsto x - \frac{b_2}{12}$ transforms (1.1) into

$$E:\ y^2 = x^3 + ax + b, \qquad a, b \in K, \tag{1.2}$$

with

$$\Delta = -16(4a^3 + 27b^2), \qquad j = 1728 \cdot \frac{4a^3}{4a^3 + 27b^2}. \tag{1.3}$$

This is the form used by NIST P-curves and secp256k1 (for which $a = 0$, $b = 7$).

Theorem 1.11 (Nonsingularity)**.** *Let* $\operatorname{char} K \neq 2, 3$ *and* $f(x) = x^3 + ax + b$*. The following are equivalent:*

(a) the affine curve $y^2 = f(x)$ *has no singular point over* $\overline{K}$ *(and the point* $\mathcal{O}$ *is nonsingular);*
(b) f *has three distinct roots in* $\overline{K}$*;*
(c) $4a^3 + 27b^2 \neq 0$*.*

Proof. (a)⇔(b): put $F(x, y) = y^2 - f(x)$. A singular point satisfies $F_y = 2y = 0$ and $F_x = -f'(x) = 0$ and lies on the curve, so $y = 0$, $f(x) = 0 = f'(x)$: x is a repeated root of f. Conversely a repeated root x_0 gives the singular point $(x_0, 0)$. At $\mathcal{O} = [0 : 1 : 0]$ set $y = 1$ in $Y^2Z = X^3 + aXZ^2 + bZ^3$: $G(X, Z) = Z - X^3 - aXZ^2 - bZ^3$ has $\partial_Z G(0, 0) = 1 \neq 0$, hence $\mathcal{O}$ is smooth. (b)⇔(c): the discriminant of $x^3 + ax + b$ equals $-4a^3 - 27b^2$ (Exercise 1.1) and vanishes iff f has a repeated root. □

Example 1.12 (Testing curves for singularity, and computing j)**.**

- $y^2 = x^3 - 3x + 2$: $4a^3 + 27b^2 = 4(-27) + 27 \cdot 4 = 0$. Indeed $x^3 - 3x + 2 = (x - 1)^2(x + 2)$ has a double root, so the point $(1, 0)$ is singular (a node). This is the last curve of Figure 1.3: *not* an elliptic curve.
- $y^2 = x^3 + 2x + 2$ over $\mathbb{F}_{17}$: $4a^3 + 27b^2 = 32 + 108 = 140 \equiv 4 \neq 0$: nonsingular. Its j-invariant is $j = 1728 \cdot \frac{4a^3}{4a^3 + 27b^2} = 1728 \cdot \frac{32}{140} \equiv 11 \cdot \frac{15}{4} = 11 \cdot 15 \cdot 13 \equiv 3 \pmod{17}$.
- $y^2 = x^3 + 7$ (secp256k1): $a = 0$ so $j = 0$. $y^2 = x^3 - x$: $b = 0$ so $j = 1728$.

Example 1.13 (Isomorphism classes)**.** Over $\overline{K}$, $E_{a,b} \cong E_{a',b'}$ iff $j(E_{a,b}) = j(E_{a',b'})$. Over K, the curves are isomorphic iff there is $u \in K^\times$ with $a' = u^4 a$ and $b' = u^6 b$. Thus $y^2 = x^3 + b$ ($j = 0$) gives $|K^\times/(K^\times)^6|$ isomorphism classes over K (twists), and $y^2 = x^3 + ax$ ($j = 1728$) gives $|K^\times/(K^\times)^4|$ classes. secp256k1 has $j = 0$, which is why it admits an efficient endomorphism $(x, y) \mapsto (\beta x, y)$, $\beta^3 = 1$ (used for speed-ups, see Section 2.6.4).

Remark 1.14 (Characteristic 2 and 3). In characteristic 2 (binary fields $\mathbb{F}_{2^m}$, historically popular, e.g. NIST B- and K-curves) the non-supersingular curves are $y^2 + xy = x^3 + a_2x^2 + a_6$ with $\Delta = a_6 \neq 0$. In characteristic 3 one uses $y^2 = x^3 + a_2x^2 + a_4x + a_6$. We will mostly treat $\operatorname{char} K > 3$.

1.4.2 Elliptic curves over the reals

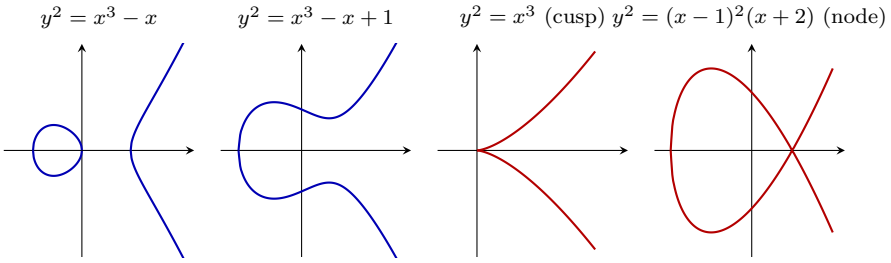


Figure 1.3: Cubic curves over $\mathbb{R}$. Left two: elliptic curves ($\Delta \neq 0$), with two connected components (f has three real roots) or one component (f has one real root). Right two: singular cubics ($\Delta = 0$), which are *not* elliptic curves.

1.5 The chord-and-tangent group law

Fix $E : y^2 = x^3 + ax + b$ with $\Delta \neq 0$ over a field K of characteristic $\neq 2, 3$. Write $E(K) = \{(x, y) \in K^2 : y^2 = x^3 + ax + b\} \cup \{\mathcal{O}\}$.

Definition 1.15 (Addition of points)**.** Let $P, Q \in E(K)$. Let L be the line through P and Q (the tangent line at P if $P = Q$; the line through $\mathcal{O}$ and P is the vertical line $x = x_P$). By Theorem 1.9, L meets E in a third point $R \in E(K)$ (with multiplicities). Let L' be the line through R and $\mathcal{O}$ (vertical line); it meets E in a third point which we call

$$P + Q.$$

Equivalently: $P + Q$ is the reflection of R in the x-axis. If $P = \mathcal{O}$ define $\mathcal{O} + Q = Q$.

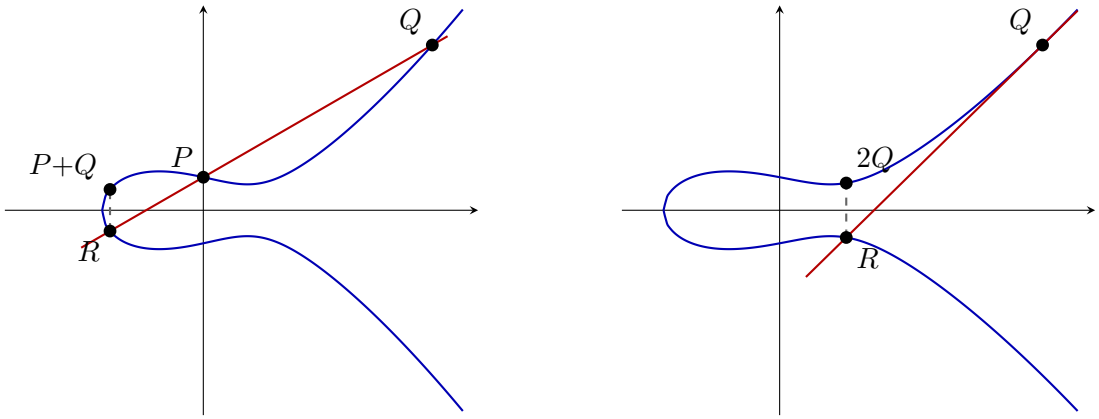
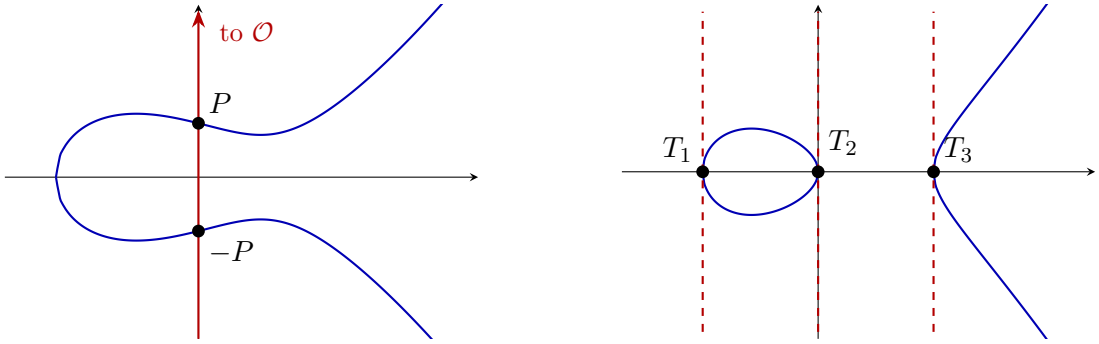


Figure 1.4: Left: addition on $E : y^2 = x^3 - x + 1$ over $\mathbb{R}$. The line through $P = (0, 1)$ and $Q = (3, 5)$ meets E again at $R = (-\frac{11}{9}, -\frac{17}{27})$; reflecting R gives $P + Q = (-\frac{11}{9}, \frac{17}{27})$. Right: doubling Q via the tangent line; $2Q = (\frac{19}{25}, \frac{103}{125})$.



(a) vertical line: $P + (-P) = \mathcal{O}$ (b) vertical tangents: $2T_i = \mathcal{O}$

Figure 1.5: Special cases of the chord-and-tangent rule. (a) The line through $P = (x, y)$ and $-P = (x, -y)$ is vertical; it meets the curve only a third time "at infinity", so $P + (-P) = \mathcal{O}$. (b) On $y^2 = x^3 - x$ the three points T_i with $y = 0$ have vertical tangent lines, hence $2T_i = \mathcal{O}$: they are the points of order 2, and $E[2] = \{\mathcal{O}, T_1, T_2, T_3\} \cong (\mathbb{Z}/2)^2$.

1.5.1 Explicit formulas

Theorem 1.16 (Addition formulas)**.** *Let $P_1 = (x_1, y_1)$, $P_2 = (x_2, y_2) \in E(K) \setminus \{\mathcal{O}\}$.*

(i) If $x_1 = x_2$ and $y_1 = -y_2$ then $P_1 + P_2 = \mathcal{O}$. In particular $-(x, y) = (x, -y)$.

(ii) Otherwise, define the slope

$$\lambda = \begin{cases} \dfrac{y_2 - y_1}{x_2 - x_1} & \text{if } P_1 \neq P_2, \\ \dfrac{3x_1^2 + a}{2y_1} & \text{if } P_1 = P_2, \end{cases}$$

then $P_1 + P_2 = (x_3, y_3)$ *with*

$$x_3 = \lambda^2 - x_1 - x_2, \qquad y_3 = \lambda(x_1 - x_3) - y_1. \tag{1.4}$$

Proof. (i) The line through P_1, P_2 is vertical, so its third intersection with E is $\mathcal{O}$ and the reflection of $\mathcal{O}$ is $\mathcal{O}$. (ii) The line L is $y = \lambda x + \nu$ with $\nu = y_1 - \lambda x_1$; for $P_1 = P_2$ the slope is obtained by implicit differentiation of $y^2 = x^3 + ax + b$: $2y\,y' = 3x^2 + a$ (note $y_1 \neq 0$ here, since otherwise the tangent is vertical and $2P_1 = \mathcal{O}$ by (i)). Substituting into the curve equation,

$$(\lambda x + \nu)^2 = x^3 + ax + b \iff x^3 - \lambda^2 x^2 + (a - 2\lambda\nu)x + (b - \nu^2) = 0.$$

This cubic has roots x_1, x_2 and the x-coordinate x_R of the third intersection point R; the coefficient of x^2 gives $x_1 + x_2 + x_R = \lambda^2$. Hence $x_R = \lambda^2 - x_1 - x_2$ and $R = (x_R, \lambda x_R + \nu)$. Reflecting, $P_1 + P_2 = (x_R, -\lambda x_R - \nu) = (x_3, \lambda(x_1 - x_3) - y_1)$. □

Theorem 1.17 (Group law)**.** $(E(K), +, \mathcal{O})$ *is an abelian group.*

Proof (outline). Identity: by definition. Inverses: $-P$ is the reflection of P. Commutativity: the line through P and Q is symmetric in P, Q. The nontrivial point is *associativity*. There are three standard proofs: (1) a brute-force computation with the formulas (1.4) and case distinction (rigorous but tedious; can be done by a computer algebra system); (2) the Cayley–Bácharach theorem for cubics; (3) identification of $(E, +)$ with the divisor class group $\mathrm{Pic}^0(E)$ so that associativity is inherited from addition of divisors. We prove it via (3) in Lecture A. □

Key idea

Three points $P, Q, R \in E$ are collinear iff $P + Q + R = \mathcal{O}$. This is the one rule that generates everything: addition is "find the third point of the line, then negate".

Example 1.18 (Arithmetic over $\mathbb{Q}$)**.** On $E : y^2 = x^3 - x + 1$ take $P = (0, 1)$ and $Q = (3, 5)$ (indeed $27 - 3 + 1 = 25 = 5^2$). Then $\lambda = \frac{5-1}{3-0} = \frac{4}{3}$ and

$$x_3 = \tfrac{16}{9} - 0 - 3 = -\tfrac{11}{9}, \qquad y_3 = \tfrac{4}{3}\big(0 + \tfrac{11}{9}\big) - 1 = \tfrac{44}{27} - 1 = \tfrac{17}{27}.$$

Check: $\big(-\frac{11}{9}\big)^3 - \big(-\frac{11}{9}\big) + 1 = \frac{289}{729} = \big(\frac{17}{27}\big)^2$. ✓
Doubling Q: $\lambda = \frac{3\cdot 9 - 1}{2\cdot 5} = \frac{13}{5}$, $x_3 = \frac{169}{25} - 6 = \frac{19}{25}$, $y_3 = \frac{13}{5}\big(3 - \frac{19}{25}\big) - 5 = \frac{728}{125} - 5 = \frac{103}{125}$; so $2Q = \big(\frac{19}{25}, \frac{103}{125}\big)$ (this is Figure 1.4). Note the growth of the heights of the coordinates: the denominators of nP grow like n^2 in bit-size. This is why rational points are not used in cryptography, and why we pass to finite fields.

1.5.2 Elliptic curves over $\mathbb{Q}$: a glimpse

Theorem 1.19 (Mordell–Weil)**.** *For $E/\mathbb{Q}$ the group $E(\mathbb{Q})$ is finitely generated: $E(\mathbb{Q}) \cong \mathbb{Z}^r \oplus E(\mathbb{Q})_{\mathrm{tors}}$.*

Theorem 1.20 (Mazur, 1977)**.** *$E(\mathbb{Q})_{\mathrm{tors}}$ is isomorphic to one of $\mathbb{Z}/N$ ($1 \le N \le 10$ or $N = 12$) or $\mathbb{Z}/2 \times \mathbb{Z}/2N$ ($1 \le N \le 4$).*

The rank r is the subject of the Birch and Swinnerton-Dyer conjecture; no algorithm to compute it is known to terminate. These results are mentioned for culture; they explain why cryptography works over *finite* fields, where everything is finite and explicitly computable.

1.5.3 Example: a complete group over $\mathbb{F}_{17}$

Example 1.21 (The curve $y^2 = x^3 + 2x + 2$ over $\mathbb{F}_{17}$)**.** Here $\Delta = -16(4\cdot 8 + 27\cdot 4) = -16\cdot 140$, and $140 \equiv 4 \pmod{17}$ is nonzero, so E is nonsingular. The curve has exactly 19 points (18 affine ones and $\mathcal{O}$); since 19 is prime the group is cyclic and every point other than $\mathcal{O}$ is a generator. Take $G = (5,1)$. First double and add:

$$\begin{aligned} 2G:\ \lambda &= \tfrac{3\cdot 25+2}{2\cdot 1} = \tfrac{77}{2} \equiv 9\cdot 9 = 81 \equiv 13, \\ x_3 &= 169 - 10 = 159 \equiv 6, \quad y_3 = 13(5-6) - 1 = -14 \equiv 3 \ \Rightarrow\ 2G = (6,3). \\ 3G = G + 2G:\ \lambda &= \tfrac{3-1}{6-5} = 2,\ x_3 = 4 - 5 - 6 = -7 \equiv 10, \\ y_3 &= 2(5-10) - 1 = -11 \equiv 6 \ \Rightarrow\ 3G = (10,6). \end{aligned}$$

The whole group is listed in Table 1.2; Figure 1.6 shows the point set (the "scatter" is symmetric about the line $y = 17/2$ because $-(x,y) = (x, 17-y)$).

Table 1.2: Multiples kG for $G = (5,1)$ on $y^2 = x^3 + 2x + 2$ over $\mathbb{F}_{17}$.

k	1	2	3	4	5	6	7	8	9
kG	$(5,1)$	$(6,3)$	$(10,6)$	$(3,1)$	$(9,16)$	$(16,13)$	$(0,6)$	$(13,7)$	$(7,6)$
k	10	11	12	13	14	15	16	17	18
kG	$(7,11)$	$(13,10)$	$(0,11)$	$(16,4)$	$(9,1)$	$(3,16)$	$(10,11)$	$(6,14)$	$(5,16)$

and $19G = \mathcal{O}$. Observe $kG = -(19-k)G$: for instance $18G = (5,16) = -G$.

1.6 Elliptic curves over $\mathbb{C}$: the torus picture

Theorem 1.22 (Uniformisation)**.** *Let $\Lambda = \mathbb{Z}\omega_1 + \mathbb{Z}\omega_2 \subset \mathbb{C}$ be a lattice (ω_1, ω_2 linearly independent over $\mathbb{R}$). Define the Weierstrass function*

$$\wp(z) = \frac{1}{z^2} + \sum_{\omega \in \Lambda\setminus 0} \Big(\frac{1}{(z-\omega)^2} - \frac{1}{\omega^2} \Big),$$

$$g_2 = 60 \sum_{\omega \neq 0} \omega^{-4}, \qquad g_3 = 140 \sum_{\omega \neq 0} \omega^{-6}.$$

Then $\wp'(z)^2 = 4\wp(z)^3 - g_2\wp(z) - g_3$ with $g_2^3 - 27g_3^2 \neq 0$, and the map

$$\Phi : \mathbb{C}/\Lambda \longrightarrow E_\Lambda(\mathbb{C}) :\ y^2 = 4x^3 - g_2 x - g_3, \qquad z \mapsto (\wp(z), \wp'(z)),\quad 0 \mapsto \mathcal{O}$$

is an isomorphism of complex Lie groups. Conversely every elliptic curve over $\mathbb{C}$ arises in this way.

Thus topologically $E(\mathbb{C}) \cong \mathbb{R}^2/\mathbb{Z}^2$ is a *torus*, and the geometric group law is just addition modulo the lattice. The substitution $(x,y) \mapsto (x, y/2)$ turns $y^2 = 4x^3 - g_2x - g_3$ into short Weierstrass form.

Corollary 1.23 (Torsion)**.** *For every $n \geq 1$, $E(\mathbb{C})[n] = \{P : nP = \mathcal{O}\} \cong \frac{1}{n}\Lambda/\Lambda \cong (\mathbb{Z}/n\mathbb{Z})^2$.*

This has a deep consequence: over *any* algebraically closed field $\overline{K}$ of characteristic $p \geq 0$, $E[n] \cong (\mathbb{Z}/n)^2$ whenever $p \nmid n$ (a fact proved algebraically via the degree of multiplication-by-n, Theorem 2.4).

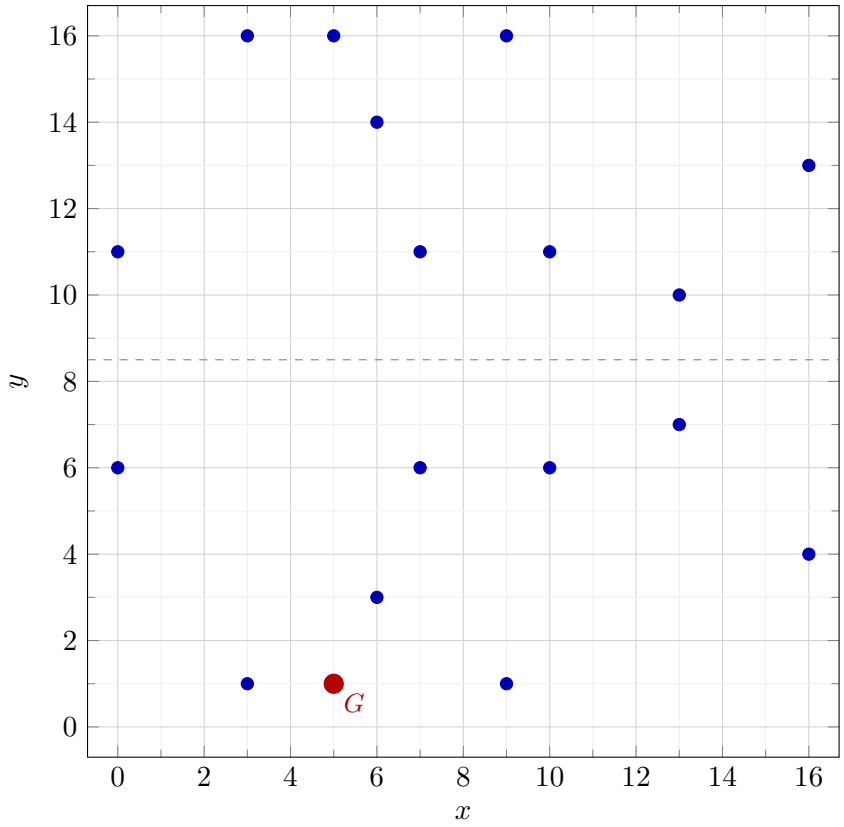


Figure 1.6: The 18 affine points of $E : y^2 = x^3 + 2x + 2$ over $\mathbb{F}_{17}$ (the point $\mathcal{O}$ is not drawn). The group law has no visible geometric meaning in this picture: the structure is purely algebraic. The dashed line $y = 17/2$ is the axis of the negation symmetry.

Key idea

The "$(\mathbb{Z}/n)^2$ shape" of the torsion is the single most important structural property of elliptic curves: it will yield Frobenius characteristic polynomials (Lecture 2), the Weil pairing and the MOV attack (Lectures 3, 5), and isogeny graphs (Lecture 5).

1.7 Application: Lenstra's elliptic curve factoring method

We end with a computation that uses the group law over a *ring* $\mathbb{Z}/N\mathbb{Z}$ and shows how cleverly one can exploit the failure of the inversion.

Pollard's $p-1$ method factors $N = pq$ when $p-1$ is smooth, because $x^{B!} \equiv 1 \pmod{p}$ then, whereas typically $\not\equiv 1 \pmod{q}$. It works in the fixed group $\mathbb{F}_p^\times$. Lenstra's insight: use the group $E(\mathbb{F}_p)$ instead, whose order $\#E(\mathbb{F}_p) \in [p+1-2\sqrt{p},\, p+1+2\sqrt{p}]$ varies with the curve; if one fails, *change the curve*.

Example 1.24. Let $N = 455839$, $E : y^2 = x^3 + 5x - 5$, $P = (1, 1)$. Computing successively $2P, 3P, \ldots$ and then $kP \leftarrow k \cdot P$ for $k = 2, \ldots, 6$ gives $720P = 6!\,P = (422839, 402728) \pmod{N}$. Then computing $7 \cdot (6!P)$ requires the inversion of a number d with $\gcd(d, N) = 599$. Hence $599 \mid N$ and $N = 599 \cdot 761$. The explanation: the order of P modulo 599 divides $7! = 5040$ (it is 7-smooth), whereas modulo 761 it is not.

Theorem 1.25 (Heuristic complexity of ECM)**.** *Assuming that $\#E(\mathbb{F}_p)$ behaves like a random*

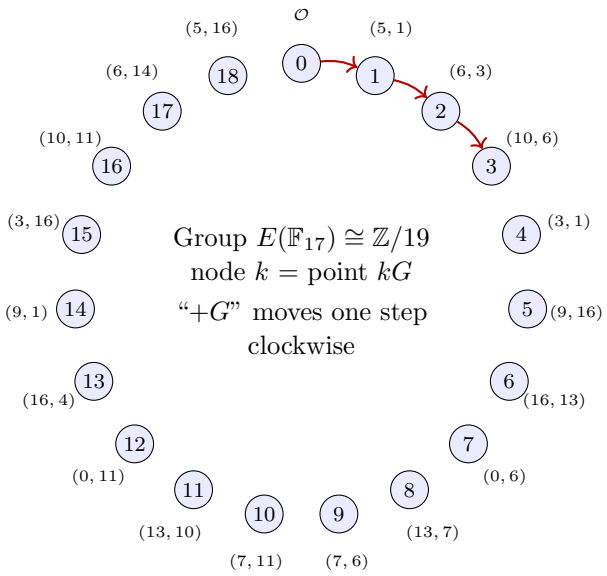


Figure 1.7: The group of Theorem 1.21 as a "clock" with 19 positions. Position k holds the point kG (coordinates outside). Adding G rotates by one position, adding kG rotates by k positions; the point $-kG$ sits at the mirror position $19 - k$. Finding $\log_G Q$ means reading the position of Q on this clock – trivial here, impossible when the clock has 2^{256} positions.

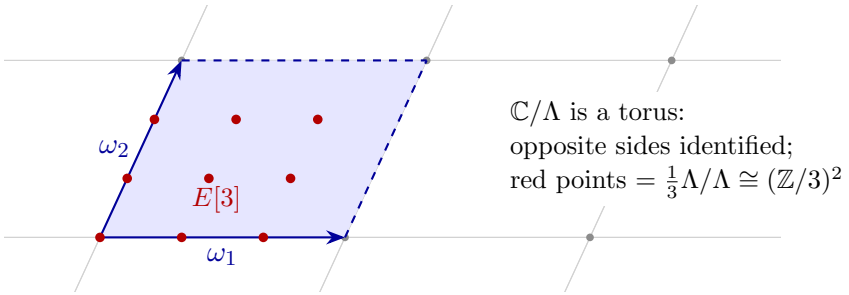


Figure 1.8: A lattice $\Lambda \subset \mathbb{C}$ with fundamental parallelogram. Gluing opposite sides yields the complex torus $E(\mathbb{C})$; the nine red points are the 3-torsion points $E[3] \cong (\mathbb{Z}/3\mathbb{Z})^2$.

integer in the Hasse interval, ECM finds a prime factor p of N in expected time $L_p[1/2, \sqrt{2} + o(1)] \cdot \mathrm{M}(\log N)$, where M *denotes multiplication cost.*

ECM remains the algorithm of choice for finding factors up to roughly 60 digits and is used inside the number field sieve ecosystem. Its "dual" application to cryptography is the rest of this course.

Exercises for Lecture 1

Exercise 1.1 (Discriminant). Let $f(x) = x^3 + ax + b$ with roots $r_1, r_2, r_3 \in \overline{K}$. Show that $\prod_{i<j}(r_i - r_j)^2 = -4a^3 - 27b^2$. *Hint:* $\prod_{i<j}(r_i - r_j)^2 = -\prod_i f'(r_i)$ and $f'(r_i) = 3r_i^2 + a$; use $r_i^3 = -ar_i - b$ to write $f'(r_i) = -(2ar_i + 3b)/r_i$.

Exercise 1.2. Prove that $E : y^2 = x^3 + ax + b$ over a field of characteristic $\neq 2, 3$ has 2-torsion $E[2] = \{\mathcal{O}\} \cup \{(r, 0) : f(r) = 0\}$ and deduce $E[2] \cong (\mathbb{Z}/2)^2$ over $\overline{K}$. Which points of $E(\mathbb{F}_{17})$ in Theorem 1.21 would be 2-torsion points? Why does $E(\mathbb{F}_{17})$ have odd order?

Algorithm 1 Elliptic Curve Method (one curve)

Input: composite N (coprime to 6), smoothness bound B.

1: pick random $a, x_0, y_0 \in \mathbb{Z}/N$; set $b = y_0^2 - x_0^3 - ax_0$ and $P = (x_0, y_0)$ on $E : y^2 = x^3 + ax + b$ over $\mathbb{Z}/N$.
2: **for** $k = 2, 3, \ldots, B$ **do**
3: $P \leftarrow kP$ using the affine formulas over $\mathbb{Z}/N$ (compute inverses with the extended Euclidean algorithm).
4: **if** some inversion of d fails, i.e. $g = \gcd(d, N) \neq 1$ **then return** g (a factor if $g \neq N$).
5: **end if**
6: **end for**
7: **return** "failure" (choose another curve).

Exercise 1.3 (Verification of associativity in a small case). In Theorem 1.21, verify $(G+2G)+4G = G + (2G + 4G)$ by direct computation.

Exercise 1.4 (Singular cubics). Let $C : y^2 = x^3$ over a field K. Show that $C_{\text{ns}} = C \setminus \{(0, 0)\}$ with the chord–tangent law is isomorphic to $(K, +)$ via $(x, y) \mapsto x/y$. For the nodal cubic $y^2 = x^2(x + 1)$ show that the non-singular points form a group isomorphic to $K^\times$ (when the tangent slopes at the node are in K) via $t = \frac{y+x}{y-x}$. *Remark:* this explains why singular parameters make the DLP easy.

Exercise 1.5 (j-invariant and twists). (a) Compute $j(y^2 = x^3 + 7)$ and $j(y^2 = x^3 - 3x + b)$. (b) Show that $E_{a,b}$ and E_{au^4,bu^6} are isomorphic over K via $(x, y) \mapsto (u^2x, u^3y)$. (c) For $d \in K^\times$ non-square show that $E^{(d)} : dy^2 = x^3 + ax + b$ is isomorphic over $K(\sqrt{d})$ but not over K in general to E (the *quadratic twist*), and that $E^{(d)} \cong E_{ad^2,bd^3}$.

Exercise 1.6 (Rational points). Show that $P = (2, 3)$ lies on $y^2 = x^3 + 1$ and compute $2P, 3P, \ldots, 6P$; deduce that P is a torsion point of order 6. (Compare with Mazur's theorem.)

Exercise 1.7 (ECM by hand). Using Theorem 1.24 as a template, factor $N = 1649$ with $y^2 = x^3 + x + b$ and $P = (1, 1)$ (so $b = -1$). Use the extended Euclidean algorithm for the inversions.

Exercise 1.8 (Challenge). Prove that $E(K)$ is closed under the operations of Theorem 1.16, i.e. the point (x_3, y_3) is again on E. (Substitute and use that x_1, x_2, x_3 are the three roots of the cubic in the proof.)

Chapter 2

Curves over Finite Fields: Structure, Counting and Arithmetic

Learning goals

After this lecture students should be able to

- state and use Hasse's theorem, the Frobenius endomorphism and the structure theorem for $E(\mathbb{F}_q)$;
- explain Schoof's point-counting algorithm and why $\#E(\mathbb{F}_q)$ can be computed in polynomial time;
- convert between Weierstrass, Montgomery and (twisted) Edwards models and know where each is used (Curve25519, Ed25519);
- implement efficient and (conceptually) constant-time scalar multiplication: double-and-add, NAF, windows, Montgomery ladder.

2.1 Counting points: the Hasse bound

Let $q = p^n$, $p > 3$, and $E : y^2 = f(x) = x^3 + ax + b$ over $\mathbb{F}_q$. For each $x \in \mathbb{F}_q$, the equation $y^2 = f(x)$ has $1 + \chi(f(x))$ solutions, where χ is the quadratic character of $\mathbb{F}_q$ ($\chi(0) = 0$). Hence

$$\#E(\mathbb{F}_q) = 1 + \sum_{x \in \mathbb{F}_q} \big(1 + \chi(f(x))\big) = q + 1 + \sum_{x \in \mathbb{F}_q} \chi(f(x)). \tag{2.1}$$

Heuristically, $\chi(f(x))$ is a "random sign" so the sum is of size $\sqrt{q}$ (a random walk with q steps). This is exactly the content of Hasse's theorem.

Theorem 2.1 (Hasse, 1933). $\big|\#E(\mathbb{F}_q) - (q+1)\big| \le 2\sqrt{q}$. *We write* $\#E(\mathbb{F}_q) = q + 1 - t$ *and call* t *the* trace of Frobenius*; thus* $|t| \le 2\sqrt{q}$.

Before proving it we need the main actor.

2.1.1 The Frobenius endomorphism

Definition 2.2. The *q-Frobenius* $\pi = \pi_q : E \to E$ is $\pi(x, y) = (x^q, y^q)$, $\pi(\mathcal{O}) = \mathcal{O}$. It is an *endomorphism* of E (a morphism of curves that is a group homomorphism, mapping $\mathcal{O}$ to $\mathcal{O}$), defined over $\mathbb{F}_q$, of degree q and inseparable.

Proposition 2.3 (Fixed points of Frobenius)**.** *For $P \in E(\overline{\mathbb{F}}_q)$: $P \in E(\mathbb{F}_q) \iff \pi(P) = P$. Hence $E(\mathbb{F}_q) = \ker(1-\pi)$, and since $1-\pi$ is separable, $\#E(\mathbb{F}_q) = \deg(1-\pi)$.*

Theorem 2.4 (Basic facts on endomorphisms)**.** *(i) The multiplication-by-m map $[m]$ has degree m^2. If $p \nmid m$ it is separable and $E[m] \cong (\mathbb{Z}/m)^2$; if $p \mid m$ then $E[p^k] \cong \mathbb{Z}/p^k$ or 0.*

(ii) $\deg : \mathrm{End}(E) \to \mathbb{Z}_{\geq 0}$ *is a positive-definite quadratic form:* $\deg(\alpha) = 0 \iff \alpha = 0$, $\deg(\alpha) = \deg(\hat{\alpha})$ *where $\hat{\alpha}$ is the dual endomorphism, and $\langle \alpha, \beta \rangle = \deg(\alpha+\beta) - \deg\alpha - \deg\beta$ is bilinear.*

(iii) $\alpha\hat{\alpha} = \deg\alpha$, $\alpha + \hat{\alpha} = 1 + \deg\alpha - \deg(1-\alpha) =: \mathrm{Tr}(\alpha)$.

Proof of Hasse's theorem. Set $t = \mathrm{Tr}(\pi) = 1 + q - \deg(1-\pi) = 1 + q - \#E(\mathbb{F}_q)$. For integers m, n with $n \neq 0$, positivity gives

$$0 \leq \deg(m - n\pi) = m^2 - mn\ \mathrm{Tr}(\pi) + n^2 \deg\pi = m^2 - mnt + n^2 q.$$

Dividing by n^2: $r^2 - tr + q \geq 0$ for every rational $r = m/n$, hence for all real r by density. A real quadratic that is nonnegative everywhere has nonpositive discriminant, so $t^2 - 4q \leq 0$. □

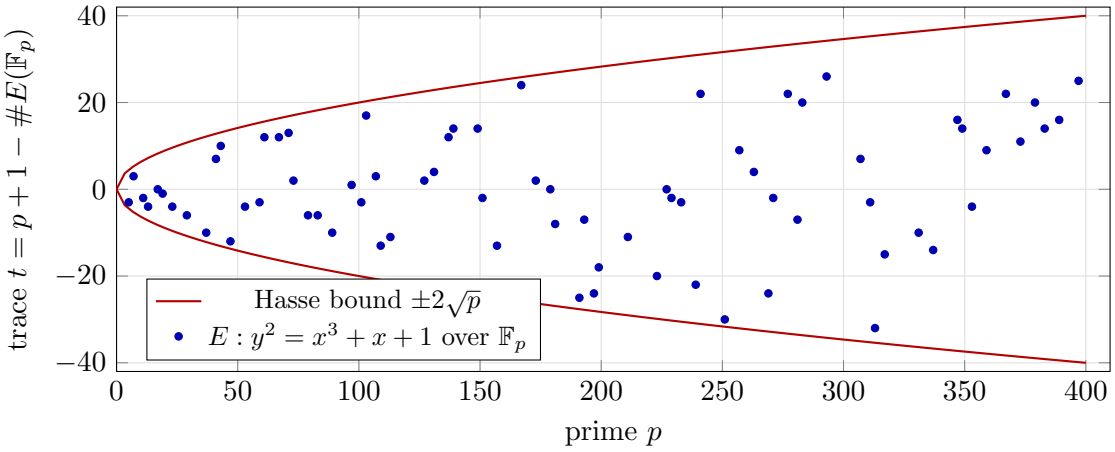


Figure 2.1: Hasse's theorem in action: the trace of Frobenius of the fixed curve $y^2 = x^3 + x + 1$ for all primes $p < 400$ (except $p = 31$, where the curve is singular). Every point lies in the strip $|t| \leq 2\sqrt{p}$, and the values fluctuate "randomly" inside it, as the random-walk heuristic predicts.

Theorem 2.5 (Characteristic polynomial of Frobenius)**.** *In $\mathrm{End}(E)$ we have $\pi^2 - t\pi + q = 0$. Equivalently π and $\hat{\pi} = t - \pi$ satisfy $\pi\hat{\pi} = q$. If $\ell \neq p$ is prime, π acts on $E[\ell] \cong (\mathbb{Z}/\ell)^2$ as a matrix with characteristic polynomial $T^2 - tT + q \pmod{\ell}$.*

Theorem 2.6 (Counting over extensions)**.** *Let $T^2 - tT + q = (T-\alpha)(T-\beta)$ with $\alpha, \beta \in \mathbb{C}$ (so $|\alpha| = |\beta| = \sqrt{q}$ by Hasse). Then for all $m \geq 1$*

$$\#E(\mathbb{F}_{q^m}) = q^m + 1 - (\alpha^m + \beta^m).$$

In particular $\#E(\mathbb{F}_{q^2}) = \#E(\mathbb{F}_q)\,(q + 1 + t)$.

Proof. The q^m-Frobenius of $E/\mathbb{F}_q$ is π^m, so by Theorem 2.3 $\#E(\mathbb{F}_{q^m}) = \deg(1-\pi^m) = (1-\pi^m)(1-\hat{\pi}^m) = 1 - (\pi^m + \hat{\pi}^m) + q^m$, using $\pi\hat{\pi} = q$. Now $\pi, \hat{\pi}$ are the two roots of $T^2 - tT + q$ (Theorem 2.5), i.e. they play the role of α, β, and $\pi^m + \hat{\pi}^m$ is a symmetric polynomial expression in them, equal to the integer $\alpha^m + \beta^m$. For $m = 2$: $\alpha^2 + \beta^2 = t^2 - 2q$ and $q^2 + 1 - t^2 + 2q = (q+1)^2 - t^2 = (q + 1 - t)(q + 1 + t)$. □

Example 2.7 (A tiny curve fully analysed)**.** Let $E : y^2 = x^3 + x + 1$ over $\mathbb{F}_5$. Squares in $\mathbb{F}_5$: $\{0, 1, 4\}$. Evaluating $f(x) = x^3 + x + 1$:

x	0	1	2	3	4
$f(x)$	1	3	1	1	4

so $\chi(f(x)) = 1, -1, 1, 1, 1$; by (2.1), $\#E(\mathbb{F}_5) = 5 + 1 + 3 = 9$ and $t = -3$. Points: $(0, \pm 1), (2, \pm 1), (3, \pm 1), (4, \pm 2), \mathcal{O}$. Hasse: $|9 - 6| = 3 \leq 2\sqrt{5} \approx 4.47$. ✓Moreover $P = (0, 1)$ has $2P = (4, 2)$, $3P = (2, 1)$, $4P = (3, 4)$, $5P = (3, 1) = -4P$, so $9P = \mathcal{O}$: $E(\mathbb{F}_5) \cong \mathbb{Z}/9$ is cyclic. By Theorem 2.6: $\#E(\mathbb{F}_{25}) = 9 \cdot (5 + 1 - 3) = 27$.

Example 2.8 (Counting by Legendre symbols: $y^2 = x^3 + 2x + 2$ over $\mathbb{F}_{17}$)**.** Compute $f(x) = x^3 + 2x + 2 \bmod 17$ and its quadratic character $\chi(f(x))$ (squares mod 17: $1, 2, 4, 8, 9, 13, 15, 16$):

x	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
$f(x)$	2	5	14	1	6	1	9	2	3	1	2	12	3	15	3	7	16
χ	+	−	−	+	−	+	+	+	−	+	+	−	−	+	−	−	+

There are 9 values with $\chi = +1$ (two points each), 8 with $\chi = -1$ (no points) and none with $\chi = 0$. So $\sum \chi = +1$ and $\#E = 17 + 1 + 1 = 19$, matching Table 1.2; the trace is $t = -1$.

Definition 2.9 (Ordinary, supersingular, anomalous)**.** $E/\mathbb{F}_q$ ($q = p^n$) is *supersingular* if $p \mid t$, *ordinary* otherwise. If $q = p$ is prime, E is *anomalous* if $\#E(\mathbb{F}_p) = p$ (i.e. $t = 1$). For $p > 3$ prime, $E/\mathbb{F}_p$ is supersingular iff $t = 0$ iff $\#E(\mathbb{F}_p) = p + 1$.

Example 2.10. For $p \equiv 3 \pmod 4$, $E : y^2 = x^3 + x$ is supersingular: $f(-x) = -f(x)$ and $\chi(-1) = -1$ so $\chi(f(-x)) = -\chi(f(x))$ and (2.1) gives $\#E(\mathbb{F}_p) = p + 1$ (cf. $p = 7, 11, 19, 23$ with $8, 12, 20, 24$ points). Supersingular curves are dangerous for ECDLP (MOV, Lecture 3) but are the cornerstone of isogeny-based cryptography (Lecture 5).

2.2 Group structure and cryptographic subgroups

Theorem 2.11 (Structure of $E(\mathbb{F}_q)$)**.** *There exist integers $n_1 \mid n_2$ with $E(\mathbb{F}_q) \cong \mathbb{Z}/n_1\mathbb{Z} \times \mathbb{Z}/n_2\mathbb{Z}$ and $n_1 \mid q - 1$. In particular if $\#E(\mathbb{F}_q)$ is squarefree the group is cyclic.*

Idea. $E(\mathbb{F}_q)$ is a finite subgroup of $E(\overline{\mathbb{F}}_q)$, and $E[\ell] \cong (\mathbb{Z}/\ell)^2$ for $\ell \neq p$ while $E[p]$ is cyclic or trivial; hence $E(\mathbb{F}_q)$ needs at most two generators, i.e. has at most two invariant factors. If $E[n_1] \subset E(\mathbb{F}_q)$ then the Weil pairing $e_{n_1} : E[n_1] \times E[n_1] \to \mu_{n_1}$ (Lecture 5) is Galois-equivariant and nondegenerate so $\mu_{n_1} \subset \mathbb{F}_q$, i.e. $n_1 \mid q - 1$. □

Example 2.12 (A non-cyclic group)**.** Let $E : y^2 = x^3 - x$ over $\mathbb{F}_5$. The three points with $y = 0$ are $(0, 0), (1, 0), (4, 0)$, and $x = 2, 3$ give $f(2) = 6 \equiv 1$, $f(3) = 24 \equiv 4$, hence four more points $(2, \pm 1), (3, \pm 2)$. Together with $\mathcal{O}$ we get 8 points. All three 2-torsion points are rational, so $E[2] \subset E(\mathbb{F}_5)$ and the group is not cyclic. A direct computation gives that $(2, 1)$ and $(2, 4)$, $(3, 2)$, $(3, 3)$ have order 4, so $E(\mathbb{F}_5) \cong \mathbb{Z}/2 \times \mathbb{Z}/4$ ($n_1 = 2$, $n_2 = 4$, and $n_1 = 2 \mid q - 1 = 4$, as the theorem requires).

Definition 2.13 (Cryptographic setting)**.** A *curve-based DLP setting* consists of (q, E, G, r, h): a finite field $\mathbb{F}_q$, a curve $E/\mathbb{F}_q$, a point $G \in E(\mathbb{F}_q)$ of *prime* order r, and the *cofactor* $h = \#E(\mathbb{F}_q)/r$. Typical standards require $h \in \{1, 2, 4, 8\}$ and $r \approx q$.

Table 2.1: Parameters of widely used curves. All have r prime and G of order r.

Curve	Field $\mathbb{F}_p$	Equation	h	Standard
NIST P-256	$p = 2^{256} - 2^{224} + 2^{192} + 2^{96} - 1$	$y^2 = x^3 - 3x + b$	1	FIPS 186-5, SEC 2
secp256k1	$p = 2^{256} - 2^{32} - 977$	$y^2 = x^3 + 7$	1	SEC 2 (Bitcoin, Ethereum)
Curve25519	$p = 2^{255} - 19$	$y^2 = x^3 + 486662x^2 + x$	8	RFC 7748 (X25519)
Edwards25519	$p = 2^{255} - 19$	$-x^2 + y^2 = 1 + dx^2y^2$	8	RFC 8032 (Ed25519)
Ed448 / Curve448	$p = 2^{448} - 2^{224} - 1$	Edwards / Montgomery	4	RFC 7748, 8032

Key idea

In practice one works in the prime-order subgroup $\langle G \rangle$ of order r. If $h > 1$ one must make sure that received points actually lie in $\langle G \rangle$ (or multiply by h, "cofactor clearing"). We return to this in Section 4.5.

2.3 Counting points efficiently

To select a secure curve one must know $\#E(\mathbb{F}_q)$: it must be divisible by a large prime r, and avoid the weak cases of Lecture 3.

Table 2.2: Point-counting algorithms for $E/\mathbb{F}_p$, p about n bits.

Algorithm	Complexity	Range
Naive: evaluate (2.1)	$\mathcal{O}(p \operatorname{polylog} p)$	$p < 2^{30}$
BSGS / Mestre	$\widetilde{\mathcal{O}}(p^{1/4})$	$p < 2^{80}$
Schoof (1985)	$\widetilde{\mathcal{O}}(n^5)$–$\widetilde{\mathcal{O}}(n^8)$ bit ops	polynomial time; practical via SEA
Schoof–Elkies–Atkin (SEA)	$\widetilde{\mathcal{O}}(n^4)$ heuristically	thousands of bits
Satoh, AGM (small characteristic)	$\widetilde{\mathcal{O}}(n^2)$–$\widetilde{\mathcal{O}}(n^3)$	$\mathbb{F}_{p^n}$, p small
CM method	construct (not count)	prescribed order

2.3.1 Schoof's algorithm

By Hasse, $t \in [-2\sqrt{q}, 2\sqrt{q}]$. If we know $t \bmod \ell$ for primes ℓ with $\prod \ell > 4\sqrt{q}$, the Chinese remainder theorem determines t. Only $\mathcal{O}(\log q)$ small primes are needed. For $\ell \neq p$ we compute $t \bmod \ell$ using Theorem 2.5: for every $P \in E[\ell]$,

$$\pi^2(P) + [\bar{q}]P = [\bar{t}]\,\pi(P), \qquad \bar{q} = q \bmod \ell,\ \bar{t} = t \bmod \ell. \tag{2.2}$$

Division polynomials. Define $\psi_0 = 0$, $\psi_1 = 1$, $\psi_2 = 2y$, $\psi_3 = 3x^4 + 6ax^2 + 12bx - a^2$, $\psi_4 = 4y(x^6 + 5ax^4 + 20bx^3 - 5a^2x^2 - 4abx - 8b^2 - a^3)$ and

$$\psi_{2m+1} = \psi_{m+2}\psi_m^3 - \psi_{m-1}\psi_{m+1}^3 \ (m \geq 2), \qquad \psi_{2m} = \frac{\psi_m(\psi_{m+2}\psi_{m-1}^2 - \psi_{m-2}\psi_{m+1}^2)}{2y} \ (m \geq 3).$$

Theorem 2.14 (Division polynomials)**.** *$\psi_n \in \mathbb{Z}[a, b, x, y]$, and (after replacing y^2 by $f(x)$) ψ_n is a polynomial in x of degree $(n^2 - 1)/2$ (n odd), resp. y times a polynomial of degree $(n^2 - 4)/2$ (n even). For $P = (x, y) \neq \mathcal{O}$ with $nP \neq \mathcal{O}$,*

$$nP = \Big(x - \frac{\psi_{n-1}\psi_{n+1}}{\psi_n^2},\ \frac{\psi_{2n}}{2\psi_n^4}\Big),$$

and the x-coordinates of the nonzero n-torsion points are exactly the roots of ψ_n (n odd) .

Using Theorem 2.14, equation (2.2) can be tested for $\bar{t} = 0, \dots, \ell - 1$ in the ring $\mathbb{F}_q[x, y]/(y^2 - f(x), \psi_\ell(x))$, which has dimension $\mathcal{O}(\ell^2)$ over $\mathbb{F}_q$ and where $\pi(x, y) = (x^q, y^q)$ is computed by fast exponentiation. The cost is polynomial in $\log q$ since $\ell = \mathcal{O}(\log q)$.

Example 2.15 (Assembling t by the Chinese remainder theorem)**.** For $E : y^2 = x^3 + 2x + 2$ over $\mathbb{F}_{17}$ we know $|t| \le 2\sqrt{17} < 8.25$, so $t \in \{-8, \dots, 8\}$ (an interval of length 17). Suppose Schoof's algorithm has produced $t \equiv 1 \pmod 2$, $t \equiv 2 \pmod 3$ and $t \equiv 4 \pmod 5$. Since $2 \cdot 3 \cdot 5 = 30 > 17$ these residues determine t inside the Hasse interval: the unique solution mod 30 is $t \equiv 29 \equiv -1$, and $-1 \in [-8, 8]$. So $t = -1$ and $\#E = 17 + 1 + 1 = 19$. In real implementations the primes ℓ are chosen until $\prod \ell > 4\sqrt{q}$, about $\log q$ primes in total.

Example 2.16 (Schoof for $\ell = 2$)**.** 2-torsion points are $(r, 0)$ with $f(r) = 0$. Thus t is even $\iff$ $q + 1 - t$ is even $\iff$ E has a point of order 2 over $\mathbb{F}_q$ $\iff$ $\gcd(x^q - x, f(x)) \neq 1$. For $E : y^2 = x^3 + 2x + 2$ over $\mathbb{F}_{17}$ one finds no root (see Table 1.2: no point has $y = 0$), so t is odd, in accordance with $\#E = 19$, $t = -1$.

Remark 2.17 (SEA and the CM method $\star$). *Elkies primes*: if $T^2 - tT + q$ splits mod ℓ, then π has an eigenvector in $E[\ell]$ generating a subgroup that is the kernel of an ℓ-isogeny, whose kernel polynomial has degree $(\ell - 1)/2$ instead of $(\ell^2 - 1)/2$; this gives a quadratic speed-up. *Atkin primes* yield only partial information about t. In the *CM method* one fixes a trace t and a small discriminant $D < 0$ with $4q - t^2 = Dv^2$ and builds a curve with exactly this trace as a root of the Hilbert class polynomial $H_D(X)$ mod q. This is the technique to generate pairing-friendly curves (Lecture 5).

2.4 Other curve models

For cryptographic speed and safety, Weierstrass form is not always optimal. Two other models dominate modern practice.

2.4.1 Montgomery curves

Definition 2.18. A *Montgomery curve* over K (char $\neq 2$) is $M_{A,B} : By^2 = x^3 + Ax^2 + x$ with $B(A^2 - 4) \neq 0$.

An elliptic curve in short Weierstrass form is birationally equivalent to a Montgomery curve iff $f(x)$ has a root α with $3\alpha^2 + a$ a square. In particular its group order is divisible by 4. The main attraction: *the x-coordinate alone suffices* to compute $[k]P$ with a *differential addition* (see Section 2.6.3), and the arithmetic is uniform, short, and has no exceptional cases.

2.4.2 Twisted Edwards curves

Definition 2.19. A *twisted Edwards curve* over K (char $\neq 2$) is $E_{a,d} : ax^2 + y^2 = 1 + dx^2y^2$ with $a, d \neq 0$, $a \neq d$.

Theorem 2.20 (Edwards addition law; Bernstein–Lange)**.** *On $E_{a,d}$ the set of affine points forms a group with neutral element $(0, 1)$, $-(x, y) = (-x, y)$ and*

$$(x_1, y_1) + (x_2, y_2) = \Big(\frac{x_1y_2 + x_2y_1}{1 + d\,x_1x_2y_1y_2},\ \frac{y_1y_2 - a\,x_1x_2}{1 - d\,x_1x_2y_1y_2} \Big).$$

If a is a square and d is a non-square in K, the denominators never vanish: the law is complete *(valid for all pairs of points, including doubling and the inverse).*

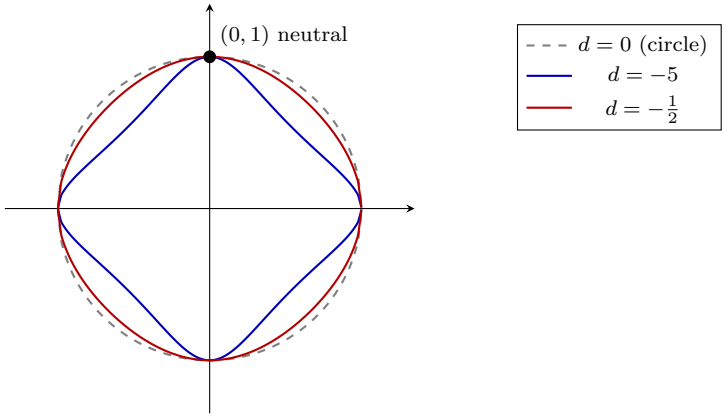


Figure 2.2: Real points of the Edwards curves $x^2 + y^2 = 1 + dx^2y^2$ for $d = 0, -\frac{1}{2}, -5$. The group law generalises the angle addition formulas of the circle ($d = 0$): $(\cos\theta_1, \sin\theta_1) + (\cos\theta_2, \sin\theta_2) = (\cos(\theta_1 + \theta_2), \sin(\theta_1 + \theta_2))$.

Theorem 2.21 (Birational equivalence)**.** *Every Montgomery curve $By^2 = x^3 + Ax^2 + x$ is birationally equivalent to the twisted Edwards curve $E_{a,d}$ with $a = \frac{A+2}{B}$, $d = \frac{A-2}{B}$ via*

$$(u, v) = \left(\frac{1+y}{1-y},\ \frac{1+y}{(1-y)x}\right) \quad \textit{from Edwards } (x, y) \textit{ to Montgomery } (u, v).$$

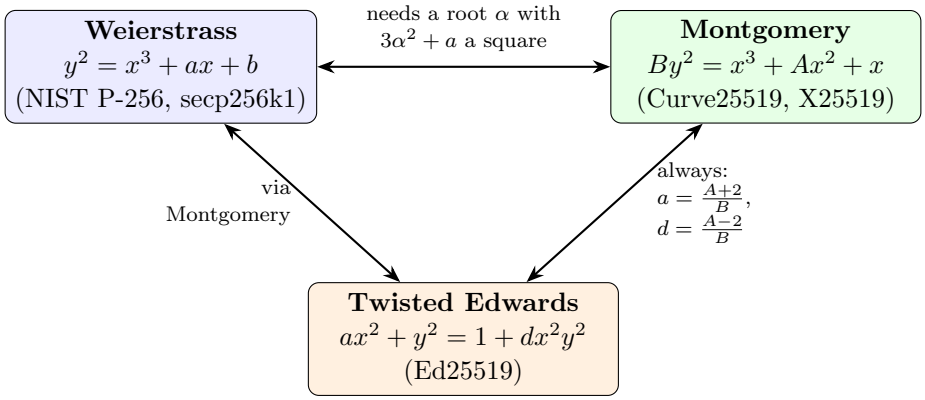


Figure 2.3: The three curve models and how they are related by birational maps. All three describe the *same* group; they differ in which formulas are fast and which implementation pitfalls they avoid.

Example 2.22 (Curve25519 and Ed25519)**.** $p = 2^{255} - 19$. The Montgomery curve $v^2 = u^3 + 486662u^2 + u$ ($A = 486662, B = 1$) has $\#E = 8r$ with

$$r = 2^{252} + 27742317777372353535851937790883648493 \quad \text{prime};$$

the base point is $u = 9$. Theorem 2.21 gives $a = 486664$, $d = 486660$; after rescaling one obtains Edwards25519: $-x^2 + y^2 = 1 + dx^2y^2$ with $d = -121665/121666$ (a non-square mod p, so the law is complete since -1 is a square as $p \equiv 1 \pmod 4$). The twist of Curve25519 has order $4r'$ with r' prime ("twist-secure").

2.5 Coordinate systems and the cost of a group operation

The affine formulas (1.4) need one field inversion per operation. An inversion costs $\mathbf{I} \approx 20$–$100\,\mathbf{M}$ (via Fermat a^{p-2} or extended Euclid), so in practice one avoids it.

Definition 2.23 (Projective coordinates)**.** *Standard projective:* $(X : Y : Z) \leftrightarrow (X/Z, Y/Z)$. *Jacobian:* $(X : Y : Z) \leftrightarrow (X/Z^2, Y/Z^3)$, with $\mathcal{O} = (1 : 1 : 0)$ and equation $Y^2 = X^3 + aXZ^4 + bZ^6$.

Proposition 2.24 (Jacobian formulas)**.** *Let* $P_1 = (X_1, Y_1, Z_1)$, $P_2 = (X_2, Y_2, Z_2)$ *be Jacobian points on* $y^2 = x^3 + ax + b$. Doubling $2P_1 = (X_3, Y_3, Z_3)$*:*

$$S = 4X_1Y_1^2,\ M = 3X_1^2 + aZ_1^4,\quad X_3 = M^2 - 2S,\ Y_3 = M(S - X_3) - 8Y_1^4,\ Z_3 = 2Y_1Z_1.$$

Addition $P_1 + P_2$ *(when* $P_1 \neq \pm P_2$*): with* $U_1 = X_1Z_2^2$, $U_2 = X_2Z_1^2$, $S_1 = Y_1Z_2^3$, $S_2 = Y_2Z_1^3$, $H = U_2 - U_1$, $R = S_2 - S_1$,

$$X_3 = R^2 - H^3 - 2U_1H^2,\quad Y_3 = R(U_1H^2 - X_3) - S_1H^3,\quad Z_3 = HZ_1Z_2.$$

Table 2.3: Textbook operation counts (Hankerson–Menezes–Vanstone) for $a = -3$. Modern formulas (Explicit-Formulas Database) are slightly cheaper.

	Affine	Jacobian ($a = -3$)	Edwards (extended)
Doubling	$1\mathbf{I} + 2\mathbf{M} + 2\mathbf{S}$	$4\mathbf{M} + 4\mathbf{S}$	$\approx 4\mathbf{M} + 4\mathbf{S}$
Addition	$1\mathbf{I} + 2\mathbf{M} + 1\mathbf{S}$	$12\mathbf{M} + 4\mathbf{S}$ (mixed: $8\mathbf{M} + 3\mathbf{S}$)	≈ 8–$9\mathbf{M}$ (complete)

Warning

Standard Weierstrass formulas have *exceptional cases* (adding $P + P$, $P + (-P)$, $\mathcal{O} + P$). A secure implementation either handles them with branches (a side-channel risk) or uses a *complete* formula (Renes–Costello–Batina for Weierstrass; Edwards formulas of Theorem 2.20).

2.6 Scalar multiplication

The fundamental operation of ECC is $k \mapsto kP$. In additive notation $kP = P + \cdots + P$, which is the analogue of exponentiation.

2.6.1 Double-and-add

It uses $n - 1$ doublings and, on average, $(n - 1)/2$ additions. For $n = 256$: about $255\,\mathrm{D} + 128\,\mathrm{A}$. Its running time depends on the Hamming weight of k, which makes it vulnerable to *simple power analysis* (Section 4.6).

Example 2.25 (Computing $13G$ on $E(\mathbb{F}_{17})$)**.** Here $13 = 1101_2$. Start with $R = G$ (the leading bit) and process the remaining bits $1, 0, 1$ from left to right, using the points of Table 1.2:

bit	after doubling	after adding G (if bit $= 1$)	R
(start)			$G = (5, 1)$
1	$2G = (6, 3)$	$3G = (10, 6)$	$3G$
0	$6G = (16, 13)$	—	$6G$
1	$12G = (0, 11)$	$13G = (16, 4)$	$13G = (16, 4)$

Algorithm 2 Left-to-right double-and-add

Input: $k = \sum_{i=0}^{n-1} k_i 2^i$ with $k_{n-1} = 1$, point P
Output: kP

```
R ← P
for i = n − 2 down to 0 do
    R ← 2R
    if k_i = 1 then R ← R + P
    end if
end for
return R
```

Three doublings and two additions instead of twelve additions. The binary digits of k are read off by the pattern of operations: "D A" for a bit 1 and "D" for a bit 0.

2.6.2 Signed digits and windows

Point negation is free, so one can use digits in $\{-1, 0, 1\}$.

Definition 2.26 (NAF)**.** A *non-adjacent form* of k is $k = \sum_i u_i 2^i$ with $u_i \in \{-1, 0, 1\}$ and $u_i u_{i+1} = 0$ for all i.

Theorem 2.27 (Reitwiesner)**.** *Every integer has a unique NAF, which has the minimal Hamming weight among all signed-binary representations; its length is at most one more than the binary length and its average density of non-zero digits is* $1/3$.

Algorithm 3 Computing the NAF

Input: integer $k > 0$
Output: digits $u_0, u_1, \ldots$

```
i ← 0
while k > 0 do
    if k odd then u_i ← 2 − (k mod 4); k ← k − u_i
    else u_i ← 0
    end if
    k ← k/2; i ← i + 1
end while
```

Example 2.28. For $k = 7 = 111_2$: k odd, $k \bmod 4 = 3$, so $u_0 = -1$, $k = 8 \to 4$; $u_1 = u_2 = 0$; $k = 1$: $u_3 = 1$. So $7 = 2^3 - 1$ and $7P = 2^3 P - P$: 3 doublings and 1 addition instead of 2 doublings and 2 additions. Likewise $k = 13 = 1101_2$ has NAF digits $(u_4, \ldots, u_0) = (1, 0, -1, 0, 1)$, i.e. $13 = 16 - 4 + 1$ (the algorithm outputs $u_0 = 1, u_1 = 0, u_2 = -1, u_3 = 0, u_4 = 1$).

Windowed NAF (w-NAF) with 2^{w-2} precomputed odd multiples $P, 3P, \ldots$ reduces the number of additions to about $n/(w+1)$. For fixed base points (signing), *comb* methods precompute tables to trade memory for time; for variable base (ECDH) one uses a ladder or windowing.

2.6.3 The Montgomery ladder

Proposition 2.29 (Ladder invariant)**.** *At the start of each iteration,* $R_1 - R_0 = P$*, and after processing bits* $k_{n-1}, \ldots, k_i$*,* $R_0 = \lfloor k/2^i \rfloor P$.

Algorithm 4 Montgomery ladder (uniform, no key-dependent branches except by selection)

Input: $k = \sum_{i=0}^{n-1} k_i 2^i$, point P
Output: kP

```
R0 ← O; R1 ← P
for i = n − 1 down to 0 do
    if k_i = 0 then R1 ← R0 + R1; R0 ← 2R0
    else R0 ← R0 + R1; R1 ← 2R1
    end if
end for
return R0
```

Proof. Initially $R_1 - R_0 = P$ and $R_0 = 0$. If $k_i = 0$ the new pair is $(2R_0, R_0 + R_1)$ with difference $R_1 - R_0 = P$, and $2\lfloor k/2^{i+1} \rfloor = \lfloor k/2^i \rfloor$ since the bit is 0. If $k_i = 1$ the new pair is $(R_0 + R_1, 2R_1)$ with difference $R_1 - R_0 = P$ and $R_0 + R_1 = (2\lfloor k/2^{i+1} \rfloor + 1)P$ – indeed $R_0 + R_1 = 2R_0 + P$. □

Example 2.30 (Ladder trace for $11G$)**.** For $k = 11 = 1011_2$ on $E(\mathbb{F}_{17})$ with $G = (5, 1)$ (points from Table 1.2); in the notation of Algorithm 4:

bit k_i	operation	R_0	R_1	$R_1 - R_0$
(start)		$\mathcal{O}$	G	G
1	$R_0 \leftarrow R_0 + R_1,\ R_1 \leftarrow 2R_1$	G	$2G$	G
0	$R_1 \leftarrow R_0 + R_1,\ R_0 \leftarrow 2R_0$	$2G$	$3G$	G
1	$R_0 \leftarrow R_0 + R_1,\ R_1 \leftarrow 2R_1$	$5G$	$6G$	G
1	$R_0 \leftarrow R_0 + R_1,\ R_1 \leftarrow 2R_1$	$11G = (13, 10)$	$12G$	G

Notice that the difference $R_1 - R_0$ is always G, and that every bit costs one addition and one doubling – whatever the bit is.

Each iteration performs exactly one doubling and one addition, regardless of the bit. Moreover, since $R_1 - R_0 = P$ is invariant, only *differential addition* is needed: on a Montgomery curve with $(A - 2)/4 = a_{24}$ one can work with x-coordinates in projective form $x = X/Z$ only (this is the X25519 ladder of RFC 7748). With $x_1 = x(P)$:

$$A = X_2 + Z_2,\ AA = A^2,\ B = X_2 - Z_2,\ BB = B^2,\ E = AA - BB,$$
$$C = X_3 + Z_3,\ D = X_3 - Z_3,\ DA = D \cdot A,\ CB = C \cdot B,$$
$$X_3' = (DA + CB)^2,\ Z_3' = x_1(DA - CB)^2,\ X_2' = AA \cdot BB,\ Z_2' = E\,(AA + a_{24}E).$$

Cost per bit: $5\mathbf{M} + 4\mathbf{S} + 1\mathbf{M}_{a_{24}}$; in implementations the swap at the beginning of the iteration is done by a branch-free *conditional swap*. The final answer is X_2/Z_2.

2.6.4 Endomorphism-accelerated multiplication (GLV/GLS) ⋆

Theorem 2.31 (Gallant–Lambert–Vanstone, 2001)**.** *Suppose E has an efficiently computable endomorphism ψ with $\psi(P) = \lambda P$ for P of prime order r ($\lambda^2 + \lambda + 1 \equiv 0 \bmod r$ for $j = 0$). Then every $k \in \mathbb{Z}/r$ can be written $k \equiv k_1 + k_2\lambda \pmod r$ with $|k_1|, |k_2| = \mathcal{O}(\sqrt{r})$, and $kP = k_1P + k_2\psi(P)$ can be computed with one simultaneous multiplication (Shamir's trick) at half the number of doublings.*

Example 2.32 (secp256k1)**.** Let $\beta = \texttt{7AE96A2B...01EE} \in \mathbb{F}_p$ be a primitive cube root of unity ($\beta^3 = 1$) and $\lambda = \texttt{5363AD4C...BD72} \bmod r$ a cube root of unity in $\mathbb{Z}/r$. Then $\psi(x, y) = (\beta x, y)$ satisfies $\psi(P) = \lambda P$ for every $P \in E(\mathbb{F}_p)$ of order r. (We verified $\lambda \cdot G = (\beta G_x, G_y)$ numerically for the standard generator G.) The multiplication ψ costs a single field multiplication.

Exercises for Lecture 2

Exercise 2.1. Count the points of $y^2 = x^3 + x + 1$ over $\mathbb{F}_7$ and $\mathbb{F}_{11}$ using (2.1); check Hasse's bound and compute t.

Exercise 2.2 (Extension fields). For $E/\mathbb{F}_5$ of Theorem 2.7 ($t = -3$) compute $\#E(\mathbb{F}_{5^m})$ for $m = 1, 2, 3, 4$. Verify that $\#E(\mathbb{F}_5)$ divides $\#E(\mathbb{F}_{5^m})$ in each case, as it must (subgroup).

Exercise 2.3 (Supersingular). Let $p \equiv 2 \pmod 3$. Show that $y^2 = x^3 + b$ over $\mathbb{F}_p$ has exactly $p + 1$ points. *Hint:* $x \mapsto x^3$ is a bijection of $\mathbb{F}_p$.

Exercise 2.4 (Anomalous curve). Show that over $\mathbb{F}_p$ with $p = 11$ there is a curve $y^2 = x^3 + ax + b$ with $\#E = 11$ (i.e. $t = 1$) by trying small (a, b).

Exercise 2.5 (Structure). (a) Let $q = 101$ and $\#E(\mathbb{F}_q) = 101$. What is the group? (b) Let $q = 101$ and $\#E(\mathbb{F}_q) = 100$ (so $t = 2$). List all abelian groups of order 100 with at most two invariant factors and check which are compatible with Theorem 2.11. (c) Repeat with $q = 103$ and $\#E(\mathbb{F}_q) = 100$ ($t = 4$). Which groups remain? (Answer: $\mathbb{Z}/100$ and $\mathbb{Z}/2 \times \mathbb{Z}/50$.)

Exercise 2.6 (Schoof by hand, $\ell = 3$). Compute ψ_3 for $E : y^2 = x^3 + 2x + 2$ over $\mathbb{F}_{17}$ and show it has no root in $\mathbb{F}_{17}$, consistent with $\#E(\mathbb{F}_{17}) = 19$ not being divisible by 3. Then use Theorem 2.6 with $t = -1$ to compute $\#E(\mathbb{F}_{17^2}) = 19 \cdot 17 = 323$ and verify $E(\mathbb{F}_{17^2}) \cong \mathbb{Z}/19 \times \mathbb{Z}/17$ (hence cyclic).

Exercise 2.7 (Jacobian formulas). Prove the Jacobian doubling formula in Theorem 2.24: substitute $x = X/Z^2, y = Y/Z^3$ into the affine doubling formula and clear denominators (take $Z_3 = 2Y_1Z_1$).

Exercise 2.8 (Edwards group law). For $a = 1$, $d = -\frac{1}{2}$ over $\mathbb{R}$ show that $(0, -1)$ has order 2 and $(\pm 1, 0)$ has order 4. Show that $\{(0, 1), (0, -1), (1, 0), (-1, 0)\}$ is a subgroup isomorphic to $\mathbb{Z}/4$.

Exercise 2.9 (NAF). Compute the NAF of $k = 255$ and $k = 1000$. How many additions does double-and-add (binary) vs. NAF require for each?

Exercise 2.10 (Ladder). Execute the Montgomery ladder (Algorithm 4) on $E : y^2 = x^3 + 2x + 2$ over $\mathbb{F}_{17}$ for $k = 13$, $P = (5, 1)$ and compare with Table 1.2 ($13G = (16, 4)$). Check that $R_1 - R_0 = P$ at each step.

Exercise 2.11 (Programming). Implement double-and-add and the ladder in Python (Lecture B) and count, for random 256-bit k on secp256k1, the number of point additions and doublings for each method. Plot the histogram of the number of additions for double-and-add and comment on its security implication.

Chapter 3

The Elliptic Curve Discrete Logarithm Problem

Learning goals

After this lecture students should be able to

- define ECDLP, CDH, DDH and discuss their relations;
- state the generic-group lower bound and describe the three generic algorithms: baby-step giant-step, Pohlig–Hellman, Pollard's rho (with parallelisation and negation map);
- explain why index calculus does not (currently) apply to $E(\mathbb{F}_p)$, and where it does (extension fields, Weil descent);
- describe the special attacks (anomalous curves, MOV/Frey–Rück, GHS) and use them to explain how secure curves are selected.

3.1 Problems: ECDLP, CDH, DDH

Let $E/\mathbb{F}_q$ be an elliptic curve and $G \in E(\mathbb{F}_q)$ a point of prime order r (Theorem 2.13). We now write the group additively, so the discrete logarithm of $Q \in \langle G \rangle$ is the integer $k \in \mathbb{Z}/r$ with $Q = kG$.

Definition 3.1.

- ECDLP: given (E, G, Q) with $Q \in \langle G \rangle$, find $k \in \mathbb{Z}/r$ with $Q = kG$.
- CDH: given (G, aG, bG) for uniform $a, b \in \mathbb{Z}/r$, compute abG.
- DDH: distinguish (G, aG, bG, abG) from (G, aG, bG, cG) with $a, b, c \in \mathbb{Z}/r$ uniform.

Obviously $\mathsf{DDH} \leq \mathsf{CDH} \leq \mathsf{ECDLP}$ (any oracle for the right problem solves the left one). The reductions in the other direction are not known in general; there are partial results:

Theorem 3.2 (Maurer, Wolf; Muzereau–Smart–Vercauteren)**.** *If $r - 1$ is smooth, or more generally if some auxiliary elliptic curve over $\mathbb{F}_r$ with smooth order is known, then* ECDLP *and* CDH *are polynomial-time equivalent (with a generic constant slowdown).*

Remark 3.3 (Gap groups). On curves with an efficiently computable bilinear pairing e (Lecture 5), DDH is easy: (G, aG, bG, cG) is a DH tuple iff $e(aG, bG) = e(G, cG)$. Yet CDH is believed hard: such groups are called *gap Diffie–Hellman groups* and underlie BLS signatures.

3.2 Generic algorithms

3.2.1 Lower bound in the generic group model

A *generic algorithm* sees group elements only as opaque labels and can only apply the group operation and test equality.

Theorem 3.4 (Nechaev 1994, Shoup 1997)**.** *Let r be prime. Any generic algorithm that solves the DLP in a group of order r with probability ε must perform $\Omega(\sqrt{\varepsilon\, r})$ group operations.*

Idea. Each group element handed to the algorithm is $uG + vQ$ for some known pair $(u, v) \in \mathbb{Z}_r^2$ (as a formal expression in the unknown k, $u + vk$). A collision between two different labels $(u, v) \neq (u', v')$ reveals k by $(u - u') + (v - v')k \equiv 0$. After m operations at most $\binom{m}{2}$ pairs can collide, and for a uniformly random k any fixed pair collides with probability $\leq 1/r$. So success probability is $\leq \binom{m}{2}/r + 1/r$. □

Key idea

Generic lower bound $\sqrt{r} \Longrightarrow$ a curve with $r \approx 2^{2s}$ offers s bits of security *if* the group admits no non-generic attack. That is the whole point of ECC: for $E(\mathbb{F}_p)$, no non-generic attack is known.

3.2.2 Baby-step giant-step (Shanks)

Algorithm 5 Baby-step giant-step

Input: G of order r (or an upper bound r on it), $Q \in \langle G \rangle$
Output: k with $Q = kG$

1: $m \leftarrow \lceil \sqrt{r} \rceil$
2: store table $T = \{(jG,\ j) : 0 \leq j < m\}$ in a hash table keyed by the point (*baby steps*)
3: $S \leftarrow mG$; $R \leftarrow Q$
4: **for** $i = 0, 1, \ldots, m-1$ **do** ▷ *giant steps*
5: **if** R is a key of T, with value j **then return** $k = im + j$
6: **end if**
7: $R \leftarrow R - S$
8: **end for**

Correctness. Write $k = im + j$ with $0 \leq j < m$, $0 \leq i < m$. Then $Q - i\,(mG) = jG \in T$. **Cost.** $\mathcal{O}(\sqrt{r})$ group operations and $\mathcal{O}(\sqrt{r})$ memory. Because memory is the bottleneck (a table of 2^{64} points is impossible), rho methods are preferred in practice.

Example 3.5 (BSGS in $E(\mathbb{F}_{17})$)**.** Let $G = (5, 1)$ (order 19) and $Q = (16, 13)$. Take $m = \lceil \sqrt{19} \rceil = 5$. Baby steps: $\{\mathcal{O} \mapsto 0,\ (5,1) \mapsto 1,\ (6,3) \mapsto 2,\ (10,6) \mapsto 3,\ (3,1) \mapsto 4\}$. Giant step: $5G = (9, 16)$. $i = 0$: $Q = (16, 13)$ is not in the table. $i = 1$: $Q - 5G = (16, 13) - (9, 16) = (5, 1)$, which is in the table with $j = 1$. Hence $k = i \cdot m + j = 1 \cdot 5 + 1 = 6$, and indeed $6G = (16, 13)$.

3.2.3 Pohlig–Hellman

Theorem 3.6 (Pohlig–Hellman, 1978)**.** *Let G have order $N = \prod_i p_i^{e_i}$ and let $Q = kG$. Then $k \bmod N$ can be computed from the discrete logarithms in subgroups of prime order p_i using $\mathcal{O}\big(\sum_i e_i(\log N + \sqrt{p_i})\big)$ group operations (with BSGS or rho for the prime-order step).*

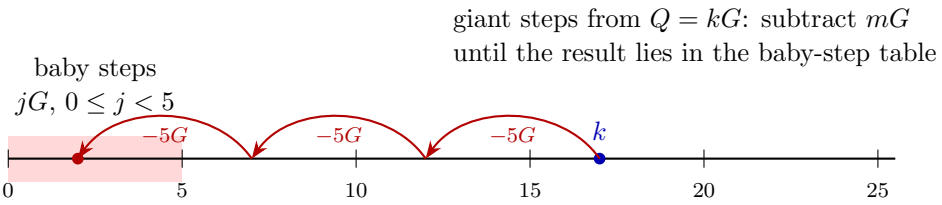


Figure 3.1: Baby-step giant-step on the exponent line $0, \dots, m^2 - 1$ ($m = 5$). The table covers the first m exponents; the giant steps (jumps of length m) are guaranteed to land in it after at most m jumps. Here $k = 17$ lands at $j = 2$ after $i = 3$ jumps, so $k = 3 \cdot 5 + 2$.

Algorithm and correctness. By the Chinese remainder theorem it suffices to find $k \bmod p^e$ for each prime power $p^e \parallel N$. Put $N' = N/p^e$, $G' = N'G$, $Q' = N'Q$: then $Q' = kG'$ with G' of order p^e. Write $k \equiv k_0 + k_1 p + \cdots + k_{e-1} p^{e-1} \pmod{p^e}$ with $0 \le k_i < p$. Then

$$p^{e-1} Q' = k_0 \, (p^{e-1} G'),$$

where $p^{e-1} G'$ has order p; solve this DLP to get k_0. Inductively, given $k_0, \dots, k_{s-1}$,

$$p^{e-1-s} \big(Q' - (k_0 + \cdots + k_{s-1} p^{s-1}) G' \big) = k_s \, (p^{e-1} G')$$

yields k_s. Combine all residues using CRT. □

Example 3.7 (Pohlig–Hellman on $E(\mathbb{F}_5)$)**.** Take $E : y^2 = x^3 + x + 1$ over $\mathbb{F}_5$ (Theorem 2.7) with $P = (0, 1)$ of order $9 = 3^2$ and $Q = (4, 3)$. Find k with $Q = kP$. Here $p = 3$, $e = 2$. First $3Q = (2, 1)$ and $3P = (2, 1)$, so $k_0 = 1$. Next $Q - k_0 P = (4, 3) - (0, 1) = (2, 4) = 6P$ and we need $6P = k_1 \cdot (3P)$, so $k_1 = 2$. Thus $k = k_0 + 3k_1 = 1 + 6 = 7$ and indeed $Q = 7P = (4, 3)$.

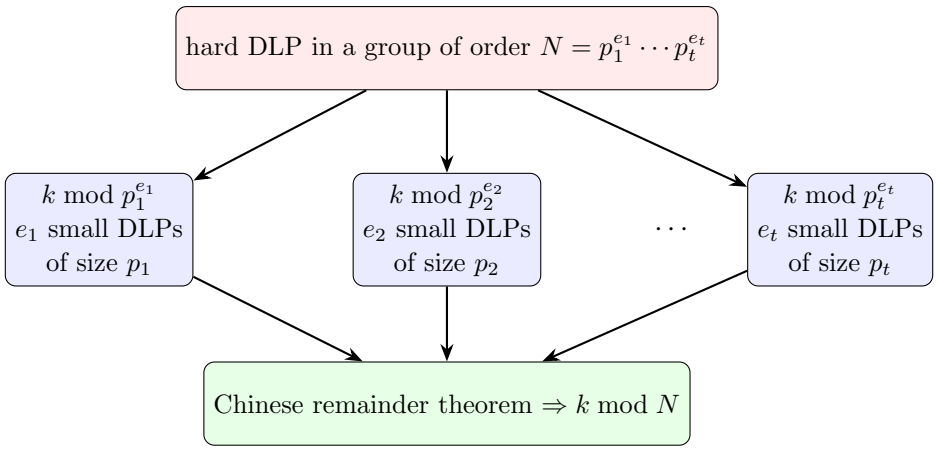


Figure 3.2: Pohlig–Hellman: the DLP is only as hard as the *largest prime factor* of the group order. If N is smooth, every sub-problem is tiny; if $N = r$ is prime, there is nothing to decompose.

Consequence for parameter choice

The DLP in $E(\mathbb{F}_q)$ is only as hard as the *largest prime factor* r of the order of the subgroup in use. Hence all standards use groups of prime order r (or $h \cdot r$ with tiny cofactor h).

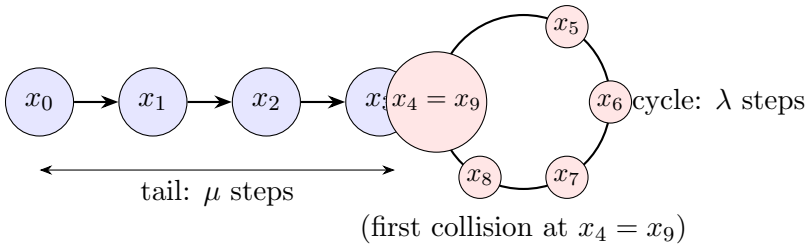


Figure 3.3: The "ρ" shape of the sequence $x_{i+1} = \mathcal{W}(x_i)$ for a pseudo-random map $\mathcal{W}$ on a finite set. A collision $x_i = x_j$ with $i \neq j$ yields (via the stored coefficients) a linear relation between $\log_G Q$ and known integers. Here $\mu = 4$, $\lambda = 5$.

Algorithm 6 Pollard's rho for ECDLP (Floyd cycle finding, 3-partition walk)

Input: G of prime order r, $Q \in \langle G \rangle$; a partition $\langle G \rangle = S_1 \sqcup S_2 \sqcup S_3$ (e.g. by x-coordinate mod 3)
Output: k with $Q = kG$

pick random $a_0, b_0 \in \mathbb{Z}_r$; $R_0 \leftarrow a_0 G + b_0 Q$; set $(R, a, b) \leftarrow (R_0, a_0, b_0)$ and $(R', a', b') \leftarrow (R_0, a_0, b_0)$
repeat
    $(R, a, b) \leftarrow \textsc{Step}(R, a, b)$; $(R', a', b') \leftarrow \textsc{Step}(\textsc{Step}(R', a', b'))$
until $R = R'$
if $b \equiv b' \pmod r$ **then return** failure (restart)
end if
return $k \equiv (a' - a)(b - b')^{-1} \pmod r$

function $\textsc{Step}(R, a, b)$
    if $R \in S_1$ **then return** $(R + G,\ a + 1,\ b)$
    else if $R \in S_2$ **then return** $(2R,\ 2a,\ 2b)$
    else return $(R + Q,\ a,\ b + 1)$
    end if
end function

3.2.4 Pollard's rho

Pollard (1978) devised a memoryless method with the same $\sqrt{r}$ running time. The idea: random-walk in $\langle G \rangle$ until two walkers collide, remembering how each point was reached as $aG + bQ$.

Invariant. $R = aG + bQ$ throughout. At a collision $aG + bQ = a'G + b'Q$, so $(a - a')G = (b' - b)Q = (b' - b)kG$, i.e. $k(b' - b) \equiv a - a' \pmod r$. If $b \not\equiv b'$ this gives k (the sign in Algorithm 6 is consistent with this).

Example 3.8 (A complete rho run in $E(\mathbb{F}_{17})$)**.** Again $G = (5, 1)$, $r = 19$, $Q = (16, 13)$ (secretly $Q = 6G$). Partition by x mod 3: $S_1 = \{x \equiv 0\}$ (add G), $S_2 = \{x \equiv 1\}$ (double), $S_3 = \{x \equiv 2\}$ (add Q). Start at $R_0 = G$, i.e. $(a, b) = (1, 0)$. The walk, with $R_i = a_i G + b_i Q$:

i	R_i	x mod 3	rule	(a_i, b_i)	check $(a_i + 6b_i \bmod 19)$
0	$(5, 1)$	2	$+Q$	$(1, 0)$	$1 \Rightarrow G$
1	$(0, 6)$	0	$+G$	$(1, 1)$	$7 \Rightarrow 7G = (0, 6)$
2	$(13, 7)$	1	double	$(2, 1)$	$8 \Rightarrow 8G$
3	$(10, 11)$	1	double	$(4, 2)$	$16 \Rightarrow 16G$
4	$(16, 4)$	1	double	$(8, 4)$	$32 \equiv 13 \Rightarrow 13G$
5	$(0, 6)$			$(16, 8)$	$64 \equiv 7 \Rightarrow 7G$

We have a collision $R_5 = R_1 = (0, 6)$ (a tail of length 1 and a cycle of length 4, the shape of Figure 3.3). So $1 \cdot G + 1 \cdot Q = 16G + 8Q$, i.e. $-15G = 7Q$, so $4 \equiv 7k \pmod{19}$ and $k = 4 \cdot 7^{-1} = 4 \cdot 11 = 44 \equiv 6$. The logarithm is recovered without ever storing more than a few points (in a real run one uses Floyd's or Brent's cycle detection, or distinguished points, to avoid storing the trail).

Theorem 3.9 (Birthday paradox / expected rho length)**.** *If $\mathcal{W}$ behaves like a random function on a set of size r, the expected length of the tail plus cycle, i.e. the number of steps before the first repetition, is $\sqrt{\pi r/2} + \mathcal{O}(1)$ (tail $\approx \sqrt{\pi r/8}$, cycle $\approx \sqrt{\pi r/8}$). Floyd's detection costs an additional constant factor (≈ 3).*

Practical improvements.

1. *Parallel collision search (van Oorschot–Wiener):* m processors each walk from random starting points until they hit a *distinguished point* (e.g. x-coordinate with d leading zero bits) which they report to a central server; the server detects collisions between trails. Expected time: $\approx \sqrt{\pi r/2}/m$ steps per processor, memory only for distinguished points.
2. *Negation map:* work in equivalence classes $\{R, -R\}$ (represented by the point with smaller y); this cuts the search space by 2 and gives a speed-up $\sqrt{2}$ (with careful handling of fruitless 2-cycles).
3. *Frobenius and other automorphisms:* for Koblitz curves (binary) the Frobenius gives classes of size $2m$, with a $\sqrt{2m}$ speed-up (Gallant–Lambert–Vanstone; Wiener–Zuccherato).

Definition 3.10 (Rho-security)**.** For a group of prime order r, we call $\log_2 \sqrt{\pi r/4}$ the *rho-security* in bits (the $\pi/4$ accounts for the negation map).

For $r \approx 2^{252}$ (Curve25519/Ed25519): $\approx 2^{125.8}$; for $r \approx 2^{256}$ (P-256, secp256k1): $\approx 2^{127.8}$.

Remark 3.11 (Records). The largest ECDLP instances solved by Pollard-rho-type algorithms have size of roughly 110–120 bits of group order (e.g. a 112-bit prime-field curve in 2009 on a cluster of PlayStation 3 consoles, and binary Koblitz-curve challenges of similar size using GPU/FPGA clusters; consult the literature for current records). Each additional bit doubles the effort of the square root, so a 256-bit curve is far outside the reach of classical computing.

3.3 Why not index calculus?

In $\mathbb{F}_p^\times$, index calculus works because integers factor into small primes: relations between a *factor base* of small primes are accumulated and a linear system modulo $p - 1$ is solved. For $E(\mathbb{F}_p)$ there is *no* natural notion of "smooth point"; no known lifting to a characteristic-zero object with a good size function exists (the points of $E(\mathbb{Q})$ have heights, but reduction mod p destroys it; this is related to the failure of *lifting* approaches).

Semaev's summation polynomials (2004). For $E : y^2 = x^3 + ax + b$ and $m \geq 2$ there exist symmetric polynomials $S_m(X_1, \ldots, X_m) \in \mathbb{F}_q[X_1, \ldots, X_m]$, of degree 2^{m-2} in each variable, such that

$$S_m(x_1, \ldots, x_m) = 0 \iff \exists\, y_i \text{ with } (x_i, y_i) \in E(\overline{\mathbb{F}}_q) \text{ and } \pm P_1 \pm \cdots \pm P_m = \mathcal{O}.$$

For $m = 3$: $S_3 = (X_1 - X_2)^2 X_3^2 - 2\big((X_1 + X_2)(X_1X_2 + a) + 2b\big)X_3 + \big((X_1X_2 - a)^2 - 4b(X_1 + X_2)\big)$. To decompose a point R as a sum of m "factor-base" points (those whose x-coordinate lies in a

subspace V of $\mathbb{F}_q$ of dimension n/m over $\mathbb{F}_p$ when $q = p^n$), one solves $S_{m+1}(x(R), X_1, \dots, X_m) = 0$ with $X_i \in V$, a multivariate polynomial system of degree $\approx 2^{m-1}$ in nm variables. In prime fields $n = 1$ and no such subspace exists, and the complexity of the Gröbner step kills the algorithm: *no subexponential index calculus is known for* $E(\mathbb{F}_p)$.

Theorem 3.12 (Gaudry 2009; Diem 2011)**.** *For E over $\mathbb{F}_{q^n}$ with $n \geq 3$ fixed,* ECDLP *can be solved (heuristically/rigorously in various regimes) in time $\widetilde{\mathcal{O}}\big(q^{2-2/n}\big)$, which beats the generic $\mathcal{O}(q^{n/2})$ for $n \geq 3$.*

Hence curves over extension fields $\mathbb{F}_{q^n}$ with $n \geq 3$ should be avoided, as well as the special case of Weil descent:

Theorem 3.13 (Frey's Weil descent; Gaudry–Hess–Smart (GHS), 2002)**.** *For $E/\mathbb{F}_{2^{\ell m}}$ $(\ell, m > 1)$ certain curves can be mapped (Weil restriction + covering) to the Jacobian of a curve C of genus g over $\mathbb{F}_{2^\ell}$, where index calculus on hyperelliptic Jacobians is faster than $\sqrt{r}$. For this reason NIST binary curves use* prime *extension degrees $m \in \{163, 233, 283, 409, 571\}$.*

Remark 3.14. It is still open whether ECDLP over $\mathbb{F}_p$ admits a subexponential algorithm. This is among the most important open questions in the area: either result (algorithm or proof of generic hardness) would be spectacular.

3.4 Special attacks on weak curves

3.4.1 Anomalous curves (Smart; Satoh–Araki; Semaev)

Theorem 3.15 (Smart 1999)**.** *If $E/\mathbb{F}_p$ has $\#E(\mathbb{F}_p) = p$ ("anomalous", trace $t = 1$), then* ECDLP *in $E(\mathbb{F}_p)$ can be solved in polynomial time (in $\log p$).*

Sketch. Let $\tilde{E}/\mathbb{Q}_p$ be a lift of E (lift a, b arbitrarily to $\mathbb{Z}_p$), and lift $P, Q \in E(\mathbb{F}_p)$ to $\tilde{P}, \tilde{Q} \in \tilde{E}(\mathbb{Q}_p)$ (Hensel). Since $\#E(\mathbb{F}_p) = p$, $p\tilde{P}$ and $p\tilde{Q}$ reduce to $\mathcal{O}$, i.e. lie in the subgroup $\tilde{E}_1(\mathbb{Q}_p)$ of points reducing to $\mathcal{O}$. On this subgroup the *formal logarithm* $\ell : \tilde{E}_1(\mathbb{Q}_p) \to p\mathbb{Z}_p$ is a homomorphism computed by a power series with p-adic convergence, and $\tilde{E}_1/\tilde{E}_2 \cong \mathbb{F}_p^+$ (additive group!). If $Q = kP$ then $\ell(p\tilde{Q}) \equiv k\,\ell(p\tilde{P}) \pmod{p^2}$, so $k \equiv \frac{\ell(p\tilde{Q})/p}{\ell(p\tilde{P})/p} \pmod{p}$ (the lifts $\tilde{P}, \tilde{Q}$ must be chosen generically so that $\ell(p\tilde{P}) \not\equiv 0 \pmod{p^2}$). □

Thus anomalous curves are *explicitly excluded* by all standards (SafeCurves: trace $\neq 1$).

3.4.2 The MOV / Frey–Rück attack

Let r be a prime divisor of $\#E(\mathbb{F}_q)$, $r \neq$ char. The *embedding degree* k is the smallest integer with $r \mid q^k - 1$. Using the Weil or Tate pairing (introduced in Lecture 5) ECDLP in $\langle P \rangle \subset E(\mathbb{F}_q)$ reduces to the DLP in $\mathbb{F}_{q^k}^\times$, where index calculus (NFS) applies in time $L_{q^k}[1/3]$.

Theorem 3.16 (Menezes–Okamoto–Vanstone 1993; Frey–Rück 1994)**.** *There is a probabilistic algorithm that reduces* ECDLP *in $\langle P \rangle$ to a DLP in $\mu_r \subset \mathbb{F}_{q^k}^\times$ using $\mathcal{O}(\mathrm{poly}(k \log q))$ operations. It is efficient (i.e. yields a subexponential attack) if k is small, e.g. $k \leq \mathcal{O}((\log q)^2)$.*

Example 3.17 (Supersingular curves)**.** For supersingular $E/\mathbb{F}_p$ $(p > 3)$: $\#E = p + 1$, so for $r \mid p + 1$: $p \equiv -1 \pmod{r}$ and $k = 2$. Then the problem reduces to a DLP in $\mathbb{F}_{p^2}^\times$ of size about

p^2, solved in $L_{p^2}[1/3]$. For a 256-bit p this is a 512-bit field: broken. (In general supersingular curves have $k \in \{1, 2, 3, 4, 6\}$.) Note the case $r = p$ for anomalous and the case $k = 1$ ($r \mid p - 1$) giving the Frey–Rück attack in $\mathbb{F}_p^\times$.

For *random* curves the embedding degree is huge: $k \approx r$ with overwhelming probability (Balasubramanian–Koblitz), so MOV is harmless; SafeCurves requires $k \geq (r - 1)/100$.

3.4.3 Other considerations: CM discriminant, twists, rigidity

- **CM field discriminant.** Curves whose endomorphism ring has a small discriminant $|D|$ (e.g. $j = 0$ and $D = -3$, $j = 1728$ and $D = -4$) have extra structure. No attack is known beyond speed-ups (e.g. GLV, $\sqrt{6}$ in the rho method for $D = -3$, due to automorphisms of order 6), but conservative designers require $|D| > 2^{100}$.
- **Twist security.** The quadratic twist E' of E has order $2q + 2 - \#E$. Points on E' may be accepted by implementations that only check x-coordinates (X25519). A secure curve has a twist with a large prime factor (Curve25519: twist order is $4\cdot$ a 253-bit prime).
- **Rigidity / NUMS.** The choice of constants should be verifiable ("nothing up my sleeve"): e.g. Curve25519 is the smallest A satisfying security criteria; secp256k1 has $a = 0, b = 7$. In contrast, the NIST P-curves' seeds `c49d3608...` remain unexplained and Brainpool curves use derivation from digits of π and e.

Example 3.18 (Twist of $E : y^2 = x^3 + 2x + 2$ over $\mathbb{F}_{17}$)**.** Every $x \in \mathbb{F}_{17}$ is the x-coordinate of a point on E *or* on its quadratic twist E' (or $f(x) = 0$). From the table of Theorem 2.8, 9 values of x have $\chi(f(x)) = +1$ (they give the 18 affine points of E) and 8 values have $\chi(f(x)) = -1$ (they give 16 affine points of E'). So $\#E' = 16 + 1 = 17 = 2 \cdot 17 + 2 - 19$, as the formula $\#E' = 2q + 2 - \#E$ predicts. Here 17 is prime, so this toy curve is twist-secure.

Table 3.1: Summary of attacks on ECDLP and the corresponding mitigation in curve selection.

Attack	Condition (weakness)	Mitigation
BSGS, Pollard rho	always applicable	$r \geq 2^{2s}$ for s-bit security
Pohlig–Hellman	$\#E$ has only small primes	$\#E = hr$, r large prime
Smart (anomalous)	$\#E(\mathbb{F}_p) = p$	check $t \neq 1$
MOV / Frey–Rück	small embedding degree k	check $r \nmid q^k - 1$ for $k \leq 100$ or $k \geq (r - 1)/100$
Weil descent (GHS)	$E/\mathbb{F}_{q^m}$, m composite	prime-degree extensions or prime fields
Gaudry–Diem	$E/\mathbb{F}_{q^n}$, $n \geq 3$	use prime fields
Invalid curve / small subgroup (Lecture 4)	implementation does not validate points	validate inputs, cofactor clearing

3.5 Putting it together: choosing a curve

A modern "safe" prime-field curve is chosen as follows (cf. Bernstein–Lange, *SafeCurves*).

1. Choose a prime p with fast arithmetic (e.g. Mersenne-like $2^{255} - 19$, $2^{448} - 2^{224} - 1$) and a curve model with fast complete formulas (Montgomery, Edwards).
2. Compute $\#E(\mathbb{F}_p)$ by SEA and require $\#E = h \cdot r$ with r prime, $h \in \{4, 8\}$ for Montgomery/Edwards (or $h = 1$ for Weierstrass), the same for the quadratic twist.

3. Check rho-security $\geq 2^{100}$; $t \neq 1$; embedding degree large; CM discriminant large.
4. Make the choice reproducible: smallest coefficient meeting the criteria.

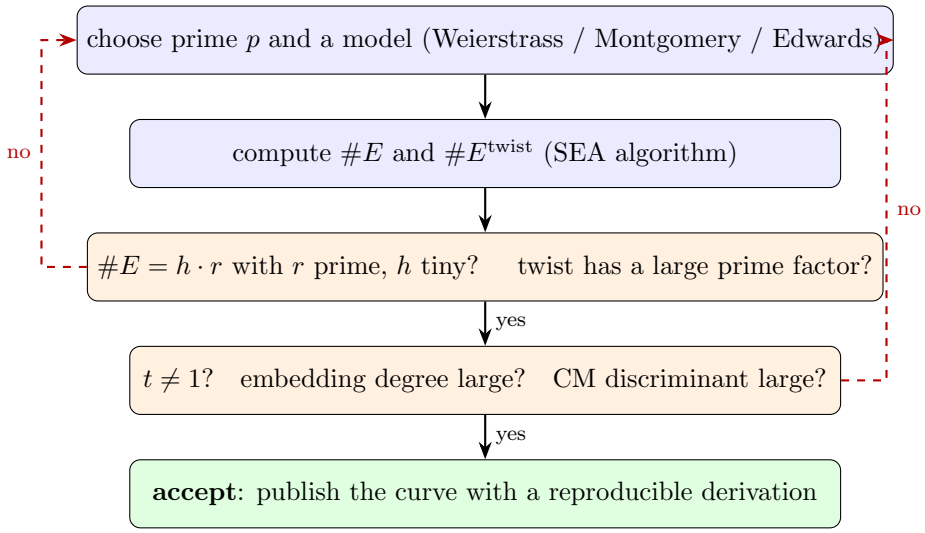


Figure 3.4: A schematic curve-selection procedure. If any security check fails, a parameter (for instance the coefficient A or b) is changed and the search restarts.

Exercises for Lecture 3

Exercise 3.1 (BSGS by hand). In $E : y^2 = x^3 + 2x + 2$ over $\mathbb{F}_{17}$ with $G = (5, 1)$ of order 19, solve $Q = kG$ for $Q = (13, 10)$ using BSGS with $m = 5$ (answer: $k = 11$). Build the baby-step table $\{jG : 0 \leq j < 5\}$ using Table 1.2 and the giant steps $Q - i(5G)$.

Exercise 3.2 (Pohlig–Hellman). Let $E/\mathbb{F}_p$ have $\#E = 2^4 \cdot 3^2 \cdot 7 \cdot r$ with $r \approx 2^{20}$ prime. Estimate the cost of ECDLP in the full group using Theorem 3.6, compared with $\sqrt{\#E}$.

Exercise 3.3 (Birthday bound). Show that if one draws m elements uniformly at random with replacement from a set of size n, the probability of no collision is $\leq \exp(-m(m-1)/(2n))$ and hence a collision is expected after $\approx \sqrt{2n \ln 2}$ draws (median) or $\sqrt{\pi n/2}$ draws (mean).

Exercise 3.4 (Rho walk invariant). Verify that in Algorithm 6 the invariant $R = aG + bQ$ holds for each of the three branches of Step. Explain why the factor 2 branch ($R \mapsto 2R$, $(a, b) \mapsto (2a, 2b)$) is included, and what could go wrong if only branches S_1 and S_3 were used.

Exercise 3.5 (Distinguished points). In the van Oorschot–Wiener method assume each walk ends at a distinguished point after an expected $1/\theta$ steps (θ the proportion of distinguished points). Show that the expected memory is $\mathcal{O}(\theta\sqrt{r})$ and the wasted work is $\mathcal{O}(1/\theta)$ per collision. How should θ be chosen?

Exercise 3.6 (Anomalous curve, tiny case). In Theorem 2.7 replace $\mathbb{F}_5$ by $\mathbb{F}_{11}$ and consider $E : y^2 = x^3 + x + 5$, which is anomalous ($\#E(\mathbb{F}_{11}) = 11$). Why is Smart's attack hardly interesting for $p = 11$, and what would one need to implement to make it nontrivial (hint: p-adic lifts mod p^2; division by p)?

Exercise 3.7 (MOV for a supersingular curve). Let $p = 23$ and $E : y^2 = x^3 + x$ over $\mathbb{F}_{23}$ ($\#E = 24$). Find the embedding degree for $r = 3$ (note $3 \mid p + 1 = 24$) and for $r = 2$. Compute the smallest k with $3 \mid 23^k - 1$.

Exercise 3.8 (Security level). Compute the rho-security of a curve with $\#E = h \cdot r$ and $r = 2^{224}$-ish. How many bits of security does a prime-order curve over a 192-bit field provide? Compare with Table 1.1.

Exercise 3.9 (Challenge: Twist order). Show that if $E : y^2 = f(x)$ over $\mathbb{F}_p$ has $\#E(\mathbb{F}_p) = p+1-t$, its quadratic twist $E^{(d)} : dy^2 = f(x)$ (with d non-square) has $\#E^{(d)}(\mathbb{F}_p) = p + 1 + t$. (*Hint:* for each x, $\chi(f(x))$ points on one and $-\chi(f(x))$ on the other.)

Chapter 4

Protocols, Standards and Implementation Security

Learning goals

After this lecture students should be able to

- describe and analyse ECDH, ECIES, ECDSA, Schnorr signatures and EdDSA, including correctness proofs;
- recover a private key from nonce reuse or biased nonces and explain deterministic nonce generation;
- mount (on paper) an invalid-curve attack and a small-subgroup attack, and state the countermeasures;
- identify the main side-channel threats to scalar multiplication and the standard countermeasures;
- orient themselves among the standard curves and the lessons from Dual_EC_DRBG.

Throughout this lecture the *domain parameters* are $\mathcal{D} = (q, E, G, r, h)$ as in Theorem 2.13: G has prime order r and the cofactor is h. A *key pair* is (d, Q) with $d \xleftarrow{\$} \{1, \ldots, r-1\}$ and $Q = dG$.

4.1 Key agreement: ECDH

Theorem 4.1 (Correctness and security of ECDH)**.** *Both parties obtain $S = abG$. Passive security (indistinguishability of* $\mathrm{KDF}(x(S))$ *from random) follows from* DDH *(or from* CDH *in the random-oracle model, if the KDF is modelled as a random oracle).*

Example 4.2 (Toy ECDH on $E(\mathbb{F}_{17})$)**.** Public parameters: $y^2 = x^3 + 2x + 2$ over $\mathbb{F}_{17}$, $G = (5, 1)$, order 19. Alice picks $a = 3$ and sends $A = 3G = (10, 6)$. Bob picks $b = 11$ and sends $B = 11G = (13, 10)$ (both from Table 1.2). Alice computes $aB = 3 \cdot (13, 10)$ and Bob computes $bA = 11 \cdot (10, 6)$; both equal $33G = 14G = (9, 1)$. The shared key is derived from the x-coordinate 9. An eavesdropper sees G, A, B and must find $(9, 1)$; here she can simply read $a = 3$ off the table, which is exactly what is impossible for a 256-bit curve.

Warning

Unauthenticated ECDH is secure only against *passive* adversaries: a man in the middle sends $A' = a'G$ to Bob and $B' = b'G$ to Alice. Practical protocols (TLS 1.3, Noise, Signal)

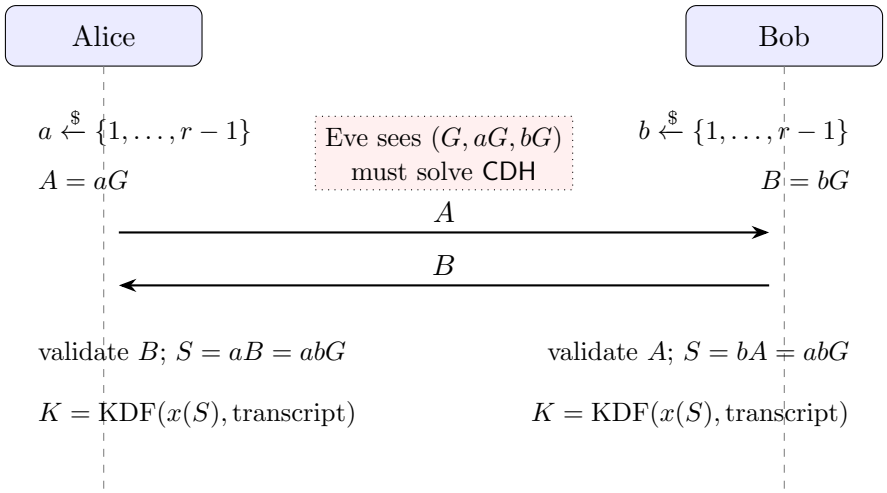


Figure 4.1: Elliptic-curve Diffie–Hellman. The key is derived from the x-coordinate of the shared point via a key-derivation function (KDF); the public values must be validated.

authenticate the exchange by signatures, MACs under pre-shared keys, or by combining static and ephemeral keys (e.g. 3DH/X3DH). Reusing a static ECDH key against an adversary able to submit invalid points is dangerous (Section 4.5).

Cofactor and X25519. With $h > 1$ the private key is multiplied by h ("cofactor ECDH") or, in X25519, "clamped": a 32-byte string k is treated as the integer with bit 255 cleared, bit 254 set, and the three lowest bits cleared, i.e. $k \in 2^{254} + 8\mathbb{Z} \cap [0, 2^{255})$. Clearing the low bits makes k a multiple of the cofactor 8, which kills any small-order component of the peer's point; setting bit 254 fixes the length of the ladder (constant time).

Example 4.3 (RFC 7748 test vector)**.** Alice's private key (hex), with the base point $u = 9$, gives the public key below (little-endian encodings; reproduced by the code in Lecture B).

```
private: 77076d0a7318a57d3c16c17251b26645df4c2f87ebc0992ab177fba51db92c2a
public : 8520f0098930a754748b7ddcb43ef75a0dbf3a0d26381af4eba4a98eaa9b4e6a
```

4.2 Public-key encryption: ElGamal and ECIES

4.2.1 EC-ElGamal encryption, step by step

What you need to know beforehand – only four facts

1. The points of an elliptic curve E over $\mathbb{F}_p$ (plus a point at infinity $\mathcal{O}$) form a group written *additively*: we can add points $P + Q$, negate $-(x, y) = (x, -y)$, and subtract $P - Q = P + (-Q)$. The neutral element is $\mathcal{O}$.
2. G is a public base point whose multiples $G, 2G, 3G, \ldots, nG = \mathcal{O}$ form a cyclic group of (prime) order n. Computing kG is fast (double-and-add). Finding k from kG is the hard problem (ECDLP).
3. Scalar multiples commute: $k(dG) = d(kG) = (kd)G$.
4. ECDH: Alice sends $A = aG$, Bob sends $B = bG$, and both compute the same point $aB = bA = abG$.

That is all. If you remember ECDH, ElGamal will take ten minutes.

The big idea: ElGamal = ECDH + a one-time pad. In ECDH the two parties end up with a shared secret point abG and then turn it into a key. ElGamal does the same thing but with the roles arranged for *encryption*:

- Bob publishes a *long-term* public key $Q = dG$ (it plays the role of "$B = bG$" in ECDH; d is his private key).
- Alice, who wants to send a message M, picks a fresh *ephemeral* secret k and sends $C_1 = kG$ (the role of "$A = aG$").
- Now Alice and Bob *both* know the shared point $S = kQ = dC_1 = kdG$, exactly as in ECDH.
- Alice uses S as a one-time mask: she *adds* it to the message, $C_2 = M + S$. Only someone who knows S can undo this.

The ciphertext is the pair (C_1, C_2). Because Alice's secret k is fresh every time, the mask S is different every time, like a fresh one-time pad.

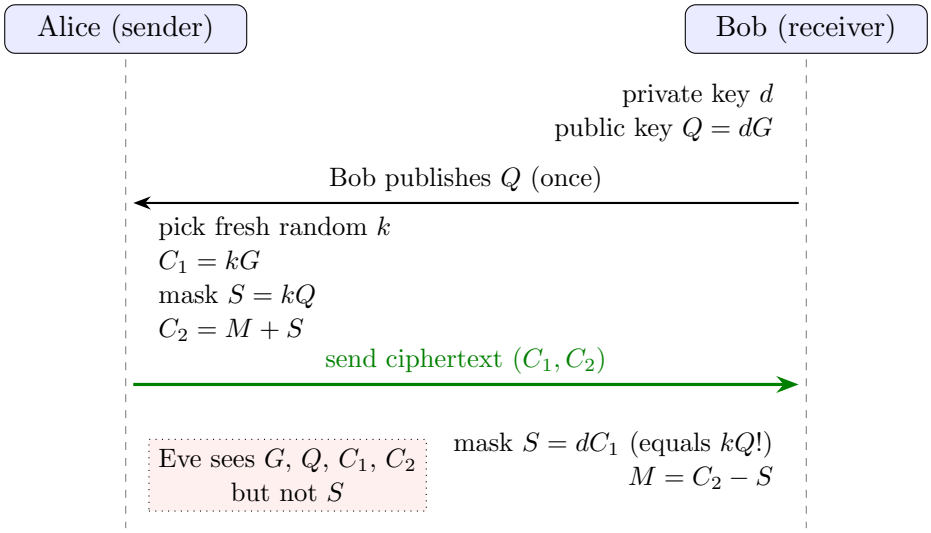


Figure 4.2: EC-ElGamal as ECDH plus masking. The point $C_1 = kG$ is Alice's ECDH public value; the shared point $S = kQ = dC_1$ is the ECDH secret; it is used to hide M by adding it.

Dictionary: from classical ElGamal to the elliptic-curve version. If you know ElGamal in $\mathbb{Z}_p^\times$ (as in Paar's book), the elliptic-curve version is the same scheme with the group written additively: *multiplication becomes addition* and *exponentiation becomes scalar multiplication.*

	Classical ElGamal in $\mathbb{Z}_p^\times$	EC-ElGamal on $E(\mathbb{F}_p)$
public base	g	G
private key	d	d
public key	$\beta = g^d$	$Q = dG$
message	integer $m \in \mathbb{Z}_p^\times$	a *point* $M \in \langle G \rangle$
ephemeral secret	k	k
first part	$c_1 = g^k$	$C_1 = kG$
mask	β^k	kQ
second part	$c_2 = m \cdot \beta^k$	$C_2 = M + kQ$
decryption	$m = c_2 \cdot (c_1^d)^{-1}$	$M = C_2 - dC_1$

The scheme

Definition 4.4 (EC-ElGamal)**.** Public parameters: curve E over $\mathbb{F}_p$, point G of prime order n.

1. **Key generation (Bob).** Choose $d \xleftarrow{\$} \{1, \dots, n-1\}$ uniformly. Public key $Q = dG$; private key d.
2. **Encryption (Alice)** of a message point $M \in \langle G \rangle$: choose a fresh random $k \xleftarrow{\$} \{1, \dots, n-1\}$ and output $(C_1, C_2) = (kG,\ M + kQ)$.
3. **Decryption (Bob)**: compute $M = C_2 - dC_1$.

Theorem 4.5 (Correctness)**.** *Decryption returns M.*

Proof. $dC_1 = d(kG) = k(dG) = kQ$, hence $C_2 - dC_1 = (M + kQ) - kQ = M$. □

Cost. Encryption needs two scalar multiplications (kG and kQ) and one point addition; decryption needs one scalar multiplication and one subtraction. The ciphertext consists of *two* points, i.e. it is about twice as long as the message point.

A problem you must solve first: messages are not points

The scheme encrypts a *point* M, but we want to encrypt numbers, text, keys. The classical (Koblitz) fix is to embed a small integer m into a point:

Koblitz embedding with parameter κ

To encode $m \in \{0, 1, \dots, \lfloor p/\kappa \rfloor - 1\}$: try $x = \kappa m,\ \kappa m + 1,\ \dots,\ \kappa m + \kappa - 1$ and take the first x for which $x^3 + ax + b$ is a square modulo p (test with Euler's criterion); let y be a square root (choose the smaller one). Then $M_m = (x, y)$. To decode, output $m = \lfloor x/\kappa \rfloor$ (only the x-coordinate is needed). Each trial succeeds with probability about $\frac{1}{2}$, so encoding fails with probability about $2^{-\kappa}$; real systems take $\kappa \approx 30$–60.

We will work on a curve small enough to do everything by hand.

The toy curve. $E: y^2 = x^3 + 2x + 2$ over $\mathbb{F}_{17}$, with base point $G = (5, 1)$ of prime order $n = 19$. All multiples of G (you will use this table all the time):

k	1	2	3	4	5	6	7	8	9
kG	$(5,1)$	$(6,3)$	$(10,6)$	$(3,1)$	$(9,16)$	$(16,13)$	$(0,6)$	$(13,7)$	$(7,6)$
k	10	11	12	13	14	15	16	17	18
kG	$(7,11)$	$(13,10)$	$(0,11)$	$(16,4)$	$(9,1)$	$(3,16)$	$(10,11)$	$(6,14)$	$(5,16)$

Remember $-kG = (19-k)G$, i.e. negation flips $y \mapsto -y$; for instance $-(6,3) = (6,14)$. And $19G = \mathcal{O}$.

Embedding table for $\kappa = 3$, so $m \in \{0,\dots,4\}$ (the squares modulo 17 are $1,2,4,8,9,13,15,16$; one checks that $f(x) = x^3 + 2x + 2 \bmod 17$ is a square exactly for $x = 0,3,5,6,7,9,10,13,16$):

m	0	1	2	3	4
x tried	0 ✓	3 ✓	6 ✓	9 ✓	12 (no), 13 ✓
M_m	$(0,6)$	$(3,1)$	$(6,3)$	$(9,1)$	$(13,7)$
$=$	$7G$	$4G$	$2G$	$14G$	$8G$

Worked example 1: encrypt and decrypt

Example 4.6 (Bob's key pair)**.** Bob chooses the private key $d = 7$ and computes $Q = 7G = (0,6)$ (table). He publishes Q.

Example 4.7 (Alice encrypts the message $m = 4$)**.** Alice wants to send $m = 4$. She embeds it: $M = (13,7) = 8G$. She chooses the fresh random nonce $k = 3$ and computes

$$\begin{aligned} C_1 &= kG = 3G = (10,6), \\ S &= kQ = 3\cdot(0,6) = 3\cdot 7G = 21G = 2G = (6,3) \qquad \text{(the mask)}, \\ C_2 &= M + S = 8G + 2G = 10G = (7,11). \end{aligned}$$

She sends $(C_1, C_2) = \big((10,6),(7,11)\big)$.

Example 4.8 (Bob decrypts)**.** Bob computes the mask from C_1 with his private key: $S' = dC_1 = 7\cdot(10,6) = 7\cdot 3G = 21G = 2G = (6,3)$. This is the *same* point S that Alice used. He subtracts it:

$$M = C_2 - S' = (7,11) + \big(-(6,3)\big) = (7,11) + (6,14).$$

The slope is $\lambda = \frac{14-11}{6-7} = \frac{3}{-1} \equiv 14$, so $x_3 = 14^2 - 7 - 6 = 183 \equiv 13$ and $y_3 = 14(7-13) - 11 = -95 \equiv 7$, giving $M = (13,7)$. Decoding: $m = \lfloor 13/3 \rfloor = 4$. ✓

What does Eve see? Only $G = (5,1)$, $Q = (0,6)$, $C_1 = (10,6)$, $C_2 = (7,11)$. To recover M she needs the mask $S = kQ = (6,3)$. Computing it needs k (from $C_1 = kG$, an ECDLP) or d (from $Q = dG$, an ECDLP). On this toy curve she simply reads $k = 3$ off the table; on a 256-bit curve that is infeasible. Computing S from (G, Q, C_1) without either secret is exactly the Diffie–Hellman problem CDH.

Example 4.9 (Same message, new ciphertext)**.** Alice encrypts $m = 4$ again with a new nonce $k = 11$: $C_1 = 11G = (13,10)$, $S = 11Q = 77G = G = (5,1)$ (as $77 = 4\cdot 19 + 1$), $C_2 = 8G + G = 9G = (7,6)$. The ciphertext $\big((13,10),(7,6)\big)$ looks completely different, yet Bob decrypts it: $dC_1 = 7\cdot 11G = 77G = G$ and $9G - G = 8G = (13,7)$. ✓This randomisation is essential: if encryption were deterministic, anyone could encrypt guesses and compare.

Why is it secure?

Theorem 4.10 (Semantic security)**.** *If the Decisional Diffie–Hellman problem is hard in $\langle G \rangle$, EC-ElGamal is IND-CPA secure: an adversary who sees the public key and a ciphertext learns nothing about which of two chosen messages was encrypted.*

Idea. The adversary sees $Q = dG$, $C_1 = kG$ and $C_2 = M + S$. If $S = kdG$ were replaced by a uniformly random point R, then $C_2 = M + R$ would be uniformly distributed whatever M is, exactly as with a one-time pad, and would carry no information about M. DDH hardness says that the genuine $S = kdG$ cannot be distinguished from such a random R given (G, dG, kG). So the real ciphertext is indistinguishable from one that hides M perfectly. □

Intuition. The point S is a "fresh one-time pad" which only Alice (who knows k) and Bob (who knows d) can compute. This is the same fact that makes ECDH secure.

Three pitfalls (and what they teach)

Pitfall 1: never reuse the nonce k. If two messages are encrypted with the same k, the same mask S is added twice, like reusing a one-time pad.

Example 4.11 (Nonce reuse)**.** Alice encrypts $M_1 = (13, 7) = 8G$ and $M_2 = (6, 3) = 2G$, both with $k = 3$. The mask is $S = 2G$ in both cases, so C_1 is the same and $C_2 = 8G + 2G = 10G = (7, 11)$, $C_2' = 2G + 2G = 4G = (3, 1)$. Eve notices the repeated $C_1 = (10, 6)$ and computes $C_2 - C_2' = (7, 11) - (3, 1) = 6G = (16, 13) = M_1 - M_2$. If she also knows (or guesses) one plaintext, say M_1, she gets the mask $S = C_2 - M_1 = (6, 3)$ and then $M_2 = C_2' - S = (3, 1) - (6, 3) = 2G = (6, 3)$. All without the private key.

The nonce must be uniformly random and fresh for *every* encryption (a bad random-number generator is as bad as reuse).

Pitfall 2: ciphertexts can be modified (malleability).

Example 4.12 (An attacker edits the plaintext)**.** Take Alice's ciphertext $\big((10, 6), (7, 11)\big)$ of $m = 4$ from Theorem 4.7. Mallory replaces C_2 by $C_2 + G = (7, 11) + (5, 1) = 11G = (13, 10)$ and forwards $\big((10, 6), (13, 10)\big)$. Bob decrypts $11G - 2G = 9G = (7, 6)$ and decodes $m = \lfloor 7/3 \rfloor = 2$. The message changed from 4 to 2, and Bob notices nothing, although Mallory never learned the key.

In general adding a point T to C_2 changes the plaintext to $M + T$. This is why EC-ElGamal alone is not secure against *active* attackers (it is not IND-CCA2). Practical systems use ECIES (Lecture 4), which authenticates the ciphertext.

Pitfall 3 (a feature in disguise): the scheme is additively homomorphic.

Example 4.13 (Adding encrypted messages)**.** Encrypt $M_1 = (13, 7) = 8G$ with $k = 3$: $(C_1, C_2) = \big((10, 6), (7, 11)\big)$. Encrypt $M_2 = (3, 1) = 4G$ (this is $m = 1$) with $k = 11$: $(C_1', C_2') = \big((13, 10), (9, 16)\big)$. Add the ciphertexts component-wise:

$$\big(C_1 + C_1',\ C_2 + C_2'\big) = \big((10, 6) + (13, 10),\ (7, 11) + (9, 16)\big) = \big((9, 1),\ (3, 16)\big).$$

Bob decrypts: $d(9,1) = 7 \cdot 14G = 98G = 3G = (10,6)$, and $(3,16) - (10,6) = 15G - 3G = 12G = (0,11)$. And indeed $M_1 + M_2 = 8G + 4G = 12G = (0,11)$. ✓
So one can add plaintexts *without decrypting*. This is used in electronic voting: encode a vote $v \in \{0,1\}$ as the point vG ("exponential ElGamal"), add all encrypted ballots, decrypt once to get $(\sum v)G$ and recover the tally $\sum v$ by trying $0G, 1G, 2G, \ldots$ (cheap for small tallies).

Practical remarks.

- Before using a received ciphertext, check that C_1, C_2 are valid points of the curve in the correct subgroup (otherwise invalid-curve attacks, Lecture 4); reject $C_1 = \mathcal{O}$.
- The message M must lie in the subgroup $\langle G \rangle$; on prime-order curves that is every point.
- The embedded message is short (about 250 bits on a 256-bit curve). Real systems encrypt long data with a symmetric cipher and use ElGamal only to transport the key: this is **ECIES**: $C_1 = kG$, key $= \mathrm{KDF}(x(kQ))$, then authenticated encryption of the data. ECIES avoids both the embedding problem and malleability.

Python implementation (toy curve)

The following code reproduces every number in this section (it uses the `Curve` class of the appendix with $E(\mathbb{F}_{17})$). It is for learning only.

```python
import secrets
from ecc_toy import Curve # class Curve from the appendix

E = Curve(17, 2, 2, G=(5, 1), n=19) # y^2 = x^3 + 2x + 2 over F_17, order of G = 19
KAPPA = 3

def keygen():
    d = secrets.randbelow(E.n - 1) + 1 # private key
    return d, E.mul(d, E.G) # (d, Q = dG)

def embed(m, kappa=KAPPA): # integer -> curve point (Koblitz)
    for j in range(kappa):
        x = kappa * m + j
        rhs = (x**3 + E.a * x + E.b) % E.p
        ys = [y for y in range(E.p) if y * y % E.p == rhs]
        if ys: return (x, min(ys))
    raise ValueError("embedding failed")

def unembed(P, kappa=KAPPA): # curve point -> integer
    return P[0] // kappa

def encrypt(Q, M, k=None):
    k = k or secrets.randbelow(E.n - 1) + 1 # FRESH random nonce
    return E.mul(k, E.G), E.add(M, E.mul(k, Q)) # (C1, C2) = (kG, M + kQ)

def decrypt(d, C):
    C1, C2 = C
    return E.add(C2, E.neg(E.mul(d, C1))) # M = C2 - d*C1

d, Q = 7, E.mul(7, E.G)
M = embed(4) # (13, 7)
C = encrypt(Q, M, k=3) # ((10, 6), (7, 11))
print(C, decrypt(d, C), unembed(decrypt(d, C))) # ... (13, 7) 4
```

One-page summary

- **Keys:** private d; public $Q = dG$.
- **Encrypt** M**:** pick random k; send $\big(kG,\ M + kQ\big)$.
- **Decrypt:** $M = C_2 - dC_1$, because $dC_1 = kQ$ (the ECDH secret).
- **Security:** kQ is a fresh one-time pad; secure if DDH is hard.
- **Must do:** fresh random k every time; embed messages as points; validate points.
- **Limits:** malleable (edit C_2 edits M), ciphertext twice the size, short messages $\Rightarrow$ use ECIES in practice.
- **Bonus:** additively homomorphic $\Rightarrow$ e-voting, private sums.

Exercises (with answers)

Exercise 4.1. With Bob's key $d = 7$, $Q = (0, 6)$ encrypt $M = (9, 1)$ (this is $m = 3$) using $k = 2$, then decrypt. *Answer:* $C_1 = 2G = (6, 3)$, $S = 2Q = 14G = (9, 1)$, $C_2 = 14G + 14G = 28G = 9G = (7, 6)$; ciphertext $\big((6, 3), (7, 6)\big)$. Decryption: $dC_1 = 14G$ and $9G - 14G = -5G = 14G = (9, 1)$.

Exercise 4.2. Decrypt $\big((13, 10), (0, 11)\big)$ with $d = 7$ and decode the message. *Answer:* $C_1 = 11G$, $dC_1 = 77G = G$, $M = 12G - G = 11G = (13, 10)$, so $m = \lfloor 13/3 \rfloor = 4$ (decoding only uses the x-coordinate).

Exercise 4.3. What happens if Alice chooses $k = 19$ (or $k = 0$)? *Answer:* $C_1 = \mathcal{O}$ and $S = \mathcal{O}$, so $C_2 = M$: the plaintext is sent in the clear. This is why k is drawn from $\{1, \dots, n - 1\}$.

Exercise 4.4. Two ciphertexts $\big((10, 6), (7, 11)\big)$ and $\big((10, 6), (3, 1)\big)$ are intercepted, and Eve knows the first plaintext is $(13, 7)$. Recover the second. *Answer:* the same C_1 means the same k; $S = (7, 11) - (13, 7) = (6, 3)$; $M_2 = (3, 1) - (6, 3) = (6, 3)$, i.e. $m = 2$.

Exercise 4.5. In an election with encrypted votes $v_i \in \{0, 1\}$ encoded as $v_i G$, explain how to obtain the tally without decrypting individual votes, and why it only works for small tallies. *Answer:* add all ciphertexts (component-wise), decrypt the sum once to get TG, $T = \sum v_i$; find T by testing $0G, 1G, \dots$ (a discrete logarithm of a small number: cheap only if T is at most about 2^{40}).

Exercise 4.6. Show that $(C_1, 2C_2)$ does *not* decrypt to $2M$, but $(2C_1, 2C_2)$ does. *Answer:* decrypting $(C_1, 2C_2)$ gives $2C_2 - dC_1 = 2M + kQ \neq 2M$, whereas $(2C_1, 2C_2)$ gives $2C_2 - 2dC_1 = 2M + 2kQ - 2kQ = 2M$.

4.2.2 ECIES / hashed ElGamal as a KEM

Algorithm 7 ECIES (KEM–DEM paradigm)

Input: recipient public key $Q = dG$, message m

1: $k \xleftarrow{\$} \mathbb{Z}_r^{\times}$; $R = kG$ ▷ ephemeral public key
2: $Z = hkQ$ (cofactor multiplication); abort if $Z = \mathcal{O}$
3: $(K_{\text{enc}}, K_{\text{mac}}) = \text{KDF}(x(Z) \,\|\, R)$
4: $c = \text{Enc}_{K_{\text{enc}}}(m)$; $\tau = \text{MAC}_{K_{\text{mac}}}(c)$
5: **return** (R, c, τ)

Decryption: the recipient computes $Z = h\,dR$, derives the keys, verifies τ and decrypts. Security (IND-CCA2) holds in the random-oracle model under the *gap* Diffie–Hellman assumption (Abdalla–Bellare–Rogaway DHIES). Modern standards: HPKE (RFC 9180).

4.3 Digital signatures

4.3.1 ECDSA

Algorithm 8 ECDSA

Sign(d, m):
$e \leftarrow H(m)$ (leftmost $\lceil \log_2 r \rceil$ bits, as an integer)
repeat
  $k \xleftarrow{\$} \{1, \ldots, r-1\}$; $(x_1, y_1) \leftarrow kG$; $\rho \leftarrow x_1 \bmod r$
  $s \leftarrow k^{-1}(e + \rho d) \bmod r$
until $\rho \neq 0$ and $s \neq 0$
return (ρ, s)
Verify$(Q, m, (\rho, s))$:
check $Q \in \langle G \rangle \setminus \{\mathcal{O}\}$, $1 \leq \rho, s \leq r-1$
$e \leftarrow H(m)$; $w \leftarrow s^{-1} \bmod r$; $u_1 \leftarrow ew$; $u_2 \leftarrow \rho w$ (all mod r)
$X \leftarrow u_1 G + u_2 Q$; **return** $[X \neq \mathcal{O}$ and $x(X) \bmod r = \rho]$

Theorem 4.14 (Correctness of ECDSA)**.** *If (ρ, s) is produced by the signing algorithm for key d and message m, then verification accepts.*

Proof. $u_1 G + u_2 Q = (e + \rho d) s^{-1} G = (e + \rho d) \cdot k(e + \rho d)^{-1} G = kG$, hence $x(X) \bmod r = \rho$. □

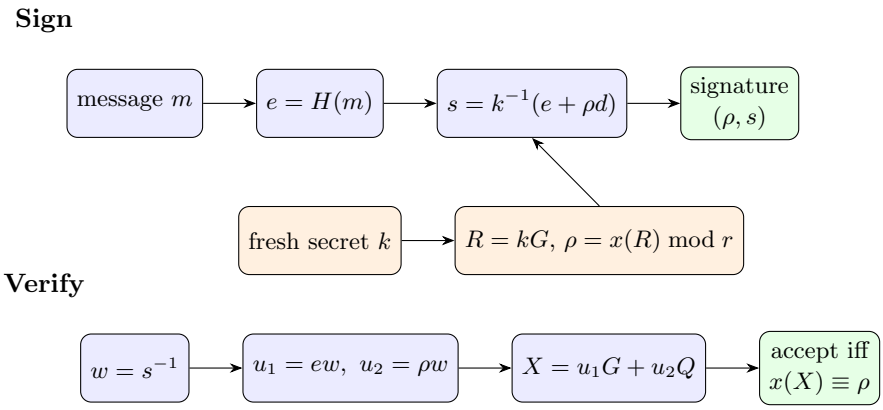


Figure 4.3: ECDSA signing and verification. The nonce k must be fresh and secret for every signature (see Theorem 4.16); verification recomputes kG from public data only.

Example 4.15 (Toy ECDSA on $y^2 = x^3 + 2x + 2$ over $\mathbb{F}_{17}$, $r = 19$)**.** Let $G = (5, 1)$, $d = 7$, $Q = 7G = (0, 6)$, hash value $e = 10$, nonce $k = 5$. Then $kG = 5G = (9, 16)$, so $\rho = 9$ and $s = 5^{-1}(10 + 9 \cdot 7) = 4 \cdot 73 \bmod 19$; as $5^{-1} = 4 \pmod{19}$, $73 \equiv 16$ and $s = 64 \equiv 7$. Signature $(\rho, s) = (9, 7)$. Verification: $w = 7^{-1} = 11$, $u_1 = 10 \cdot 11 \equiv 15$, $u_2 = 9 \cdot 11 \equiv 4 \pmod{19}$, and $15G + 4Q = 15G + 28G = 43G = 5G = (9, 16)$. Since $x = 9 = \rho$, accept.

4.3.2 Nonce misuse: why k is the crown jewel

Theorem 4.16 (Nonce reuse)**.** *If two signatures (ρ, s_1), (ρ, s_2) on distinct messages with hash values $e_1 \neq e_2$ share the same nonce k (visible as the same ρ), then*

$$k = \frac{e_1 - e_2}{s_1 - s_2}, \qquad d = \frac{s_1 k - e_1}{\rho} \pmod{r}.$$

Proof. $s_i = k^{-1}(e_i + \rho d)$, so $s_1 - s_2 = k^{-1}(e_1 - e_2)$, giving k; then $d = (s_1 k - e_1)\rho^{-1}$. (Note that $s_1 \neq s_2$ since $e_1 \neq e_2$.) □

Example 4.17. Continuing the toy example, sign a second hash $e_2 = 3$ with the same $k = 5$: $s_2 = 4(3 + 63) = 264 \equiv 17$. Then $k = (10 - 3)/(7 - 17) = 7 \cdot (-10)^{-1} = 7 \cdot 17 = 119 \equiv 5$ and $d = (7 \cdot 5 - 10)/9 = 25 \cdot 9^{-1} = 6 \cdot 17 = 102 \equiv 7$. Private key recovered.

Real-world failures

(i) 2010: Sony PlayStation 3 firmware signing used a constant k and the root key was recovered. (ii) 2013: a flaw in Android's `SecureRandom` produced repeated nonces and Bitcoin wallets were emptied. (iii) 2019: "Minerva" timing attacks on smart cards leaked the bit length of k.

Biased nonces and lattices. Even if every k is distinct, a small bias is fatal. If the top ℓ bits of k_i are known for N signatures, then each signature gives $k_i = s_i^{-1} e_i + s_i^{-1} \rho_i d \pmod r$, i.e. a *hidden number problem* instance; the unknown d is found as a short vector in a lattice of dimension $N + 1$ (Boneh–Venkatesan; Howgrave-Graham–Smart; Nguyen–Shparlinski). In practice $\ell = 3, 4$ bits known for a few hundred signatures suffice for 256-bit curves, and recent attacks work with $\ell = 1$ or less.

Deterministic nonces. RFC 6979 derives $k = \mathrm{HMAC\text{-}DRBG}(d, e)$ deterministically from the private key and message hash, eliminating the dependence on a high-quality RNG at signing time. Fault-injection can still be an issue (hedged variants add fresh randomness).

Malleability. (ρ, s) and $(\rho, -s)$ are both valid. Bitcoin enforces "low s".

4.3.3 Schnorr signatures and EdDSA

Definition 4.18 (Schnorr identification protocol, Σ-protocol)**.** Prover has $(d, Q = dG)$. (1) Prover sends $R = kG$, $k \xleftarrow{\$} \mathbb{Z}_r$. (2) Verifier sends challenge $c \xleftarrow{\$} \mathbb{Z}_r$. (3) Prover answers $s = k + cd \bmod r$. Verifier checks $sG = R + cQ$.

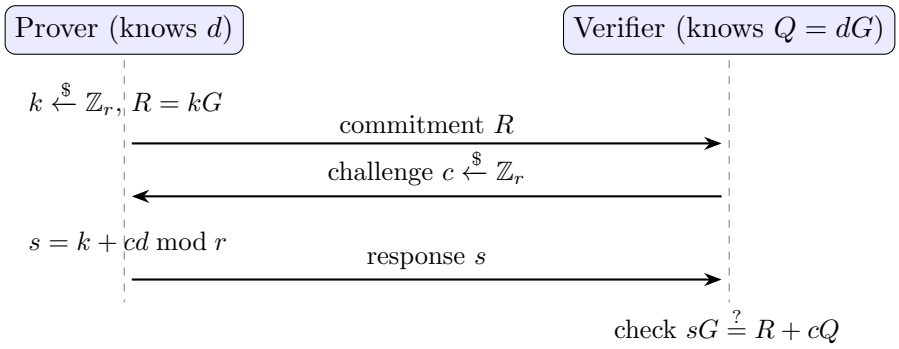


Figure 4.4: The Schnorr identification protocol. The Fiat–Shamir transform replaces the verifier's random challenge by $c = H(R \,\|\, Q \,\|\, m)$, which turns the protocol into a signature scheme.

Example 4.19 (Toy Schnorr signature on $E(\mathbb{F}_{17})$)**.** Secret key $d = 7$, public key $Q = 7G = (0, 6)$. Choose nonce $k = 5$, so $R = 5G = (9, 16)$. Suppose the hash gives $c = H(R \,\|\, Q \,\|\, m) = 4$ (toy value). Then $s = k + cd = 5 + 28 = 33 \equiv 14 \pmod{19}$ and the signature is (R, s). Verification: $sG = 14G = (9, 1)$ and $R + cQ = 5G + 4 \cdot 7G = 33G = 14G = (9, 1)$. Equal, so accept.

Proposition 4.20 (Special soundness and honest-verifier ZK)**.** *Two accepting transcripts (R, c, s) and (R, c', s') with the same R and $c \neq c'$ yield $d = (s - s')/(c - c')$. A simulator that picks c, s uniformly and sets $R = sG - cQ$ produces transcripts distributed identically to real ones.*

By the *Fiat–Shamir transform* the challenge becomes $c = H(R \,\|\, Q \,\|\, m)$, giving the Schnorr signature (R, s) (or (c, s)).

Theorem 4.21 (Pointcheval–Stern, forking lemma)**.** *In the random-oracle model, if a forger breaks Schnorr signatures with probability ε after q_H hash queries then the discrete logarithm can be computed with probability $\approx \varepsilon^2/q_H$ in about twice the time.*

Advantages over ECDSA: provable security, no modular inversion, linearity (batch verification, multi-signatures such as MuSig2, threshold schemes FROST, adaptor signatures). Bitcoin's BIP 340 adopted Schnorr over secp256k1 in 2021.

Definition 4.22 (EdDSA / Ed25519)**.** Let B be the Edwards25519 base point of order ℓ ($h = 8$), $H = \text{SHA-512}$. A secret seed k is hashed $H(k) = (h_0, \ldots, h_{511})$; the scalar a is the clamped lower half, and $A = aB$ is the public key. To sign M: $r = H(h_{256..511} \| M) \bmod \ell$ (deterministic nonce!), $R = rB$, $S = (r + H(R\|A\|M)\, a) \bmod \ell$; signature (R, S) of 64 bytes. Verification checks $8SB = 8R + 8H(R\|A\|M)A$ (cofactored) or $SB = R + H(R\|A\|M)A$ (cofactorless; the two differ for adversarial points, and this leads to subtle interoperability issues).

Key idea

Prefer Schnorr-type signatures with deterministic or hedged nonces and complete Edwards arithmetic for new designs; ECDSA remains mandatory in legacy ecosystems (TLS, WebAuthn, Bitcoin pre-Taproot).

4.4 Encoding of points and hashing to the curve

Point compression (SEC1). A point (x, y) is encoded as $\texttt{02/03} \,\|\, x$ depending on the parity of y; decompression needs a square root of $x^3 + ax + b$ (Theorem 1.4). Edwards25519 encodes y and the sign of x; X25519 uses only u.

Example 4.23 (Point compression on $E(\mathbb{F}_{17})$)**.** Take $P = (5, 1)$. Its compressed form is $\texttt{03} \,\|\, 5$ because $y = 1$ is odd (for $-P = (5, 16)$ it would be $\texttt{02} \,\|\, 5$). To decompress $\texttt{03} \,\|\, 5$: compute $x^3 + 2x + 2 = 125 + 10 + 2 = 137 \equiv 1 \pmod{17}$, so $y = \pm 1$ (here $17 \equiv 1 \pmod 4$); for $p \equiv 3 \pmod 4$ one uses $y = a^{(p+1)/4}$). The prefix 03 demands odd y, so $y = 1$ and we recover $(5, 1)$. This halves the size of public keys, at the cost of one square root.

Hash-to-curve. Many protocols (BLS, OPRFs, PAKE) need a hash $H : \{0,1\}^* \to E(\mathbb{F}_p)$ with unknown discrete logs. The naive *try-and-increment* method hashes $m\|i$ until $x^3 + ax + b$ is a square: it leaks timing information about m. Constant-time constructions (Shallue–van de Woestijne, simplified SWU, Elligator 2, standardised in RFC 9380) map field elements to curve points; to obtain a random-oracle-like map one evaluates twice and adds.

Elligator. Uniform encodings of about half of the points of Curve25519 as random-looking strings (useful against censorship: Tor obfs4, etc.).

4.5 Invalid-curve and small-subgroup attacks

Observe that the Weierstrass addition formulas (1.4) do *not* involve the coefficient b.

Theorem 4.24 (Invalid-curve attack; Biehl–Meyer–Müller 2000, Antipa et al. 2003)**.** *Suppose Bob holds a static key d, receives points P without verifying $P \in E$, and returns (or uses in an observable way) dP. Let $b' \neq b$ and let $P' = (x', y')$ be a point on $E' : y^2 = x^3 + ax + b'$ of small prime order ℓ. Then Bob computes on E' and returns $dP' = (d \bmod \ell)P'$ so the attacker learns $d \bmod \ell$. Repeating for several E' with distinct small primes $\ell_1, \ldots, \ell_t$ and $\prod \ell_i > r$, the CRT recovers d.*

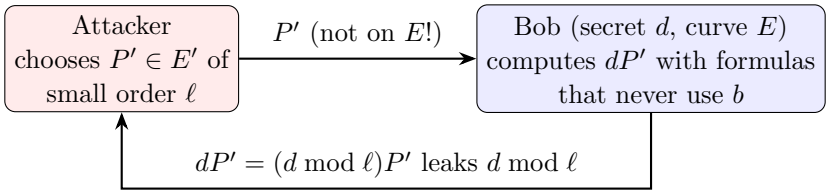


Figure 4.5: The invalid-curve attack. Bob's group-law formulas work on *any* curve $y^2 = x^3+ax+b'$, so a point from a weak curve E' makes him compute in a tiny group.

Example 4.25 (Invalid-curve attack in $E(\mathbb{F}_{17})$)**.** Bob's curve is $E : y^2 = x^3 + 2x + 2$ (order 19) with secret key $d = 13$; he returns dP for any received P without checking $P \in E$. The attacker uses $E' : y^2 = x^3 + 2x + 8$ over $\mathbb{F}_{17}$, which has $21 = 3 \cdot 7$ points (the formulas of Theorem 1.16 only involve a, so Bob computes happily on E').

- $P_3 = (11, 1)$ has order 3. Bob returns $13P_3 = P_3$ (since $13 \equiv 1 \bmod 3$): the attacker learns $d \equiv 1 \pmod 3$.
- $P_7 = (6, 7)$ has order 7. Bob returns $13P_7 = (6, 10) = -P_7 = 6P_7$ (since $13 \equiv 6 \bmod 7$): the attacker learns $d \equiv 6 \pmod 7$.

By the Chinese remainder theorem $d \equiv 13 \pmod{21}$, and since $0 < d < 19$ we get $d = 13$. With a real 256-bit curve the attacker uses several curves E' whose orders have small prime factors $\ell_1, \ell_2, \ldots$ with $\prod \ell_i > r$, and needs only $\sum \ell_i$ queries.

Countermeasures. (1) *Validate* every received public key: $P \neq \mathcal{O}$, coordinates in $[0, p-1]$, $P \in E$, and $rP = \mathcal{O}$ (or use cofactor multiplication). (2) Use ephemeral keys. (3) For x-only ladders (X25519) choose curves with *twist security*, since every u is the x-coordinate of a point on E or on its twist: if the twist has small factors, $d \bmod \ell$ leaks. (4) Use prime-order encodings (Ristretto, Decaf) to eliminate the cofactor.

Small-subgroup attacks. If $h > 1$, an attacker may send a point of small order $\ell \mid h$ (a "low-order point" such as $u = 0$ or $u = 1$ on Curve25519); Bob's result dP then depends only on $d \bmod \ell$. Clamping ($8 \mid d$) neutralises this, and so does explicitly checking for $\mathcal{O}$ in the output.

Example 4.26 (Validating a public key)**.** On $E' : y^2 = x^3 + 2x + 8$ over $\mathbb{F}_{17}$ (order $21 = 3 \cdot 7$) suppose the protocol uses the subgroup of prime order $r = 7$ generated by $(6, 7)$, so the cofactor is $h = 3$. A received point is accepted only if it is on the curve *and* $rP = \mathcal{O}$. The point $(6, 7)$ passes ($7P = \mathcal{O}$). The point $(11, 1)$ is on the curve, but $7 \cdot (11, 1) = (11, 1) \neq \mathcal{O}$ (it has order 3): rejected. The point $(0, 5)$ has order 21 and $7 \cdot (0, 5) = (11, 16) \neq \mathcal{O}$: rejected. Multiplying by the cofactor ($3P$) would instead *project* any point into the order-7 subgroup.

4.6 Side-channel and fault attacks

The ECDLP is a mathematical problem; implementations leak physical information (time, power, EM emissions, cache access patterns, faults).

Table 4.1: Side-channel threats to scalar multiplication and countermeasures.

Attack	Mechanism	Countermeasure
Timing	running time depends on key bits (branches, variable-time inversion, early-exit loops)	constant-time code; no secret-dependent branches or table indices
SPA (simple power analysis)	doubling vs. addition have different power traces in double-and-add	Montgomery ladder, regular windows, complete formulas ("atomic" operations)
DPA / CPA	statistical correlation of intermediate values with power	scalar blinding $k' = k + mr$, point blinding, randomised projective coordinates (Coron)
Cache attacks	table lookups indexed by secret	scatter-gather, constant-time table scan
Fault attacks	induced error leaks the key (e.g. invalid-curve via faults)	check output is on curve; redundant computation
Lattice + leakage (e.g. LadderLeak, Minerva)	partial nonce information	constant-time inversion/ladders, nonce blinding

Example 4.27 (SPA on double-and-add)**.** In Algorithm 2 the sequence of operations "D" (double) and "A" (add) directly reads the key: a "DA" block means bit 1, a lone "D" means bit 0. Observing a single power trace thus reveals k. The Montgomery ladder (Algorithm 4) executes the same operations for each bit, removing this leakage if the swap is implemented without branches.

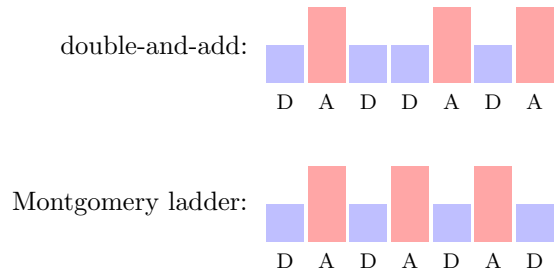


Figure 4.6: Idealised power traces (D = doubling, A = addition). In double-and-add the pattern "D A" vs. "D" spells out the key bits: here D A, D, D A, D A, i.e. bits $1, 0, 1, 1$. The ladder repeats "D, A" for every bit and reveals nothing by its shape.

Randomisation (Coron's countermeasures). (1) *Scalar blinding:* $k' = k + mr$ for a random m (kG unchanged since $rG = \mathcal{O}$). (2) *Point blinding:* compute $k(P + R) - kR$ with a secret R. (3) *Projective randomisation:* replace $(X : Y : Z)$ by $(\lambda^2 X : \lambda^3 Y : \lambda Z)$ for random λ.

4.7 Standards, deployment and the lesson of Dual_EC

Where ECC is used. TLS 1.3 (`x25519`, `secp256r1`, `secp384r1`; certificate signatures ECDSA/EdDSA), SSH (`curve25519-sha256`, `ssh-ed25519`), Signal/WhatsApp (X3DH and Double Ratchet with X25519; XEdDSA), Bitcoin and Ethereum (secp256k1, ECDSA; BIP340 Schnorr), WebAuthn/FIDO2 (P-256), e-passports and eID cards, Apple/Android secure enclaves, Zcash and many zero-knowledge ecosystems (BLS12-381, Jubjub, Pallas/Vesta).

Standards. ANSI X9.62/X9.63, SEC1/SEC2 (Certicom), NIST FIPS 186-5 (ECDSA, EdDSA) and SP 800-186 (curves), IEEE 1363, RFC 7748/8032 (Curve25519/Edwards), RFC 6979, RFC 9380, ISO/IEC 14888, Brainpool RFC 5639.

Example 4.28 (Secp256k1 parameters)**.** $p = 2^{256} - 2^{32} - 977$, $E : y^2 = x^3 + 7$, $h = 1$,

$$r = \texttt{FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFE BAAEDCE6 AF48A03B BFD25E8C D0364141},$$
$$G_x = \texttt{79BE667E F9DCBBAC 55A06295 CE870B07 029BFCDB 2DCE28D9 59F2815B 16F81798},$$
$$G_y = \texttt{483ADA77 26A3C465 5DA4FBFC 0E1108A8 FD17B448 A6855419 9C47D08F FB10D4B8}.$$

The code in Lecture B verifies $G \in E$ and $rG = \mathcal{O}$.

Dual_EC_DRBG: a cautionary tale

NIST SP 800-90A (2006) standardised a random bit generator based on two points $P, Q \in E(\mathbb{F}_p)$ (P-256): state $s_{i+1} = x(s_i P)$, output = truncate$(x(s_i Q))$ (with 16 bits dropped). Suppose $P = eQ$ for an e known to the designer. From a single output block the attacker guesses the dropped bits (about 2^{16} candidates), lifts the resulting x-coordinate to a point $A = \pm s_i Q$ and computes $eA = \pm s_i\, eQ = \pm s_i P$, whose x-coordinate is exactly the next state s_{i+1}. So the generator is backdoored by whoever generated (P, Q) (Shumow–Ferguson 2007). After the 2013 revelations the standard was withdrawn (2014). Lesson: all constants in a standard must be *verifiably* generated ("nothing up my sleeve").

Exercises for Lecture 4

Exercise 4.7 (ECDH by hand). On $E : y^2 = x^3 + 2x + 2$ over $\mathbb{F}_{17}$ with $G = (5, 1)$: Alice chooses $a = 4$, Bob $b = 9$. Compute A, B and the shared secret using Table 1.2. (Answer: $A = (3, 1)$, $B = (7, 6)$, shared point $36G = 17G = (6, 14)$.)

Exercise 4.8 (ECDSA verification). With the data of the toy example, verify that the pair $(\rho, s) = (9, 7)$ is accepted for $e = 10$ but rejected for $e = 11$.

Exercise 4.9 (Nonce reuse). Prove Theorem 4.16 and show what happens if the *same* message is signed twice with the same k. Further show that if the attacker knows $k_2 = k_1 + 1$ for two signatures, d can still be recovered. (Generalise: linearly related nonces.)

Exercise 4.10 (Hidden number problem). Suppose N signatures leak the ℓ least significant bits of each nonce. Show that for each i, the unknown d satisfies a linear congruence modulo r with a small unknown ($< r/2^{\ell}$) and explain how a lattice of dimension $N + 1$ can be built (Boneh–Venkatesan). Heuristically $N \gtrsim \log_2 r/\ell$ signatures are needed; estimate this for $r \approx 2^{256}$ and $\ell = 8$ (about 32, plus a margin).

Exercise 4.11 (Schnorr). Prove the special-soundness property and the simulator statement of the Schnorr Σ-protocol. Why is it important that R is a *fresh* random point in every run?

Exercise 4.12 (Invalid curve). (a) For $E_{b'} : y^2 = x^3 + 2x + 8$ over $\mathbb{F}_{17}$ (order 21) find a point P_3 of order 3 and P_7 of order 7 (e.g. $(11, 1)$ has order 3 and $(6, 7)$ has order 7). (b) Suppose Bob's key is $d = 16$ on $y^2 = x^3 + 2x + 2$, which returns the x-coordinate of dP. Show how querying P_3, P_7 reveals d. (c) What check in Bob's code would stop the attack?

Exercise 4.13 (Clamping). Show that for X25519 the clamped scalar k is a multiple of 8 and has bit 254 set, and explain: (i) why multiplying by 8 forces the result into the prime-order subgroup, (ii) why the leading 1 bit allows a fixed-length ladder. What attack remains if the peer's u-coordinate is of low order?

Exercise 4.14 (Dual_EC). Complete the derivation in the Dual_EC box: suppose an observer sees $o = \mathrm{trunc}(x(s_i Q))$. Explain how, knowing e with $P = eQ$, the next state s_{i+1} is computed, including the complexity of guessing the truncated bits.

Exercise 4.15 (Side channels). Given the (hypothetical) trace of operations D D A D A D D D A for Algorithm 2 (left-to-right, initial $R = P$), recover the scalar k. Then show how the Montgomery ladder prevents this.

Exercise 4.16 (Programming project). Using Lecture B, implement ECIES over secp256k1 with AES-GCM from a standard library, plus point validation. Demonstrate an invalid-curve attack against a version without validation using a small test curve of order $2^a \cdot 3^b \cdot 5^c \cdots$.

Chapter 5

Divisors, Pairings, Isogenies and the Post-Quantum Horizon

Learning goals

After this lecture students should be able to

- compute with divisors and functions on an elliptic curve and state the characterisation of principal divisors;
- define the Weil and Tate pairings, state their properties, and describe Miller's algorithm;
- prove the MOV reduction and explain the notion of pairing-friendly curves (embedding degree, ρ-value, BN and BLS families);
- describe pairing-based protocols (Joux, BLS signatures, Boneh–Franklin IBE);
- define isogenies, state Vélu's formulas, and sketch isogeny-based cryptography (SIDH's collapse, CSIDH, SQIsign);
- discuss Shor's algorithm against ECC and the transition to post-quantum cryptography.

5.1 Functions and divisors on an elliptic curve

Let E/K be an elliptic curve, $\overline{K}(E)$ its function field (rational functions $f(x,y)/g(x,y)$ modulo the curve equation). Every $f \in \overline{K}(E)^\times$ has a *zero or pole order* $\mathrm{ord}_P(f) \in \mathbb{Z}$ at each point P (using a local parameter; on a smooth curve this is well defined).

Definition 5.1 (Divisors)**.** A *divisor* is a finite formal sum $D = \sum_P n_P(P)$ with $n_P \in \mathbb{Z}$. Its *degree* is $\deg D = \sum n_P$ and its *sum* is $\mathrm{sum}(D) = \sum n_P P \in E$ (using the group law). The *divisor of a function* is $\mathrm{div}(f) = \sum_P \mathrm{ord}_P(f)(P)$; such divisors are called *principal*. Divisors form a group $\mathrm{Div}(E)$ and $D \sim D'$ if $D - D'$ is principal. $\mathrm{Pic}^0(E) = \mathrm{Div}^0(E)/\{\text{principal}\}$ is the group of classes of degree-0 divisors.

Example 5.2. On $E : y^2 = x^3 + ax + b$ with $\mathcal{O}$ at infinity: x has a double pole and y a triple pole at $\mathcal{O}$. For $P = (x_0, y_0)$: $\mathrm{div}(x - x_0) = (P) + (-P) - 2(\mathcal{O})$. If $L : y = \lambda x + \nu$ is a non-vertical line meeting E in P_1, P_2, P_3 then $\mathrm{div}(y - \lambda x - \nu) = (P_1) + (P_2) + (P_3) - 3(\mathcal{O})$. In particular for the three 2-torsion points, $\mathrm{div}(y) = (T_1) + (T_2) + (T_3) - 3(\mathcal{O})$.

Theorem 5.3 (Principal divisors on E)**.** *(i) For $f \in \overline{K}(E)^\times$, $\deg \mathrm{div}(f) = 0$ (f has as many zeros as poles), and $\mathrm{div}(f) = 0$ iff f is constant.*
(ii) A divisor D is principal iff $\deg D = 0$ and $\mathrm{sum}(D) = \mathcal{O}$.

(iii) Consequently the map $\kappa : \mathrm{Pic}^0(E) \to E$, $[D] \mapsto \mathrm{sum}(D)$, *is a group isomorphism, with inverse* $P \mapsto [(P) - (\mathcal{O})]$.

Key idea

The group law on E *is* the group law on divisor classes of degree zero: $P + Q = R$ iff $(P) + (Q) \sim (R) + (\mathcal{O})$. This viewpoint (i) proves associativity (Lecture A), (ii) generalises to Jacobians of higher genus curves, (iii) leads to pairings.

If $f \in \overline{K}(E)^\times$ and $D = \sum n_P(P)$ has support disjoint from $\mathrm{div}(f)$, define $f(D) = \prod f(P)^{n_P} \in \overline{K}^\times$.

Theorem 5.4 (Weil reciprocity)**.** *If* $f, g \in \overline{K}(E)^\times$ *have disjoint divisors then* $f(\mathrm{div}\, g) = g(\mathrm{div}\, f)$.

5.2 The Weil pairing

Fix an integer n coprime to $\mathrm{char}\, K$; let $\mu_n \subset \overline{K}^\times$ be the nth roots of unity.

Definition 5.5 (Weil pairing)**.** Let $P, Q \in E[n]$. Choose divisors $D_P \sim (P) - (\mathcal{O})$ and $D_Q \sim (Q) - (\mathcal{O})$ with disjoint supports. By Theorem 5.3(ii) nD_P and nD_Q are principal: $nD_P = \mathrm{div}(f_P)$, $nD_Q = \mathrm{div}(f_Q)$. Set

$$e_n(P, Q) = \frac{f_P(D_Q)}{f_Q(D_P)} \in \overline{K}^\times.$$

Theorem 5.6 (Properties of e_n)**.** *The Weil pairing* $e_n : E[n] \times E[n] \to \mu_n$ *is well defined (independent of the choices) and*

(i) bilinear: $e_n(P_1 + P_2, Q) = e_n(P_1, Q)e_n(P_2, Q)$, *and the same in the second variable;*
(ii) alternating: $e_n(P, P) = 1$, *hence skew-symmetric* $e_n(P, Q) = e_n(Q, P)^{-1}$;
(iii) non-degenerate: *if* $e_n(P, Q) = 1$ *for all* $Q \in E[n]$ *then* $P = \mathcal{O}$;
(iv) Galois-equivariant: $e_n(P, Q)^\sigma = e_n(P^\sigma, Q^\sigma)$ *for* $\sigma \in \mathrm{Gal}(\overline{K}/K)$;
(v) compatible: $e_{nm}(P, Q) = e_n(mP, Q)$ *for* $P \in E[nm]$, $Q \in E[n]$.

Corollary 5.7. *If* $E[n] \subset E(K)$ *then* $\mu_n \subset K$ *(surjectivity of* e_n *and equivariance). In particular, the structure theorem* $n_1 \mid q - 1$ *(Theorem 2.11) follows. If* $P \in E[n]$ *has exact order* n *then there is* $Q \in E[n]$ *with* $e_n(P, Q)$ *a primitive* n*th root of unity.*

5.3 The Tate pairing and Miller's algorithm

In cryptography one prefers the *Tate–Lichtenbaum pairing*, which is faster to compute and needs only one point of order r in a smaller field.

Let r be a prime, $r \mid \#E(\mathbb{F}_q)$, $r \nmid q$, and let k be the *embedding degree*, the smallest integer with $r \mid q^k - 1$.

Definition 5.8 (Reduced Tate pairing)**.** For $P \in E(\mathbb{F}_q)[r]$ and $Q \in E(\mathbb{F}_{q^k})/rE(\mathbb{F}_{q^k})$ choose a divisor $D_Q \sim (Q) - (\mathcal{O})$ with support disjoint from P and let $f_{r,P}$ be a function with $\mathrm{div}(f_{r,P}) = r(P) - r(\mathcal{O})$. Then

$$\tau_r(P, Q) = f_{r,P}(D_Q)^{(q^k-1)/r} \in \mu_r \subset \mathbb{F}_{q^k}^\times.$$

Theorem 5.9. *τ_r is well defined, bilinear and non-degenerate (on $E(\mathbb{F}_q)[r] \times E(\mathbb{F}_{q^k})/rE(\mathbb{F}_{q^k})$). On $E[r] \subset E(\mathbb{F}_{q^k})$ the Weil pairing is, up to a fixed power, the quotient $\tau_r(P,Q)/\tau_r(Q,P)$.*

Theorem 5.10 (Balasubramanian–Koblitz)**.** *Let $r \mid \#E(\mathbb{F}_q)$ be a prime, $r \nmid q-1$ (and $r \neq p$). Then $E[r] \subset E(\mathbb{F}_{q^k})$ iff $r \mid q^k - 1$, i.e. iff k is the embedding degree.*

5.3.1 Miller's algorithm

The functions $f_{n,P}$ (with $\operatorname{div}(f_{n,P}) = n(P) - ([n]P) - (n-1)(\mathcal{O})$) satisfy the recursion

$$f_{i+j,P} = f_{i,P}\, f_{j,P}\, \frac{\ell_{iP,jP}}{v_{(i+j)P}},$$

where $\ell_{A,B}$ is the line through A, B and v_C the vertical line through C (cf. Theorem 5.2: $\operatorname{div}(\ell_{A,B}/v_{A+B}) = (A) + (B) - (A+B) - (\mathcal{O})$).

Algorithm 9 Miller's algorithm: evaluate $f_{n,P}(Q)$

Input: $n = (n_{t-1} \dots n_0)_2$, points P, Q (Q not a zero or pole of the intermediate functions)

1: $T \leftarrow P$; $f \leftarrow 1$
2: **for** $i = t-2$ **down to** 0 **do**
3: $\quad f \leftarrow f^2 \cdot \ell_{T,T}(Q)/v_{2T}(Q)$; $T \leftarrow 2T$
4: $\quad$ **if** $n_i = 1$ **then** $f \leftarrow f \cdot \ell_{T,P}(Q)/v_{T+P}(Q)$; $T \leftarrow T + P$
5: $\quad$ **end if**
6: **end for**
7: **return** f ▷ then apply the final exponentiation $f^{(q^k-1)/r}$ for the Tate pairing

Miller's loop has the same shape as double-and-add: $\mathcal{O}(\log r)$ iterations, each costing a few multiplications in $\mathbb{F}_{q^k}$; pairings are therefore considerably more expensive than scalar multiplications (milliseconds vs. tens of microseconds). Optimised variants (*ate*, *optimal ate*, *twisted* Miller loops, denominator elimination, final-exponentiation addition chains) reduce the loop length to $\approx \log r/\varphi(k)$.

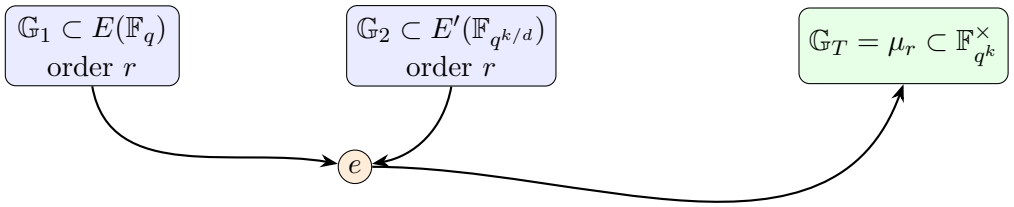
Example 5.11 (A pairing small enough to compute by machine)**.** Let $p = 131$ (so $p \equiv 3 \bmod 4$) and $E : y^2 = x^3 + x$ over $\mathbb{F}_{131}$. It is supersingular with $\#E(\mathbb{F}_{131}) = p + 1 = 132 = 12 \cdot 11$. Take $r = 11$ and $P = (121, 118)$ of order 11; then $11 \mid p+1$, so the embedding degree is $k = 2$ and the pairing takes values in $\mathbb{F}_{131^2} = \mathbb{F}_{131}[i]/(i^2+1)$. Since $E(\mathbb{F}_p)[r]$ is cyclic, $e_r(P,P) = 1$, so we use the *distortion map* $\varphi(x,y) = (-x, iy)$ and define

$$e(P,Q) = \tau_{11}\big(P, \varphi(Q)\big) = f_{11,P}\big(\varphi(Q)\big)^{(p^2-1)/11}.$$

Evaluating Miller's algorithm with $r = 11$ gives

$$\zeta := e(P,P) = 39 + 24i, \qquad \zeta^{11} = 1, \qquad \zeta^2 = 28 + 38i,\ \zeta^4 = 126 + 32i,\ \zeta^{10} = 39 + 107i.$$

So $e(P,P)$ is an element of order 11 in $\mathbb{F}_{131^2}^{\times}$. Bilinearity checks: $e(3P, 5P) = 126 + 32i = \zeta^4 = \zeta^{15}$, since $15 \equiv 4 \pmod{11}$, and $e(2P, P) = 28 + 38i = \zeta^2$. (The Python code of Lecture B computes these values.)



$$e(aP, bQ) = e(P, Q)^{ab} \quad \text{(bilinearity)}$$

$\Rightarrow$ DDH easy in $\mathbb{G}_1$ (when $\mathbb{G}_1 \neq \mathbb{G}_2$ is an *asymmetric* (Type-3) pairing, DDH remains hard in $\mathbb{G}_1$ and $\mathbb{G}_2$)

Figure 5.1: A (Type-3) pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ of three groups of prime order r. E' is a degree-d twist of E allowing the second group to live over a smaller field $\mathbb{F}_{q^{k/d}}$.

5.4 The MOV attack as a theorem

Proof of Theorem 3.16 (MOV reduction). Let $P \in E(\mathbb{F}_q)$ of prime order r, $Q = xP$, and assume $E[r] \subset E(\mathbb{F}_{q^k})$ (true after possibly extending the field; by Theorem 5.10 it holds for the embedding degree k). Since $P \neq \mathcal{O}$, Theorem 5.7 gives a $T \in E[r]$ with $\zeta = e_r(P, T) \neq 1$, i.e. a primitive rth root of unity (a random T works with probability $1 - 1/r$). By bilinearity

$$e_r(Q, T) = e_r(xP, T) = e_r(P, T)^x = \zeta^x.$$

So computing two pairings reduces the ECDLP to finding the exponent x with $e_r(Q, T) = \zeta^x$, a DLP in the subgroup μ_r of $\mathbb{F}_{q^k}^\times$. Each pairing evaluation costs $\mathcal{O}(\log r)$ operations in $\mathbb{F}_{q^k}$ with Miller's algorithm. □

Key idea

Pairings destroy security of *some* curves (small k) and enable protocols on others. Pairing-friendly curves are therefore chosen with k small enough to compute the pairing but large enough that DLP in $\mathbb{F}_{q^k}^\times$ is as hard as in E: e.g. $k = 12$ with $q \approx 2^{381}$ ($q^k \approx 2^{4572}$) corresponds to a ≈ 128-bit security level.

5.5 Pairing-based cryptography

Fix a Type-3 pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ of prime order r with generators $P_1 \in \mathbb{G}_1$, $P_2 \in \mathbb{G}_2$ and $e(P_1, P_2) \neq 1$.

5.5.1 Joux's tripartite Diffie–Hellman (2000)

Alice, Bob and Carol choose secrets a, b, c and broadcast $(aP_1, aP_2), (bP_1, bP_2), (cP_1, cP_2)$ in a *single round*. Alice computes $K = e(bP_1, cP_2)^a = e(P_1, P_2)^{abc}$; Bob computes $e(aP_1, cP_2)^b$, Carol $e(aP_1, bP_2)^c$: all equal $e(P_1, P_2)^{abc}$ by bilinearity. (Without pairings no one-round three-party key exchange from DH is known.)

5.5.2 BLS short signatures (Boneh–Lynn–Shacham, 2001)

Let $H : \{0,1\}^* \to \mathbb{G}_1$ be a hash function (hash-to-curve, Lecture 4).

- **KeyGen:** $x \xleftarrow{\$} \mathbb{Z}_r^\times$; public key $X = xP_2 \in \mathbb{G}_2$.
- **Sign**(x, m): $\sigma = xH(m) \in \mathbb{G}_1$.
- **Verify**(X, m, σ): accept iff $e(\sigma, P_2) = e(H(m), X)$.

Correctness: $e(xH(m), P_2) = e(H(m), P_2)^x = e(H(m), xP_2)$. Security: existentially unforgeable under chosen-message attacks in the random-oracle model, under the *co-CDH* assumption in $(\mathbb{G}_1, \mathbb{G}_2)$. *Aggregation:* signatures σ_i on different messages m_i under keys X_i combine as $\sigma = \sum \sigma_i$, verified by $e(\sigma, P_2) = \prod_i e(H(m_i), X_i)$; for a single message the check is $e(\sigma, P_2) = e(H(m), \sum X_i)$ (with proofs of possession against rogue-key attacks). On BLS12-381 a signature is 48 bytes compressed; the scheme is used for Ethereum's proof-of-stake consensus, Filecoin, Dfinity, etc.

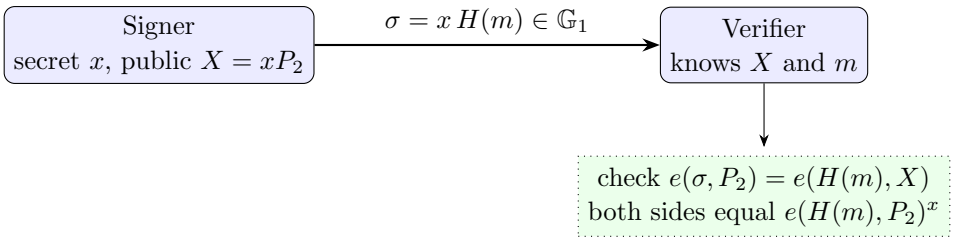


Figure 5.2: BLS signatures. The verification equation is bilinearity in action: the secret exponent x can be moved from one argument of the pairing to the other.

Example 5.12 (Toy BLS on the pairing of Theorem 5.11)**.** Let $x = 7$ be the secret key and $X = 7P = (60, 98)$ the public key. Suppose $H(m) = 3P = (49, 45)$ (a stand-in for a hash into the group). The signature is $\sigma = 7 \cdot 3P = 21P = 10P = (121, 13)$. Verification: $e(\sigma, P) = e(10P, P) = \zeta^{10} = 39 + 107i$ and $e(H(m), X) = e(3P, 7P) = \zeta^{21} = \zeta^{10} = 39 + 107i$. Equal, so accept. With $r = 11$ this is of course not secure; real systems use $r \approx 2^{255}$.

5.5.3 Identity-based encryption (Boneh–Franklin, 2001)

Setup: master secret $s \in \mathbb{Z}_r$, public $P_{\text{pub}} = sP$ (symmetric pairing view). *Extract:* for identity ID, $Q_{\text{ID}} = H_1(\text{ID}) \in \mathbb{G}_1$, private key $d_{\text{ID}} = sQ_{\text{ID}}$. *Encrypt* m: $g = e(Q_{\text{ID}}, P_{\text{pub}})$, choose ρ, output $(U, V) = (\rho P,\ m \oplus H_2(g^\rho))$. *Decrypt:* $m = V \oplus H_2(e(d_{\text{ID}}, U))$. Correctness: $e(d_{\text{ID}}, U) = e(sQ_{\text{ID}}, \rho P) = e(Q_{\text{ID}}, P)^{s\rho} = e(Q_{\text{ID}}, P_{\text{pub}})^\rho = g^\rho$. This realises Shamir's 1984 dream of using any string (an email address) as a public key. (The basic scheme is CPA-secure; the Fujisaki–Okamoto transform gives CCA security.)

5.5.4 Other applications

Attribute-based and functional encryption, short group signatures, broadcast encryption, KZG polynomial commitments (constant-size proofs of polynomial evaluations), and zk-SNARKs (Groth16: three group elements on BN254 or BLS12-381) all depend on pairings.

5.5.5 Pairing-friendly curves

We need r prime large (≈ 256 bits), k small, q^k large. The *ρ-value* is $\rho = \log q / \log r \geq 1$ (closer to 1 means more efficient). By Theorem 5.10 one needs $r \mid \#E(\mathbb{F}_q) = q + 1 - t$ and $r \mid q^k - 1$;

equivalently $r \mid \Phi_k(t-1)$ with Φ_k the kth cyclotomic polynomial. Random curves do not satisfy this: one constructs curves via the CM method (Lecture 2) with D small.

Table 5.1: Selected pairing-friendly families.

Family	k	$q(u)$, $r(u)$	Remarks
Supersingular (Menezes et al.)	2–6	—	fast but need large q for security; mostly obsolete
Barreto–Naehrig (BN)	12	$q = 36u^4 + 36u^3 + 24u^2 + 6u + 1$ $r = 36u^4 + 36u^3 + 18u^2 + 6u + 1$	$\rho \approx 1$, $t = 6u^2 + 1$, BN254 (formerly 128-bit) now $\approx$ 100 bits
BLS12 (Barreto–Lynn–Scott)	12	$r = u^4 - u^2 + 1$, $q = \frac{(u-1)^2}{3} r + u$	BLS12-381: $\lvert q\rvert = 381$, $\lvert r\rvert = 255$, $\rho \approx 1.5$
BLS24, KSS, …	24, 18	—	higher security levels

Remark 5.13 (BLS12-381 numerically). With $u = -\texttt{0xd201000000010000}$ one gets $r = u^4 - u^2 + 1$ (255 bits, prime), $q = (u-1)^2 r/3 + u$ (381 bits, prime), $t = u + 1$ and embedding degree 12 (all verified by the code of Lecture B).

Remark 5.14 (Security re-evaluation). The *extended tower number field sieve* (Kim–Barbulescu, 2016) lowered the security of DLP in $\mathbb{F}_{q^k}$ for composite k such as 12, reducing the effective security of BN254 to about 100 bits; this motivated the move to BLS12-381 and larger parameters. The security level of $\mathbb{G}_T$ must be re-examined whenever the best NFS variants improve.

5.6 Isogenies

Definition 5.15 (Isogeny)**.** An *isogeny* $\phi : E_1 \to E_2$ is a non-constant morphism of curves with $\phi(\mathcal{O}) = \mathcal{O}$. It is automatically a group homomorphism. Its *degree* is the degree of the field extension $\overline{K}(E_1)/\phi^*\overline{K}(E_2)$; if ϕ is separable, $\deg\phi = \#\ker\phi$. Every isogeny has a *dual* $\hat{\phi} : E_2 \to E_1$ with $\hat{\phi}\phi = [\deg\phi]$, so being isogenous is an equivalence relation. Examples: $[m]$ ($\deg m^2$), the Frobenius π_q ($\deg q$).

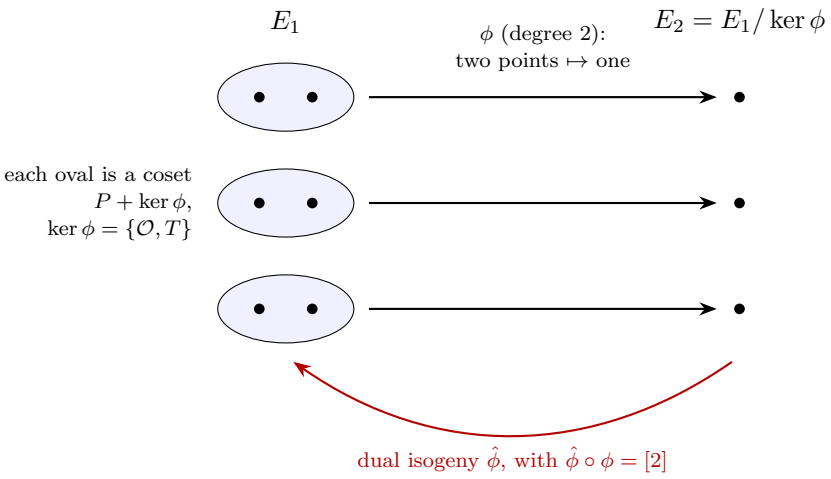


Figure 5.3: An isogeny of degree 2. It is a group homomorphism; every point of E_2 has exactly $\deg\phi$ preimages, forming a coset of the kernel. The dual isogeny goes back, and going there and back is multiplication by the degree.

Theorem 5.16 (Vélu, 1971)**.** *For every finite subgroup $F \subset E(\overline{K})$ there is a curve $E' = E/F$ and a separable isogeny $\phi : E \to E'$ with $\ker\phi = F$, unique up to isomorphism. If $E : y^2 = x^3 + ax + b$ and F has odd order, let $S \subset F \setminus \{\mathcal{O}\}$ contain one point from each pair $\{\pm Q\}$, and for $Q = (x_Q, y_Q) \in S$ put*

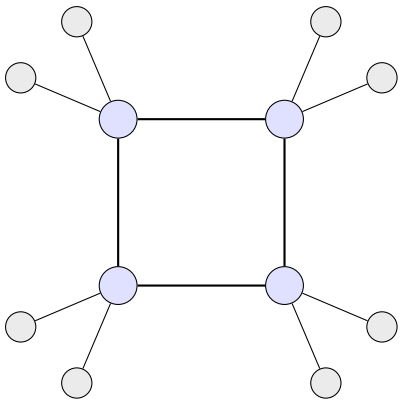
$$t_Q = 6x_Q^2 + 2a, \qquad u_Q = 4(x_Q^3 + ax_Q + b), \qquad t = \sum_{Q\in S} t_Q, \qquad w = \sum_{Q\in S}(u_Q + x_Q t_Q).$$

Then $E' : y^2 = x^3 + (a - 5t)x + (b - 7w)$ and the x-coordinate of $\phi(x, y)$ is $x + \sum_{Q\in S}\left(\frac{t_Q}{x-x_Q} + \frac{u_Q}{(x-x_Q)^2}\right)$ (the y-coordinate is obtained from it by differentiation).

Example 5.17 (A 3-isogeny over $\mathbb{F}_{17}$)**.** The curve $E : y^2 = x^3 + 2x + 8$ over $\mathbb{F}_{17}$ has 21 points, and $P = (11, 1)$ has order 3. Take $F = \{\mathcal{O}, P, -P\}$, $S = \{P\}$. Then $t_P = 6\cdot 11^2 + 2\cdot 2 = 730 \equiv 16$, $u_P = 4(1331+22+8) = 4\cdot 1361 \equiv 4$ (indeed $y_P^2 = 1$), $t = 16$, $w = u_P + x_P t_P = 4 + 11\cdot 16 = 180 \equiv 10 \pmod{17}$. Hence $a' = 2 - 5\cdot 16 \equiv 7$, $b' = 8 - 7\cdot 10 \equiv 6$, and $E' : y^2 = x^3 + 7x + 6$, which again has 21 points (as it must: isogenous curves over $\mathbb{F}_q$ have the same number of points, Theorem 5.18).

Theorem 5.18 (Tate, 1966)**.** *Two elliptic curves E_1, E_2 over $\mathbb{F}_q$ are isogenous over $\mathbb{F}_q$ iff $\#E_1(\mathbb{F}_q) = \#E_2(\mathbb{F}_q)$.*

Theorem 5.19 (Endomorphism rings)**.** *For $E/\mathbb{F}_q$,* $\mathrm{End}(E)$ *(over $\overline{\mathbb{F}}_q$) is either an order in an imaginary quadratic field (*ordinary*) or a maximal order in the quaternion algebra $B_{p,\infty}$ ramified at p and ∞ (*supersingular*). If E is ordinary with $\mathrm{End}(E) = \mathcal{O}$, the ideal class group $\mathrm{Cl}(\mathcal{O})$ acts freely and transitively on the isomorphism classes of curves with endomorphism ring $\mathcal{O}$ and given trace; the action of an ideal $\mathfrak{a}$ is by an isogeny of degree $N(\mathfrak{a})$.*



Crater: 4 curves with the largest endomorphism ring, joined by ℓ-isogenies (cycle);
Leaves: curves with smaller endomorphism ring;
each non-leaf has degree $\ell + 1 = 4$ here, leaves have degree 1.

Figure 5.4: Schematic 3-isogeny volcano of height 1 for ordinary curves: $\ell = 3$ and $\left(\frac{D}{\ell}\right) = 1$ (crater vertices have 2 neighbours in the crater and $\ell - 1 = 2$ below).

5.6.1 Isogeny-based cryptography

- **Hash functions (Charles–Goren–Lauter, 2009).** A walk in the supersingular ℓ-isogeny graph (an $(\ell+1)$-regular Ramanujan graph on $\approx p/12$ vertices over $\mathbb{F}_{p^2}$) with edges labelled by the message bits is hard to invert: finding paths between two given vertices is believed hard.
- **SIDH/SIKE (Jao–De Feo, 2011).** Alice and Bob choose secret subgroups of $E[2^a]$ resp. $E[3^b]$, compute isogenies and exchange the images of the other party's torsion basis. Key

exchange with very small public keys (≈ 330 bytes). **Broken in July 2022** by Castryck–Decru (and independently Maino–Martindale, Robert), who exploited the auxiliary torsion information using Kani's reducibility criterion for $(2,2)$-isogenies between products of elliptic curves; key recovery for SIKEp434 takes about an hour on one core. SIKE was subsequently withdrawn from the NIST process.

- **CSIDH (Castryck et al., 2018) and CSI-FiSh.** Commutative group action of the class group $\mathrm{Cl}(\mathbb{Z}[\sqrt{-p}])$ on supersingular curves over $\mathbb{F}_p$: $(\mathfrak{a}, E) \mapsto \mathfrak{a} \star E$. Gives a "post-quantum Diffie–Hellman" with non-interactive key exchange; attacked only by Kuperberg's subexponential quantum algorithm for the hidden shift problem, whose concrete cost for the parameters is debated.
- **SQIsign (De Feo, Kohel, Leroux, Petit, Wesolowski, 2020).** Signatures based on the Deuring correspondence between supersingular curves and maximal orders in quaternion algebras; the shortest post-quantum signatures plus public keys known (about 200 bytes), at the cost of slow signing. Under active development and evaluation in NIST's additional-signature process.

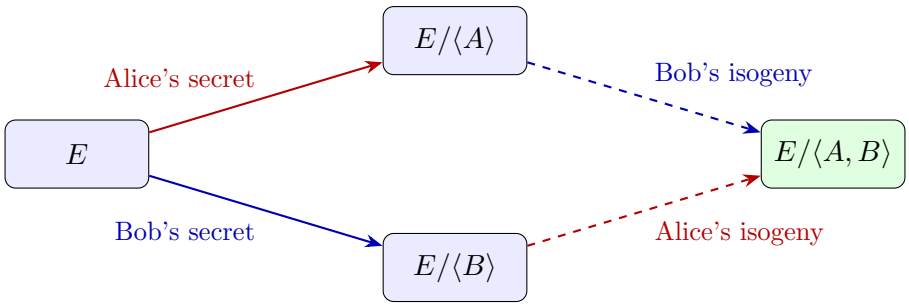


Figure 5.5: The commutative square behind isogeny key exchange (SIDH and CSIDH). Alice and Bob each apply a secret isogeny, exchange their curves (plus auxiliary information) and apply their secret again to the other's curve; both reach the same curve, the "shared secret" (its j-invariant). It is the analogue of $(g^a)^b = (g^b)^a$.

5.7 Quantum computers and post-quantum perspectives

Theorem 5.20 (Shor, 1994)**.** *On a quantum computer the DLP in any group G in which the group operation is efficiently computable (in particular in $E(\mathbb{F}_q)$) can be solved in time polynomial in $\log |G|$.*

Idea. Given $Q = xP$, consider $f : \mathbb{Z}_r^2 \to G$, $f(a,b) = aP + bQ$. This function is constant on cosets of the lattice $L = \{(a,b) : a + bx \equiv 0 \bmod r\}$ (hidden subgroup). Prepare $\sum_{a,b} |a,b\rangle|f(a,b)\rangle$, measure the second register, and apply the quantum Fourier transform over $\mathbb{Z}_r^2$ to the first. The result is a uniformly random character (u,v) that is trivial on L; since L is generated by $(-x,1)$, this means $-xu + v \equiv 0 \pmod r$. As soon as u is invertible, $x \equiv vu^{-1} \pmod r$. □

The quantum cost of Shor's algorithm for ECDLP on an n-bit prime field is roughly $9n + 2\lceil \log_2 n \rceil + 10$ *logical* qubits (Roetteler–Naehrig–Svore–Lauter, 2017): 2330 for $n = 256$, and $\mathcal{O}(n^3)$ gates. Per bit of security, breaking ECC needs *fewer* quantum resources than breaking RSA of comparable classical strength. The transition plan is therefore:

1. migrate key exchange first ("harvest now, decrypt later" threat): hybrid X25519+ML-KEM-768 is already deployed in TLS in major browsers and CDNs;
2. migrate signatures: NIST standardised ML-KEM (FIPS 203), ML-DSA (FIPS 204) and SLH-DSA (FIPS 205) in August 2024; draft NIST guidance foresees deprecating 112-bit RSA/ECC by 2030 and disallowing quantum-vulnerable public-key algorithms by 2035 (check the current status of these documents);
3. meanwhile, elliptic curves remain secure against all known *classical* attacks and are the efficient workhorse of the Internet. The lattice-based, code-based, hash-based and isogeny-based families (the latter fragile, see above) are the intended successors.

5.8 Research directions and open problems

1. **ECDLP over $\mathbb{F}_p$:** subexponential algorithm or generic hardness? Better use of summation polynomials, or p-adic lifting beyond the anomalous case.
2. **Higher genus:** Jacobians of genus-2 curves (Kummer surfaces, Gaudry–Schost) give fast arithmetic on a 2-dimensional group and are competitive with elliptic curves, but have attracted far less cryptanalysis.
3. **Pairing-friendly curves:** constructions with prescribed (k, ρ); security of $\mathbb{G}_T$; cycles and chains of pairing-friendly curves for recursive SNARKs (Pasta, BLS12-377/BW6-761, MNT).
4. **Isogeny-based cryptography after SIDH:** efficient and secure instantiations of the group action (CSIDH variants), SQIsign and its higher-dimensional descendants, verifiable delay functions from isogenies.
5. **Implementation security:** formally verified, constant-time, side-channel resistant implementations of curves (HACL*, fiat-crypto) and protocols; fault-resilient designs.
6. **Standardisation:** transparent generation of curves; prime-order abstractions (Ristretto); handling of the cofactor in higher-level protocols; threshold and multi-party ECDSA/Schnorr.

Exercises for Lecture 5

Exercise 5.1 (Divisors). Compute $\operatorname{div}(y)$, $\operatorname{div}(x)$ and $\operatorname{div}\left(\frac{y-\lambda x-\nu}{x-x_3}\right)$ for the curve $y^2 = x^3 - x + 1$ and the line through $(0, 1)$ and $(3, 5)$ of Figure 1.4. Verify Theorem 5.3(ii): sum $= \mathcal{O}$.

Exercise 5.2 (Divisor class proof of the group law). Use Theorem 5.3 to prove that $P + Q = R$ (chord–tangent) iff $(P) + (Q) \sim (R) + (\mathcal{O})$. Conclude associativity of $+$.

Exercise 5.3 (Properties of the Weil pairing). Show from the definition and bilinearity that $e_n(P, Q)^n = 1$, and that if $P, Q \in E[n]$ generate $E[n]$ then $e_n(P, Q)$ is a primitive nth root of unity.

Exercise 5.4 (Embedding degrees). Compute the embedding degree of $r = 19$ in $E(\mathbb{F}_{17})$ (order 19) and of $r = 3$, $r = 7$ in $E : y^2 = x^3 + 2x + 8$ over $\mathbb{F}_{17}$ (order 21). Compare with $r \mid 17^k - 1$. (Answers: $k = 9$, 2 and 6.) Why does this toy curve say nothing about real security?

Exercise 5.5 (BN curves). Check that for the BN polynomials, $q + 1 - t = r$ with $t = 6u^2 + 1$. Show that $r \mid q^{12} - 1$. (Hint: $r \mid \Phi_{12}(t - 1)$ and $\Phi_{12}(x) = x^4 - x^2 + 1$; verify $\Phi_{12}(6u^2) \equiv 0$ modulo $r(u)$ over $\mathbb{Z}[u]$.)

Exercise 5.6 (BLS correctness and aggregation). Prove correctness of BLS verification, of the aggregate verification equation, and explain the *rogue-key attack*: an adversary registers $X_* =$

$x_*P_2 - \sum_{i\neq *} X_i$ and forges a signature on any message by the "group". How do proofs of possession prevent it?

Exercise 5.7 (Joux). Why is $e(P,P) \neq 1$ essential for the symmetric version of Joux's protocol, and what is the role of a distortion map on a supersingular curve? Show that without it, $e(P,P) = 1$ always holds for $P \in E[n]$ (use the alternating property).

Exercise 5.8 (Vélu). For $E : y^2 = x^3 + 2x + 8$ over $\mathbb{F}_{17}$, use $P_7 = (6,7)$ of order 7 to compute the 7-isogenous curve with Theorem 5.16 and verify that it has 21 points (the answer is $y^2 = x^3 + 12x + 14$).

Exercise 5.9 (Tate's theorem). Show that if $\phi : E_1 \to E_2$ is an isogeny over $\mathbb{F}_q$ then $\#E_1(\mathbb{F}_q) = \#E_2(\mathbb{F}_q)$. (Hint: $\phi\pi_1 = \pi_2\phi$, so the characteristic polynomials of Frobenius coincide since ϕ induces an isomorphism on the ℓ-adic Tate modules.)

Exercise 5.10 (Shor and ECC). Estimate the number of logical qubits needed for ECDLP on secp256k1 using the formula above and compare with the $\approx 2n + 3$ (about 4100 for $n = 2048$) logical qubits for factoring an n-bit RSA modulus in the best known circuits. Which is the cheaper quantum target? Discuss the role of error correction overhead.

Exercise 5.11 (Essay / seminar topic). Choose one: (a) The Castryck–Decru attack in detail (Kani's lemma); (b) the exTNFS and the re-parametrisation of pairing curves; (c) Dual_EC and NUMS curves: how can one demonstrably avoid trapdoors in standards?

Appendix A

Proof of Associativity via Divisors

We prove Theorem 1.17. We use the following standard fact.

Theorem A.1 (Riemann–Roch for genus 1)**.** *For a divisor D on E put $\ell(D) = \dim_{\overline{K}}\{f \in \overline{K}(E) : \operatorname{div}(f) + D \geq 0\} \cup \{0\}$. If $\deg D > 0$ then $\ell(D) = \deg D$. Also $\ell(0) = 1$ (only constants have no poles) and $\ell(D) = 0$ if $\deg D < 0$.*

Proof of Theorem 1.17. Step 1: the map $\psi : E \to \operatorname{Pic}^0(E)$, $P \mapsto [(P) - (\mathcal{O})]$ is additive. Let $P, Q \in E$, let $L :\ \lambda x + \mu y + \nu = 0$ be the line through P, Q (tangent if $P = Q$), meeting E in a third point R, and let V be the vertical line through R, which meets E in R, $P + Q$ and $\mathcal{O}$ (by definition of $+$). By Theorem 5.2,

$$\operatorname{div}\left(\frac{L}{V}\right) = \big[(P) + (Q) + (R) - 3(\mathcal{O})\big] - \big[(R) + (P + Q) - 2(\mathcal{O})\big] = (P) + (Q) - (P + Q) - (\mathcal{O}).$$

(Degenerate cases are handled by counting multiplicities: $P = Q$ tangent, $Q = -P$ where $L = V$ is vertical and $P + Q = \mathcal{O}$.) Hence $\big[(P + Q) - (\mathcal{O})\big] = \big[(P) - (\mathcal{O})\big] + \big[(Q) - (\mathcal{O})\big]$ in $\operatorname{Pic}^0(E)$.

Step 2: ψ is injective. If $(P) - (\mathcal{O}) \sim (Q) - (\mathcal{O})$ with $P \neq Q$, there is f with $\operatorname{div} f = (P) - (Q)$, so $f \in L((Q))$ is non-constant; but $\ell((Q)) = 1$ by Riemann–Roch, so $L((Q))$ contains only constants. Contradiction.

Step 3: ψ is surjective. Let D have degree 0. Then $\deg(D + (\mathcal{O})) = 1$ and $\ell(D + (\mathcal{O})) = 1$, so there is $f \neq 0$ with $\operatorname{div} f + D + (\mathcal{O}) \geq 0$ of degree 1, hence equal to (P) for some P. Therefore $D \sim (P) - (\mathcal{O})$.

Step 4. ψ is thus a bijection transporting the chord–tangent operation to the group law of $\operatorname{Pic}^0(E)$, which is an abelian group (it is a quotient of the free abelian group Div^0). Hence $(E, +)$ is an abelian group: for instance $\psi((P+Q)+R) = (\psi P+\psi Q)+\psi R = \psi P+(\psi Q+\psi R) = \psi(P+(Q+R))$ and ψ is injective. The identity is $\mathcal{O}$ and $-P$ is the reflection of P. □

(The same argument also proves Theorem 5.3(ii)–(iii).)

Appendix B

Self-Contained Python Code

The following educational library implements affine arithmetic, double-and-add, ECDSA and the X25519 ladder. It is *not* constant-time and must not be used to protect real data. Running it reproduces the tables of this text, verifies the secp256k1 generator and the RFC 7748 test vector quoted in Lecture 4.

```python
"""Educational elliptic-curve library (NOT constant-time, NOT for production)."""
import hashlib, secrets

class Curve:
    """Short Weierstrass curve y^2 = x^3 + a x + b over F_p, p > 3 prime."""
    def __init__(self, p, a, b, G=None, n=None):
        self.p, self.a, self.b, self.G, self.n = p, a, b, G, n
        assert (4*a**3 + 27*b**2) % p != 0, "singular curve"

    def on_curve(self, P):
        if P is None: return True # None encodes the point at infinity O
        x, y = P
        return (y*y - x**3 - self.a*x - self.b) % self.p == 0

    def neg(self, P):
        return None if P is None else (P[0], (-P[1]) % self.p)

    def add(self, P, Q):
        p = self.p
        if P is None: return Q
        if Q is None: return P
        (x1, y1), (x2, y2) = P, Q
        if x1 == x2 and (y1 + y2) % p == 0: # Q = -P
            return None
        if P == Q: # tangent
            lam = (3*x1*x1 + self.a) * pow(2*y1, -1, p) % p
        else: # chord
            lam = (y2 - y1) * pow(x2 - x1, -1, p) % p
        x3 = (lam*lam - x1 - x2) % p
        y3 = (lam*(x1 - x3) - y1) % p
        return (x3, y3)

    def mul(self, k, P):
        """Left-to-right double-and-add."""
        if k < 0: return self.mul(-k, self.neg(P))
        R = None
        for bit in bin(k)[2:]:
            R = self.add(R, R)
```

```
            if bit == '1': R = self.add(R, P)
        return R

    def order_naive(self, P):
        k, Q = 1, P
        while Q is not None:
            Q, k = self.add(Q, P), k + 1
        return k

def H(msg, n): # hash to integer mod n
    return int.from_bytes(hashlib.sha256(msg).digest(), 'big') % n

def keygen(C):
    d = secrets.randbelow(C.n - 1) + 1
    return d, C.mul(d, C.G)

def ecdsa_sign(C, d, msg):
    e = H(msg, C.n)
    while True:
        k = secrets.randbelow(C.n - 1) + 1
        r = C.mul(k, C.G)[0] % C.n
        if r == 0: continue
        s = pow(k, -1, C.n) * (e + r*d) % C.n
        if s != 0: return (r, s)

def ecdsa_verify(C, Q, msg, sig):
    r, s = sig
    if not (0 < r < C.n and 0 < s < C.n): return False
    e, w = H(msg, C.n), pow(s, -1, C.n)
    X = C.add(C.mul(e*w % C.n, C.G), C.mul(r*w % C.n, Q))
    return X is not None and X[0] % C.n == r

def x25519(k_bytes, u_bytes):
    """Montgomery ladder as in RFC 7748."""
    p = 2**255 - 19; a24 = 121665
    k = bytearray(k_bytes); k[0] &= 248; k[31] &= 127; k[31] |= 64
    k = int.from_bytes(k, 'little')
    x1 = int.from_bytes(u_bytes, 'little') & ((1 << 255) - 1)
    x2, z2, x3, z3, swap = 1, 0, x1, 1, 0
    for t in reversed(range(255)):
        kt = (k >> t) & 1
        swap ^= kt
        if swap: x2, x3, z2, z3 = x3, x2, z3, z2
        swap = kt
        A = (x2 + z2) % p; AA = A*A % p; B = (x2 - z2) % p; BB = B*B % p
        E = (AA - BB) % p; C = (x3 + z3) % p; D = (x3 - z3) % p
        DA = D*A % p; CB = C*B % p
        x3 = (DA + CB)**2 % p; z3 = x1 * (DA - CB)**2 % p
        x2 = AA*BB % p; z2 = E*(AA + a24*E) % p
    if swap: x2, x3, z2, z3 = x3, x2, z3, z2
    return (x2 * pow(z2, p-2, p) % p).to_bytes(32, 'little')

if __name__ == "__main__":
    E17 = Curve(17, 2, 2, G=(5, 1), n=19)
    print([E17.mul(k, E17.G) for k in (1, 2, 3, 19)])
    d, Q = keygen(E17); sig = ecdsa_sign(E17, d, b"hello")
    print("toy ECDSA:", ecdsa_verify(E17, Q, b"hello", sig))
    p = 2**256 - 2**32 - 977
    n = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141
```

```
    G = (0x79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798,
         0x483ADA7726A3C4655DA4FBFC0E1108A8FD17B448A68554199C47D08FFB10D4B8)
    S = Curve(p, 0, 7, G, n)
    print("secp256k1: G on curve:", S.on_curve(G), " nG = O:", S.mul(n, G) is None)
    d, Q = keygen(S); sig = ecdsa_sign(S, d, b"hello")
    print("secp256k1 ECDSA:", ecdsa_verify(S, Q, b"hello", sig), ecdsa_verify(S, Q, b"hellp",
        sig))
    sk = bytes.fromhex("77076d0a7318a57d3c16c17251b26645df4c2f87ebc0992ab177fba51db92c2a")
    base = (9).to_bytes(32, 'little')
    print("X25519 pub:", x25519(sk, base).hex())
```

Checking the BLS12-381 parameters and Pohlig–Hellman

```
# BLS12-381: u, r(u), q(u), trace, cofactor, embedding degree
u = -0xd201000000010000
r = u**4 - u**2 + 1
q = (u - 1)**2 * r // 3 + u
assert (u - 1)**2 * r % 3 == 0
print(q.bit_length(), r.bit_length()) # 381 255
print(pow(2, q-1, q) == 1, pow(2, r-1, r) == 1) # Fermat test: True True
t = u + 1
assert (q + 1 - t) % r == 0 # r divides #E(F_q)
k = 1
while (q**k - 1) % r: k += 1
print("embedding degree k =", k) # 12

# Pohlig-Hellman, E: y^2 = x^3 + x + 1 over F_5, P = (0,1) of order 9
from ecc_toy import Curve
E = Curve(5, 1, 1)
P = (0, 1); Q = E.mul(7, P)
k0 = next(i for i in range(3) if E.mul(3, Q) == E.mul(i, E.mul(3, P)))
Q1 = E.add(Q, E.neg(E.mul(k0, P)))
k1 = next(i for i in range(3) if Q1 == E.mul(i, E.mul(3, P)))
print(k0 + 3*k1) # 7
```

A toy Tate pairing (used in Example 5.11)

```
p=131; r=11
class F2:
    def __init__(s,a,b=0): s.a=a%p; s.b=b%p
    def __add__(s,o): o=o if isinstance(o,F2) else F2(o); return F2(s.a+o.a,s.b+o.b)
    def __sub__(s,o): o=o if isinstance(o,F2) else F2(o); return F2(s.a-o.a,s.b-o.b)
    def __mul__(s,o): o=o if isinstance(o,F2) else F2(o); return F2(s.a*o.a-s.b*o.b,s.a*o.b+s.
        b*o.a)
    def inv(s):
        n=pow(s.a*s.a+s.b*s.b,-1,p); return F2(s.a*n,-s.b*n)
    def __pow__(s,e):
        R=F2(1);B=s
        while e:
            if e&1:R=R*B
            B=B*B;e>>=1
        return R
    def __eq__(s,o): o=o if isinstance(o,F2) else F2(o); return s.a==o.a and s.b==o.b
    def __repr__(s): return f"{s.a}+{s.b}i"
```

```
    def __hash__(s): return hash((s.a,s.b))
# curve y^2=x^3+x over F_p
def add(P,Q):
    if P is None:return Q
    if Q is None:return P
    x1,y1=P;x2,y2=Q
    if x1==x2 and (y1+y2)%p==0:return None
    l=(3*x1*x1+1)*pow(2*y1,-1,p)%p if P==Q else (y2-y1)*pow(x2-x1,-1,p)%p
    x3=(l*l-x1-x2)%p;return (x3,(l*(x1-x3)-y1)%p)
def mul(k,P):
    R=None
    for _ in range(k):R=add(R,P)
    return R
pts=[(x,y) for x in range(p) for y in range(p) if (y*y-x**3-x)%p==0]
print("order",len(pts)+1)
# point of order 11: multiply by cofactor 12
for P in pts:
    P11=mul(12,P)
    if P11 is not None: break
print("P",P11, mul(11,P11))
P=P11
def dist(Q): return (F2(-Q[0]),F2(0,Q[1])) # (x,y)->(-x, i y)
def miller(P,S):
    # f_{r,P}(S), denominators dropped (eliminated by final exp)
    T=P;f=F2(1)
    bits=bin(r)[3:]
    for b in bits:
        # doubling: line tangent at T evaluated at S
        x,y=T;l=(3*x*x+1)*pow(2*y,-1,p)%p
        line=lambda S,x=x,y=y,l=l:(S[1]-F2(y)-F2(l)*(S[0]-F2(x)))
        f=f*f*line(S); T=add(T,T)
        if b=='1':
            x1,y1=T;x2,y2=P
            if x1==x2: # vertical, T=-P : skip line (value in F_p)
                T=add(T,P);continue
            l=(y2-y1)*pow(x2-x1,-1,p)%p
            f=f*(S[1]-F2(y1)-F2(l)*(S[0]-F2(x1))); T=add(T,P)
    return f
def e(P,Q):
    S=dist(Q)
    return miller(P,S)**((p*p-1)//r)
z=e(P,P);print("e(P,P)=",z,"z^11",z**11)
for a,b in [(3,5),(2,1),(7,4)]:
    print(a,b,e(mul(a,P),mul(b,P)),z**(a*b%r), e(mul(a,P),mul(b,P))==z**(a*b%r))
# toy BLS: x=7, M=3P, sigma=7M
x=7;M=mul(3,P);sig=mul(x,M);X=mul(x,P)
print("BLS",M,sig,X,e(sig,P),e(M,X),e(sig,P)==e(M,X))
print("powers of z",[ (k,z**k) for k in range(1,12)])
```

Appendix C

SageMath Cheat Sheet

The following commands (SageMath) are useful for experiments.

```
E = EllipticCurve(GF(17), [2, 2]) # y^2 = x^3 + 2x + 2
E.cardinality(), E.abelian_group() # 19, cyclic
G = E(5, 1); G.order() # 19
2*G, 3*G # (6 : 3 : 1), (10 : 6 : 1)
E.trace_of_frobenius(), E.frobenius_polynomial()
E.is_supersingular(), E.j_invariant(), E.discriminant()
Q = 6*G; discrete_log(Q, G, G.order(), operation='+') # 6

p = 2**255 - 19 # Curve25519 (Montgomery form)
M = EllipticCurve(GF(p), [0, 486662, 0, 1, 0])
M.cardinality() // 8 # prime order l (takes a while)

E2 = EllipticCurve(GF(17), [2, 8]) # order 21
P7 = E2(6, 7); P7.order() # 7
E2.isogeny(P7).codomain() # 7-isogenous curve (Velu)
P, Q = E2.gens() if hasattr(E2, 'gens') else (None, None)
# Weil pairing, Tate pairing: P.weil_pairing(Q, n), P.tate_pairing(Q, n, k)
# Pairing-friendly constructions: sage.schemes.elliptic_curves (CM_orders,
    hilbert_class_polynomial)
```

Appendix D

One-Page Formula Sheet

Short Weierstrass	$y^2 = x^3 + ax + b$, $\Delta = -16(4a^3 + 27b^2) \neq 0$, $j = 1728\,\frac{4a^3}{4a^3+27b^2}$
Addition	$\lambda = \frac{y_2 - y_1}{x_2 - x_1}$ (or $\frac{3x_1^2 + a}{2y_1}$), $x_3 = \lambda^2 - x_1 - x_2$, $y_3 = \lambda(x_1 - x_3) - y_1$
Negation	$-(x, y) = (x, -y)$
Hasse	$\lvert \#E(\mathbb{F}_q) - q - 1 \rvert \leq 2\sqrt{q}$, $\#E = q + 1 - t$
Frobenius	$\pi^2 - t\pi + q = 0$, $\#E(\mathbb{F}_{q^m}) = q^m + 1 - \alpha^m - \beta^m$
Structure	$E(\mathbb{F}_q) \cong \mathbb{Z}/n_1 \times \mathbb{Z}/n_2$, $n_1 \mid n_2$, $n_1 \mid q - 1$
Rho / BSGS cost	$\sqrt{\pi r/2}$ (rho, $\sqrt{\pi r/4}$ with negation), $\sqrt{r}$ (BSGS)
ECDSA	$s = k^{-1}(e + \rho d)$; verify $x(s^{-1}eG + s^{-1}\rho Q) \equiv \rho$
Schnorr	$s = k + cd$; verify $sG = R + cQ$
Weil pairing	$e_n(aP, bQ) = e_n(P, Q)^{ab}$, $e_n(P, P) = 1$
Embedding degree	least k with $r \mid q^k - 1$
Montgomery curve	$By^2 = x^3 + Ax^2 + x$; Edwards: $ax^2 + y^2 = 1 + dx^2y^2$, $a = \frac{A+2}{B}$, $d = \frac{A-2}{B}$
Edwards addition	$\left(\frac{x_1y_2 + x_2y_1}{1 + dx_1x_2y_1y_2}, \frac{y_1y_2 - ax_1x_2}{1 - dx_1x_2y_1y_2}\right)$

Bibliography

[1] J. H. Silverman, *The Arithmetic of Elliptic Curves*, 2nd ed., Springer GTM 106, 2009. (Mathematical reference.)

[2] L. C. Washington, *Elliptic Curves: Number Theory and Cryptography*, 2nd ed., CRC Press, 2008.

[3] D. Hankerson, A. Menezes, S. Vanstone, *Guide to Elliptic Curve Cryptography*, Springer, 2004.

[4] S. D. Galbraith, *Mathematics of Public Key Cryptography*, Cambridge University Press, 2012.

[5] I. Blake, G. Seroussi, N. Smart, *Elliptic Curves in Cryptography*, LMS Lecture Notes 265, 1999.

[6] H. Cohen, G. Frey (eds.), *Handbook of Elliptic and Hyperelliptic Curve Cryptography*, CRC Press, 2005.

[7] A. Sutherland, MIT 18.783 *Elliptic Curves* lecture notes (open course material).

[8] W. Diffie, M. Hellman, New directions in cryptography, *IEEE Trans. Inform. Theory* 22 (1976).

[9] N. Koblitz, Elliptic curve cryptosystems, *Math. Comp.* 48 (1987).

[10] V. Miller, Use of elliptic curves in cryptography, CRYPTO 1985.

[11] H. W. Lenstra Jr., Factoring integers with elliptic curves, *Ann. of Math.* 126 (1987).

[12] R. Schoof, Elliptic curves over finite fields and the computation of square roots mod p, *Math. Comp.* 44 (1985).

[13] S. Pohlig, M. Hellman, An improved algorithm for computing logarithms over $GF(p)$, *IEEE Trans. Inform. Theory* 24 (1978).

[14] J. M. Pollard, Monte Carlo methods for index computation mod p, *Math. Comp.* 32 (1978).

[15] P. van Oorschot, M. Wiener, Parallel collision search with cryptanalytic applications, *J. Cryptology* 12 (1999).

[16] V. Shoup, Lower bounds for discrete logarithms and related problems, EUROCRYPT 1997.

[17] A. Menezes, T. Okamoto, S. Vanstone, Reducing elliptic curve logarithms to logarithms in a finite field, *IEEE Trans. Inform. Theory* 39 (1993).

[18] G. Frey, H.-G. Rück, A remark concerning m-divisibility and the discrete logarithm in the divisor class group of curves, *Math. Comp.* 62 (1994).

[19] N. Smart, The discrete logarithm problem on elliptic curves of trace one, *J. Cryptology* 12 (1999). (Also Satoh–Araki 1998; Semaev 1998.)

[20] I. Semaev, Summation polynomials and the discrete logarithm problem on elliptic curves, ePrint 2004/031.

[21] P. Gaudry, Index calculus for abelian varieties of small dimension and the elliptic curve discrete logarithm problem, *J. Symb. Comput.* 44 (2009); C. Diem, On the discrete logarithm problem in elliptic curves, *Compositio* 147 (2011).

[22] P. Gaudry, F. Hess, N. Smart, Constructive and destructive facets of Weil descent on elliptic curves, *J. Cryptology* 15 (2002).

[23] R. Gallant, R. Lambert, S. Vanstone, Faster point multiplication on elliptic curves with efficient endomorphisms, CRYPTO 2001.

[24] D. J. Bernstein, Curve25519: new Diffie–Hellman speed records, PKC 2006.

[25] H. Edwards, A normal form for elliptic curves, *Bull. AMS* 44 (2007); D. J. Bernstein, T. Lange, Faster addition and doubling on elliptic curves, ASIACRYPT 2007.

[26] D. J. Bernstein, P. Birkner, M. Joye, T. Lange, C. Peters, Twisted Edwards curves, AFRICACRYPT 2008.

[27] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, B.-Y. Yang, High-speed high-security signatures, *J. Cryptographic Eng.* 2 (2012).

[28] D. J. Bernstein, T. Lange, SafeCurves: choosing safe curves for elliptic-curve cryptography, https://safecurves.cr.yp.to.

[29] J. Renes, C. Costello, L. Batina, Complete addition formulas for prime order elliptic curves, EUROCRYPT 2016.

[30] I. Biehl, B. Meyer, V. Müller, Differential fault attacks on elliptic curve cryptosystems, CRYPTO 2000; Antipa et al., Validation of elliptic curve public keys, PKC 2003.

[31] D. Johnson, A. Menezes, S. Vanstone, The elliptic curve digital signature algorithm (ECDSA), *Int. J. Inf. Secur.* 1 (2001).

[32] D. Pointcheval, J. Stern, Security arguments for digital signatures and blind signatures, *J. Cryptology* 13 (2000).

[33] D. Boneh, R. Venkatesan, Hardness of computing the most significant bits of secret keys in Diffie–Hellman and related schemes, CRYPTO 1996; N. Howgrave-Graham, N. Smart, Lattice attacks on digital signature schemes, *Des. Codes Cryptogr.* 23 (2001).

[34] D. Shumow, N. Ferguson, On the possibility of a back door in the NIST SP800-90 Dual_Ec_Prng, CRYPTO 2007 rump session.

[35] A. Joux, A one round protocol for tripartite Diffie–Hellman, ANTS-IV 2000.

[36] D. Boneh, M. Franklin, Identity-based encryption from the Weil pairing, CRYPTO 2001.

[37] D. Boneh, B. Lynn, H. Shacham, Short signatures from the Weil pairing, ASIACRYPT 2001.

[38] V. Miller, The Weil pairing, and its efficient calculation, *J. Cryptology* 17 (2004).

[39] P. Barreto, M. Naehrig, Pairing-friendly elliptic curves of prime order, SAC 2005; S. Bowe, BLS12-381: new zk-SNARK elliptic curve construction (Zcash blog), 2017.

[40] D. Freeman, M. Scott, E. Teske, A taxonomy of pairing-friendly elliptic curves, *J. Cryptology* 23 (2010).

[41] T. Kim, R. Barbulescu, Extended tower number field sieve: a new complexity for the medium prime case, CRYPTO 2016.

[42] J. Vélu, Isogénies entre courbes elliptiques, *C. R. Acad. Sci. Paris* 273 (1971).

[43] D. Charles, E. Goren, K. Lauter, Cryptographic hash functions from expander graphs, *J. Cryptology* 22 (2009).

[44] D. Jao, L. De Feo, Towards quantum-resistant cryptosystems from supersingular elliptic curve isogenies, PQCrypto 2011.

[45] W. Castryck, T. Decru, An efficient key recovery attack on SIDH, EUROCRYPT 2023; L. Maino, C. Martindale, et al.; D. Robert, Breaking SIDH in polynomial time, EUROCRYPT 2023.

[46] W. Castryck, T. Lange, C. Martindale, L. Panny, J. Renes, CSIDH: an efficient post-quantum commutative group action, ASIACRYPT 2018.

[47] L. De Feo, D. Kohel, A. Leroux, C. Petit, B. Wesolowski, SQISign: compact post-quantum signatures from quaternions and isogenies, ASIACRYPT 2020.

[48] P. Shor, Algorithms for quantum computation: discrete logarithms and factoring, FOCS 1994.

[49] M. Roetteler, M. Naehrig, K. Svore, K. Lauter, Quantum resource estimates for computing elliptic curve discrete logarithms, ASIACRYPT 2017.

[50] IETF RFC 6979 (deterministic DSA/ECDSA), RFC 7748 (X25519/X448), RFC 8032 (EdDSA), RFC 9380 (hashing to elliptic curves), RFC 9180 (HPKE); NIST FIPS 186-5, SP 800-186, SP 800-57; SEC 1 and SEC 2 (Certicom/SECG).